

__shfl_up_sync(): Copy from a lane with a lower ID than the caller's.

The intrinsic function calculates a source lane ID by subtracting `delta` from the caller's lane ID. The value of `value` held by the resulting lane ID is returned: in effect, `value` is shifted up the warp by `delta` lanes.

- ▶ If `width` is less than `warpSize`, then each subsection of the warp behaves as a separate entity with a starting logical lane ID of 0.
- ▶ The source lane index will not wrap around the value of `width`, so the lower `delta` lanes will remain unchanged.

__shfl_down_sync(): Copy from a lane with a higher ID than the caller's.

The intrinsic function calculates a source lane ID by adding `delta` to the caller's lane ID. The value of `value` held by the resulting lane ID is returned: this has the effect of shifting `value` down the warp by `delta` lanes.

- ▶ If `width` is less than `warpSize`, then each subsection of the warp behaves as a separate entity with a starting logical lane ID of 0.
- ▶ As for `__shfl_up_sync()`, the ID number of the source lane will not wrap around the value of `width` and so the upper `delta` lanes will effectively remain unchanged.

__shfl_xor_sync(): Copy from a lane based on bitwise XOR of own lane ID.

The intrinsic function calculates a source lane ID by performing a bitwise XOR of the caller's lane ID and `laneMask`: the value of `value` held by the resulting lane ID is returned. This mode implements a butterfly addressing pattern, which is used in tree reduction and broadcast.

- ▶ If `width` is less than `warpSize`, then each group of `width` consecutive threads are able to access elements from earlier groups. However, if they attempt to access elements from later groups of threads their own value of `value` will be returned.

T can be:

- ▶ `int`, `unsigned`, `long`, `unsigned long`, `long long`, `unsigned long long`, `float` or `double`.
- ▶ `__half` and `__half2` with the `cuda_fp16.h` header included.
- ▶ `__nv_bfloat16` and `__nv_bfloat162` with the `cuda_bf16.h` header included.

Threads may only read data from another thread that is actively participating in the intrinsics. If the target thread is *inactive*, the retrieved value is undefined.

`width` must be a power of two in the range `[1, warpSize]`, namely 1, 2, 4, 8, 16, or 32. Other values will produce undefined results.

The functions are subject to the *Warp __sync Intrinsic Constraints*.

Examples of valid warp shuffle usage:

```
int laneId = threadIdx.x % warpSize;
int data   = ...

// all warp threads get 'data' from lane 0
```

(continues on next page)

(continued from previous page)

```

int result1 = __shfl_sync(0xFFFFFFFF, data, 0);

if (laneId < 4) {
    // lanes 0, 1, 2, 3 get 'data' from lane 1
    int result2 = __shfl_sync(0xb1111, data, 1);
}

// lanes [0 - 15] get 'data' from lane 0
// lanes [16 - 31] get 'data' from lane 16
int result3 = __shfl_sync(0xFFFFFFFF, value, warpSize / 2);

// each lane gets 'data' from the lane two positions above
// lanes 30, 31 get their original value
int result4 = __shfl_down_sync(0xFFFFFFFF, data, 2);

```

Examples of invalid warp shuffle usage:

```

int laneId = threadIdx.x % warpSize;
int value = ...
// undefined behavior: lane 0 does not participate in the call
int result = (laneId > 0) ? __shfl_sync(0xFFFFFFFF, value, 0) : 0;

if (laneId <= 4) {
    // undefined behavior: destination lanes 5, 6 are not active for lanes 3, 4
    result = __shfl_down_sync(0xb1111, value, 2);
}

// undefined behavior: width is not a power of 2
__shfl_sync(0xFFFFFFFF, value, 0, /*width=*/31);

```

Warning

These intrinsics do not imply a memory barrier. They do not guarantee any memory ordering.

Example 1: Broadcast of a single value across a warp

CUDA C++

```

#include <cassert>
#include <cuda/warp>

__global__ void warp_broadcast_kernel(int input) {
    int laneId = threadIdx.x % 32;
    int value;
    if (laneId == 0) { // unused variable for all threads except lane 0
        value = input;
    }
    value = cuda::device::warp_shuffle_idx(value, 0); // Synchronize all
    ↪ threads in warp, and get "value" from lane 0
    assert(value == input);
}

```

(continues on next page)

(continued from previous page)

```

}

int main() {
    warp_broadcast_kernel<<<1, 32>>>(1234);
    cudaDeviceSynchronize();
    return 0;
}

```

Intrinsics

```

#include <assert.h>

__global__ void warp_broadcast_kernel(int input) {
    int laneId = threadIdx.x % 32;
    int value;
    if (laneId == 0) { // unused variable for all threads except lane 0
        value = input;
    }
    value = __shfl_sync(0xFFFFFFFF, value, 0); // Synchronize all threads in
    →warp, and get "value" from lane 0
    assert(value == input);
}

int main() {
    warp_broadcast_kernel<<<1, 32>>>(1234);
    cudaDeviceSynchronize();
    return 0;
}

```

See the example on [Compiler Explorer](#).

Example 2: Inclusive plus-scan across sub-partitions of 8 threads

Hint

It is suggested to use the `cub::WarpScan` function for efficient and generalized warp scan functions.

CUDA C++

```

#include <cstdio>
#include <cub/cub.cuh>

__global__ void scan_sub_partition_with_8_threads_kernel() {
    using WarpScan = cub::WarpScan<int, 8>;
    using TempStorage = typename WarpScan::TempStorage;
    __shared__ TempStorage temp_storage;

    int laneId = threadIdx.x % 32;
    int value = 31 - laneId; // starting value to accumulate
    int partial_sum;

```

(continues on next page)

(continued from previous page)

```

    WarpScan(temp_storage).InclusiveSum(value, partial_sum);
    printf("Thread %d final value = %d\n", threadIdx.x, partial_sum);
}

int main() {
    scan_sub_partition_with_8_threads_kernel<<<1, 32>>>();
    cudaDeviceSynchronize();
    return 0;
}

```

Intrinsics

```

#include <stdio.h>

__global__ void scan_sub_partition_with_8_threads_kernel() {
    int laneId = threadIdx.x % 32;
    int value = 31 - laneId; // starting value to accumulate
    // Loop to accumulate scan within my partition.
    // Scan requires log2(8) == 3 steps for 8 threads
    for (int delta = 1; delta <= 4; delta *= 2) {
        int tmp = __shfl_up_sync(0xFFFFFFFF, value, delta, /*width=*/
→8); // read from laneId - delta
        int source_lane = laneId % 8 - delta;
        if (source_lane >= 0) // lanes with 'source_lane < 0' have their value
→unchanged
            value += tmp;
    }
    printf("Thread %d final value = %d\n", threadIdx.x, value);
}

int main() {
    scan_sub_partition_with_8_threads_kernel<<<1, 32>>>();
    cudaDeviceSynchronize();
    return 0;
}

```

See the example on [Compiler Explorer](#).

Example 3: Reduction across a warp

Hint

It is suggested to use the `cub::WarpReduce` function for efficient and generalized warp reduction functions.

CUDA C++

```

#include <cstdio>
#include <cub/cub.cuh>
#include <cuda/warp>

```

(continues on next page)

(continued from previous page)

```

__global__ void warp_reduce_kernel() {
    using WarpReduce = cub::WarpReduce<int>;
    using TempStorage = typename WarpReduce::TempStorage;
    __shared__ TempStorage temp_storage;

    int laneId      = threadIdx.x % 32;
    int value        = 31 - laneId; // starting value to accumulate
    auto aggregate = WarpReduce(temp_storage).Sum(value);
    aggregate        = cuda::device::warp_shuffle_idx(aggregate, 0);
    printf("Thread %d final value = %d\n", threadIdx.x, aggregate);
}

int main() {
    warp_reduce_kernel<<<1, 32>>>>();
    cudaDeviceSynchronize();
    return 0;
}

```

Intrinsics

```

#include <stdio.h>

__global__ void warp_reduce_kernel() {
    int laneId = threadIdx.x % 32;
    int value = 31 - laneId; // starting value to accumulate
    // Use XOR mode to perform butterfly reduction
    // A full-warp reduction requires log2(32) == 5 steps
    for (int i = 1; i <= 16; i *= 2)
        value += __shfl_xor_sync(0xFFFFFFFF, value, i);
    // "value" now contains the sum across all threads
    printf("Thread %d final value = %d\n", threadIdx.x, value);
}

int main() {
    warp_reduce_kernel<<<1, 32>>>>();
    cudaDeviceSynchronize();
    return 0;
}

```

See the example on [Compiler Explorer](#).

5.4.6.6 Warp __sync Intrinsic Constraints

All warp __sync intrinsics, such as:

- ▶ __shfl_sync, __shfl_up_sync, __shfl_down_sync, __shfl_xor_sync
- ▶ __match_any_sync, __match_all_sync
- ▶ __reduce_add_sync, __reduce_min_sync, __reduce_max_sync, __reduce_and_sync, __reduce_or_sync, __reduce_xor_sync
- ▶ __syncwarp

use the `mask` parameter to indicate which warp threads participate in the call. This parameter ensures proper convergence before the hardware executes the intrinsic.

Each bit in the `mask` corresponds to a thread's lane ID (`threadIdx.x % warpSize`). The intrinsic waits until all non-exited warp threads specified in the `mask` reach the call.

The following constraints must be met for correct execution:

- ▶ Each calling thread must have its corresponding bit set in the `mask`.
- ▶ Each non-calling thread must have its corresponding bit set to zero in the `mask`. Exited threads are ignored.
- ▶ All non-exited threads specified in the `mask` must execute the intrinsic with the same `mask` value.
- ▶ Warp threads may call the intrinsic concurrently with different `mask` values, provided the masks are disjoint. Such condition is valid even in divergent control flow.

The behavior of warp `__sync` functions is invalid, such as kernel hang, or undefined if:

- ▶ A calling thread is not specified in the `mask`.
- ▶ A non-exited thread specified in the `mask` fails to either eventually exit or call the intrinsic at the same program point with the same `mask` value.
- ▶ In conditional code, all conditions must evaluate identically across all non-exited threads specified in the `mask`.

Note

The intrinsics achieve the best efficiency when all warp threads participate in the call, namely when the `mask` is set to `0xFFFFFFFF`.

Examples of valid warp intrinsics usage:

```
__global__ void valid_examples() {
    if (threadIdx.x < 4) {           // threads 0, 1, 2, 3 are active
        __all_sync(0b1111, pred); // CORRECT, threads 0, 1, 2, 3 participate
    }                               in the call

    if (threadIdx.x == 0)
        return; // exit
    // CORRECT, all non-exited threads participate in the call
    __all_sync(0xFFFFFFFF, pred);
}
```

Disjoint mask examples:

```
__global__ void example_syncwarp_with_mask(int* input_data, int* output_data)
{
    if (threadIdx.x < warpSize) {
        __shared__ int shared_data[warpSize];
        shared_data[threadIdx.x] = input_data[threadIdx.x];

        unsigned mask = threadIdx.x < 16 ? 0xFFFF : 0xFFFF0000; // CORRECT
    }
}
```

(continues on next page)

(continued from previous page)

```

    __syncwarp(mask);
    if (threadIdx.x == 0 || threadIdx.x == 16)
        output_data[threadIdx.x] = shared_data[threadIdx.x + 1];
}
}

__global__ void example_syncwarp_with_mask_branches(int* input_data, int*
→output_data) {
    if (threadIdx.x < warpSize) {
        __shared__ int shared_data[warpSize];
        shared_data[threadIdx.x] = input_data[threadIdx.x];

        if (threadIdx.x < 16) {
            unsigned mask = 0xFFFF; // CORRECT
            __syncwarp(mask);
            output_data[threadIdx.x] = shared_data[15 - threadIdx.x];
        }
        else {
            unsigned mask = 0xFFFF0000; // CORRECT
            __syncwarp(mask);
            output_data[threadIdx.x] = shared_data[31 - threadIdx.x];
        }
    }
}
}

```

Examples of invalid warp intrinsics usage:

```

if (threadIdx.x < 4) { // threads 0, 1, 2, 3 are active
    __all_sync(0b00000011, pred); // WRONG, threads 2, 3 are active but not set
→in mask
    __all_sync(0b11111111, pred); // WRONG, threads 4, 5, 6 are not active but
→set in mask
}

// WRONG, participating threads have a different and overlapping mask
__all_sync(threadIdx.x == 0 ? 1 : 0xFFFFFFFF, pred);

```

5.4.7. CUDA-Specific Macros

5.4.7.1 __CUDA_ARCH__

The macro `__CUDA_ARCH__` represents the **virtual architecture** of the NVIDIA GPU for which the code is being compiled. Its value may differ from the device's actual compute capability. This macro enables the writing of code paths that are specialized for particular GPU architectures, which may be necessary for optimal performance or to use architecture-specific features and instructions. The macro can also be used to distinguish between host and device code.

`__CUDA_ARCH__` is only defined in device code, namely in the `__device__`, `__host__`, `__device__`, and `__global__` functions. The value of the macro is associated with the nvcc option `compute_<version>`, with the relation `__CUDA_ARCH__ = <version> * 10`.

Example:

```
nvcc --generate-code arch=compute_80,code=sm_90 prog.cu
```

defines `__CUDA_ARCH__` as 800.

`__CUDA_ARCH__` Constraints

1. The type signatures of the following entities shall not depend on whether `__CUDA_ARCH__` is defined, nor on its value.

- ▶ `__global__` functions and function templates.
- ▶ `__device__` and `__constant__` variables.
- ▶ Textures and surfaces.

Example:

```
#if !defined(__CUDA_ARCH__)
    typedef int my_type;
#else
    typedef double my_type;
#endif

__device__ my_type my_var;           // ERROR: my_var's type depends on __CUDA_
↪ARCH__

__global__ void kernel(my_type in) { // ERROR: kernel's type depends on __CUDA_
↪ARCH__
    ...
}
```

2. Instantiations of `__global__` function templates must not depend on whether `__CUDA_ARCH__` is defined or its value. That is, the same instantiations, with equivalent template arguments, must be present in *all* device programs and the host program (irrespective of whether any of those instantiations are ever launched at runtime).

Example:

```
__device__ int result;

template <typename T>
__global__ void kernel(T in) {
    result = in;
}

__host__ __device__ void host_device_function(void) {
#if !defined(__CUDA_ARCH__)
    kernel<<<1, 1>>>>(1); // ERROR: "kernel<int>" instantiation only
                        //      when __CUDA_ARCH__ is undefined!
#endif
}

int main(void) {
    host_device_function();
    cudaDeviceSynchronize();
}
```

(continues on next page)

(continued from previous page)

```

    return 0;
}

```

This issue can be avoided by instead moving the computation proper from the `__global__` function into a `__device__` function template that is invoked by the former. Within the `__global__` function, `__CUDA_ARCH__` can then be used to conditionally instantiate the `__device__` function template with different arguments.

3. In separate compilation mode, the presence or absence of a function or variable definition with external linkage shall not depend on the definition of `__CUDA_ARCH__` or on its value.

Example:

```

#if !defined(__CUDA_ARCH__)
    void host_function(void) {} // ERROR: The definition of host_function()
                                // is only present when __CUDA_ARCH__
                                // is undefined
#endif

```

4. In separate compilation, the preprocessor macro `__CUDA_ARCH__` must not be used in headers to prevent objects from having different behaviors. Alternatively, all objects must be compiled for the same virtual architecture. If a weak or template function is defined in a header and its behavior depends on `__CUDA_ARCH__`, then instances of that function in different objects could conflict if those objects are compiled for different compute architectures.

For example, if a header file `a.h` contains:

```

template<typename T>
__device__ T* get_ptr() {
    #if __CUDA_ARCH__ == 900
        return nullptr; /* no address */
    #else
        __shared__ T arr[256];
        return arr;
    #endif
}

```

Then if `a.cu` and `b.cu` both include `a.h` and instantiate `get_ptr()` for the same type, and `b.cu` expects a non-NULL address, and compile with:

```

nvcc -arch=compute_70 -dc a.cu
nvcc -arch=compute_80 -dc b.cu
nvcc -arch=sm_80 a.o b.o

```

Only one version of the `get_ptr()` function is used at link time, so the behavior depends on which version is chosen. To avoid this issue, either `a.cu` and `b.cu` must be compiled for the same compute architecture, or `__CUDA_ARCH__` should not be used in the shared header function.

The compiler does not guarantee that a diagnostic will be generated for the unsupported uses of `__CUDA_ARCH__` described above.

5.4.7.2 `__CUDA_ARCH_SPECIFIC__` and `__CUDA_ARCH_FAMILY_SPECIFIC__`

The macros `__CUDA_ARCH_SPECIFIC__` and `__CUDA_ARCH_FAMILY_SPECIFIC__` are defined to identify GPU devices with *architecture-* and *family-* specific features, respectively. See *Feature Set Compiler Targets* section for more information.

Similarly to `__CUDA_ARCH__`, `__CUDA_ARCH_SPECIFIC__` and `__CUDA_ARCH_FAMILY_SPECIFIC__` are only defined in the device code, namely in the `__device__`, `__host__ __device__`, and `__global__` functions. The macros are associated with the nvcc options `compute_<version>a` and `compute_<version>f`.

```
nvcc --generate-code arch=compute_100a,code=sm_100a prog.cu
```

- ▶ `__CUDA_ARCH__` == 1000.
- ▶ `__CUDA_ARCH_SPECIFIC__` == 1000.
- ▶ `__CUDA_ARCH_FAMILY_SPECIFIC__` == 1000.

```
nvcc --generate-code arch=compute_100f,code=sm_103f prog.cu
```

- ▶ `__CUDA_ARCH__` == 1000.
- ▶ `__CUDA_ARCH_FAMILY_SPECIFIC__` == 1000.
- ▶ `__CUDA_ARCH_SPECIFIC__` is not defined.

```
nvcc -arch=sm_100 prog.cu
```

- ▶ `__CUDA_ARCH__` == 1000.
- ▶ `__CUDA_ARCH_FAMILY_SPECIFIC__` is not defined.
- ▶ `__CUDA_ARCH_SPECIFIC__` is not defined.

```
nvcc -arch=sm_100a prog.cu
```

equivalent to:

```
nvcc --generate-code arch=sm_100a,compute_100,compute_100a prog.cu
```

- ▶ `__CUDA_ARCH__` == 1000.
- ▶ `__CUDA_ARCH_FAMILY_SPECIFIC__` is not defined.
- ▶ `__CUDA_ARCH_SPECIFIC__` == 1000 and `__CUDA_ARCH_SPECIFIC__` not defined are both generated.

5.4.7.3 CUDA Feature Testing Macros

nvcc provides the following preprocessor macros for feature testing. The macros are defined when a particular feature is supported by the CUDA front-end compiler.

- ▶ `__CUDACC_DEVICE_ATOMIC_BUILTINS__`: Supports *device atomic compiler builtins*.
- ▶ `__NVCC_DIAG_PRAGMA_SUPPORT__`: Supports *diagnostic control pragmas*.
- ▶ `__CUDACC_EXTENDED_LAMBDA__`: Supports *extended lambdas*. Enabled by `--expt-extended-lambda` or `--extended-lambda` flag.
- ▶ `__CUDACC_RELAXED_CONSTEXPR__`: Support for *relaxed constexpr functions*. Enabled by the `--expt-relaxed-constexpr` flag.

5.4.7.4 `__nv_pure__` Attribute

In C/C++, a pure function has no side effects on its parameters and can access global variables, though it does not modify them.

CUDA provides `__nv_pure__` attribute supported for both host and device functions. The compiler translates `__nv_pure__` to the pure GNU attribute or to the Microsoft Visual Studio `noalias` attribute.

```
__device__ __nv_pure__
int add(int a, int b) {
    return a + b;
}
```

5.4.8. CUDA-Specific Functions

5.4.8.1 Address Space Predicate Functions

Address space predicate functions are used to determine the address space of a pointer.

Hint

It is suggested to use the `cuda::device::is_address_from()` and `cuda::device::is_object_from()` functions provided by `libcu++` as a portable and safer alternative to Address Space Predicate intrinsic functions.

```
__device__ unsigned __isGlobal      (const void* ptr);
__device__ unsigned __isShared      (const void* ptr);
__device__ unsigned __isConstant    (const void* ptr);
__device__ unsigned __isGridConstant(const void* ptr);
__device__ unsigned __isLocal       (const void* ptr);
```

The functions return 1 if `ptr` contains the generic address of an object in the specified address space, 0 otherwise. Their behavior is unspecified if the argument is a NULL pointer.

- ▶ `__isGlobal()`: global memory space.
- ▶ `__isShared()`: shared memory space.
- ▶ `__isConstant()`: constant memory space.
- ▶ `__isGridConstant()`: kernel parameter annotated with `__grid_constant__`.
- ▶ `__isLocal()`: local memory space.

5.4.8.2 Address Space Conversion Functions

CUDA pointers (`T*`) can access objects regardless of where the objects are stored. For example, an `int*` can access `int` objects whether they reside in global or shared memory.

Address space conversion functions are used to convert between generic addresses and addresses in specific address spaces. These functions are useful when the compiler cannot determine a pointer's address space, for example, when crossing translation units or interacting with PTX instructions.

```

__device__ size_t __cvta_generic_to_global (const void* ptr); // PTX: cvta.
↳to.global
__device__ size_t __cvta_generic_to_shared (const void* ptr); // PTX: cvta.
↳to.shared
__device__ size_t __cvta_generic_to_constant(const void* ptr); // PTX: cvta.
↳to.const
__device__ size_t __cvta_generic_to_local (const void* ptr); // PTX: cvta.
↳to.local

```

```

__device__ void* __cvta_global_to_generic (size_t raw_ptr); // PTX: cvta.
↳global
__device__ void* __cvta_shared_to_generic (size_t raw_ptr); // PTX: cvta.
↳shared
__device__ void* __cvta_constant_to_generic(size_t raw_ptr); // PTX: cvta.
↳const
__device__ void* __cvta_local_to_generic (size_t raw_ptr); // PTX: cvta.
↳local

```

As an example of inter-operating with PTX instructions, the `ld.shared.s32 r0, [ptr];` PTX instruction expects `ptr` to refer to the shared memory address space. A CUDA program with an `int*` pointer to an object in `__shared__` memory needs to convert this pointer to the shared address space before passing it to the PTX instruction by calling `__cvta_generic_to_shared` as follows:

```

__shared__ int smem_var;
smem_var = 42;
size_t smem_ptr = __cvta_generic_to_shared(&smem_var);
int output;
asm volatile("ld.shared.s32 %0, [%1];" : "=r"(output) : "l"(smem_ptr) :
↳"memory");
assert(output == 42);

```

A common optimization that exploits these address representations is reducing data structure size by leveraging the fact that the address ranges of shared, local, and constant spaces are smaller than 32 bits, which allows storing 32-bit addresses instead of 64-bit pointers and save registers. Additionally, 32-bit arithmetic is faster than 64-bit arithmetic. To obtain the 32-bit integer representation of these addresses, truncate the 64-bit value to 32 bits by casting from an unsigned 64-bit integer to an unsigned 32-bit integer:

```

__shared__ int smem_var;
uint32_t smem_ptr_32bit = static_cast<uint32_t>(__cvta_generic_to_
↳shared(&smem_var));

```

To recover a generic address from such a 32-bit representation, zero-extend the address back to an unsigned 64-bit integer and then call the corresponding address space conversion function:

```

size_t smem_ptr_64bit = static_cast<size_t>(smem_ptr_32bit); // zero-extend
↳to 64 bits
void* generic_ptr = __cvta_shared_to_generic(smem_ptr_64bit);
assert(generic_ptr == &smem_var);

```

5.4.8.3 Low-Level Load and Store Functions

```
T __ldg(const T* address);
```

The function `__ldg()` performs a read-only L1/Tex cache load. It supports all C++ fundamental types, CUDA vector types (except x3 components), and extended floating-point types, such as `__half`, `__half2`, `__nv_bfloat16`, and `__nv_bfloat162`.

```
T __ldcg(const T* address);
T __ldca(const T* address);
T __ldcs(const T* address);
T __ldlu(const T* address);
T __ldcv(const T* address);
```

The functions perform a load using the cache operator specified in the [PTX ISA](#) guide. They support all C++ fundamental types, CUDA vector types (except x3 components), and extended floating-point types, such as `__half`, `__half2`, `__nv_bfloat16`, and `__nv_bfloat162`.

```
void __stwb(T* address, T value);
void __stcg(T* address, T value);
void __stcs(T* address, T value);
void __stwt(T* address, T value);
```

The functions perform a store using the cache operator specified in the [PTX ISA](#) guide. They support all C++ fundamental types, CUDA vector types (except x3 components), and extended floating-point types, such as `__half`, `__half2`, `__nv_bfloat16`, and `__nv_bfloat162`.

5.4.8.4 __trap()

Hint

It is suggested to use the `cuda::std::terminate()` function provided by `libc++` ([C++ reference](#)) as a portable alternative to `__trap()`.

A trap operation can be initiated by calling the `__trap()` function from any device thread.

```
void __trap();
```

Execution of the kernel is aborted, raising an interrupt in the host program. Calling `__trap()` results in a corrupted CUDA context, causing subsequent CUDA calls and kernel invocations to fail.

5.4.8.5 __nanosleep()

```
__device__ void __nanosleep(unsigned nanoseconds);
```

The function `__nanosleep(ns)` suspends the thread for a sleep duration of approximately ns nanoseconds. The maximum sleep duration is approximately one millisecond.

Example:

The following code implements a mutex with exponential back-off.

```

__device__ void mutex_lock(unsigned* mutex) {
    unsigned ns = 8;
    while (atomicCAS(mutex, 0, 1) == 1) {
        __nanosleep(ns);
        if (ns < 256) {
            ns *= 2;
        }
    }
}

__device__ void mutex_unlock(unsigned *mutex) {
    atomicExch(mutex, 0);
}

```

5.4.8.6 Dynamic Programming eXtension (DPX) Instructions

The DPX set of functions enables finding minimum and maximum values, as well as fused addition and minimum/maximum for up to three 16- or 32-bit signed or unsigned integer parameters. There is an optional ReLU, namely clamping to zero, feature.

Comparison functions:

- Three parameters. Semantic: $\max(a, b, c), \min(a, b, c)$.

```

int __vimax3_s32 (int, int, int);
unsigned __vimax3_s16x2(unsigned, unsigned, unsigned);
unsigned __vimax3_u32 (unsigned, unsigned, unsigned);
unsigned __vimax3_u16x2(unsigned, unsigned, unsigned);

int __vimin3_s32 (int, int, int);
unsigned __vimin3_s16x2(unsigned, unsigned, unsigned);
unsigned __vimin3_u32 (unsigned, unsigned, unsigned);
unsigned __vimin3_u16x2(unsigned, unsigned, unsigned);

```

- Two parameters, with ReLU. Semantic: $\max(a, b, 0), \max(\min(a, b), 0)$.

```

int __vimax_s32_relu (int, int);
unsigned __vimax_s16x2_relu(unsigned, unsigned);

int __vimin_s32_relu (int, int);
unsigned __vimin_s16x2_relu(unsigned, unsigned);

```

- Three parameters, with ReLU. Semantic: $\max(a, b, c, 0), \max(\min(a, b, c), 0)$.

```

int __vimax3_s32_relu (int, int, int);
unsigned __vimax3_s16x2_relu(unsigned, unsigned, unsigned);

int __vimin3_s32_relu (int, int, int);
unsigned __vimin3_s16x2_relu(unsigned, unsigned, unsigned);

```

- Two parameters, also returning which parameter was smaller/larger:

```

int __vibmax_s32 (int, int, bool* pred);
unsigned __vibmax_u32 (unsigned, unsigned, bool* pred);

```

(continues on next page)

(continued from previous page)

```

unsigned __vibmax_s16x2(unsigned, unsigned, bool* pred);
unsigned __vibmax_u16x2(unsigned, unsigned, bool* pred);

    int __vibmin_s32 (    int,    int,    bool* pred);
unsigned __vibmin_u32 (unsigned, unsigned, bool* pred);
unsigned __vibmin_s16x2(unsigned, unsigned, bool* pred);
unsigned __vibmin_u16x2(unsigned, unsigned, bool* pred);

```

Fused addition and minimum/maximum:

- Three parameters, comparing (first + second) with the third. Semantic: $\max(a + b, c), \min(a + b, c)$

```

    int __viaddmax_s32 (    int,    int,    int);
unsigned __viaddmax_s16x2(unsigned, unsigned, unsigned);
unsigned __viaddmax_u32 (unsigned, unsigned, unsigned);
unsigned __viaddmax_u16x2(unsigned, unsigned, unsigned);

    int __viaddmin_s32 (    int,    int,    int);
unsigned __viaddmin_s16x2(unsigned, unsigned, unsigned);
unsigned __viaddmin_u32 (unsigned, unsigned, unsigned);
unsigned __viaddmin_u16x2(unsigned, unsigned, unsigned);

```

- Three parameters, with ReLU, comparing (first + second) with the third and a zero. Semantic: $\max(a + b, c, 0), \max(\min(a + b, c), 0)$

```

    int __viaddmax_s32_relu (    int,    int,    int);
unsigned __viaddmax_s16x2_relu(unsigned, unsigned, unsigned);

    int __viaddmin_s32_relu (    int,    int,    int);
unsigned __viaddmin_s16x2_relu(unsigned, unsigned, unsigned);

```

These instructions are hardware-accelerated or software emulated depending on compute capability. See [Arithmetic Instructions](#) section for the compute capability requirements.

The full API can be found in [CUDA Math API documentation](#).

The DPX is an exceptionally useful tool for implementing dynamic programming algorithms such as the Smith-Waterman and Needleman-Wunsch algorithms in genomics and the Floyd-Warshall algorithm in route optimization.

Maximum value of three signed 32-bit integers, with ReLU:

```

int a          = -15;
int b          =  8;
int c          =  5;
int max_value_0 = __vimax3_s32_relu(a, b, c); //  $\max(-15, 8, 5, 0) = 8$ 
int d          = -2;
int e          = -4;
int max_value_1 = __vimax3_s32_relu(a, d, e); //  $\max(-15, -2, -4, 0) = 0$ 

```

Minimum value of the sum of two 32-bit signed integers, another 32-bit signed integer and a zero (ReLU):

```
int a      = -5;
int b      = 6;
int c      = -2;
int max_value_0 = __viaddmax_s32_relu(a, b, c); // max(-5 + 6, -2, 0) = max(1,
↪ -2, 0) = 1
int d      = 4;
int max_value_1 = __viaddmax_s32_relu(a, d, c); // max(-5 + 4, -2, 0) = max(-
↪ 1, -2, 0) = 0
```

Minimum value of two unsigned 32-bit integers and determining which value is smaller:

```
unsigned a = 9;
unsigned b = 6;
bool      smaller_value;
unsigned min_value = __vibmin_u32(a, b, &smaller_value); // min_value is 6,
↪ smaller_value is true
```

Maximum values of three pairs of unsigned 16-bit integers:

```
unsigned a      = 0x00050002;
unsigned b      = 0x00070004;
unsigned c      = 0x00020006;
unsigned max_value = __vimax3_u16x2(a, b, c); // max(5, 7, 2) and max(2, 4, 6),
↪ so max_value is 0x00070006
```

5.4.9. Compiler Optimization Hints

Compiler optimization hints decorate code with additional information to help the compiler optimize generated code.

- ▶ The built-in functions are always available in the device code.
- ▶ Host code support depends on the host compiler.

5.4.9.1 #pragma unroll

The compiler unrolls small loops with a known trip count by default. However, the `#pragma unroll` directive can be used to control the unrolling of any given loop. This directive must be placed immediately before the loop and only applies to that loop.

An integral constant expression may optionally follow. The following are cases for an integral constant expression:

- ▶ If it is absent, the loop will be completely unrolled if its trip count is constant.
- ▶ If it evaluates to 0 or 1, the loop will not be unrolled.
- ▶ If it is a non-positive integer or greater than `INT_MAX`, the pragma will be ignored, and a warning will be issued.

Examples:

```
struct MyStruct {
    static constexpr int value = 4;
};
```

(continues on next page)

(continued from previous page)

```

inline constexpr int Count = 4;

__device__ void foo(int* p1, int* p2) {
    // no argument specified, the loop will be completely unrolled
    #pragma unroll
    for (int i = 0; i < 12; ++i)
        p1[i] += p2[i] * 2;

    // unroll value = 5
    #pragma unroll (Count + 1)
    for (int i = 0; i < 12; ++i)
        p1[i] += p2[i] * 4;

    // unroll value = 1, loop unrolling disabled
    #pragma unroll 1
    for (int i = 0; i < 12; ++i)
        p1[i] += p2[i] * 8;

    // unroll value = 4
    #pragma unroll (MyStruct::value)
    for (int i = 0; i < 12; ++i)
        p1[i] += p2[i] * 16;

    // negative value, pragma unroll ignored
    #pragma unroll -1
    for (int i = 0; i < 12; ++i)
        p1[i] += p2[i] * 2;
}

```

See the example on [Compiler Explorer](#).

5.4.9.2 __builtin_assume_aligned()

Hint

It is suggested to use the `cuda::std::assume_aligned()` function provided by `libcud++` ([C++ reference](#)) as a portable and safer alternative to the built-in functions.

```

void* __builtin_assume_aligned(const void* ptr, size_t align)
void* __builtin_assume_aligned(const void* ptr, size_t align, <integral type>
    ↪ offset)

```

The built-in functions enable the compiler to assume that the returned pointer is aligned to at least `align` bytes.

- The three parameter version enables the compiler to assume that `(char*) ptr - offset` is aligned to at least `align` bytes.

`align` must be a power of two and an integer literal.

Examples:

```
void* res1 = __builtin_assume_aligned(ptr, 32);    // compiler can assume
↳ 'res1' is at least 32-byte aligned
void* res2 = __builtin_assume_aligned(ptr, 32, 8); // compiler can assume
↳ 'res2 = (char*) ptr - 8' is at least 32-byte aligned
```

5.4.9.3 __builtin_assume() and __assume()

```
void __builtin_assume(bool predicate)
void __assume         (bool predicate) // only with Microsoft Compiler
```

The built-in function enables the compiler to assume that the boolean argument is true. If the argument is false at runtime, the behavior is undefined. Note that if the argument has side effects, the behavior is unspecified.

Example:

```
__device__ bool is_greater_than_zero(int value) {
    return value > 0;
}

__device__ bool f(int value) {
    __builtin_assume(value > 0);
    return is_greater_than_zero(value); // returns true, without evaluating
↳ the condition
}
```

5.4.9.4 __builtin_expect()

```
long __builtin_expect(long input, long expected)
```

The built-in function tells the compiler that `input` is expected to equal `expected`, and returns the value of `input`. It is typically used to provide branch prediction information to the compiler. It behaves like the C++20 `[[likely]]` and `[[unlikely]]` attributes.

Example:

```
// indicate to the compiler that likely "var == 0"
if (__builtin_expect(var, 0))
    doit();
```

5.4.9.5 __builtin_unreachable()

```
void __builtin_unreachable(void)
```

The built-in function tells the compiler that the control flow will never reach the point at which the function is called. If the control flow does reach this point at runtime, the program has undefined behavior.

This function is useful for avoiding code generation of unreachable branches and disabling compiler warnings for unreachable code.

Example:

```
// indicates to the compiler that the default case label is never reached.
switch (in) {
    case 1: return 4;
    case 2: return 10;
    default: __builtin_unreachable();
}
```

5.4.9.6 Custom ABI Pragmas

The `#pragma nv_abi` directive enables applications compiled in *separate compilation* mode to achieve performance similar to that of *whole program compilation* by preserving the number of registers used by a function.

The syntax for using this pragma is as follows, where `EXPR` refers to any integral constant expression:

```
#pragma nv_abi preserve_n_data(EXPR) preserve_n_control(EXPR)
```

- ▶ The arguments that follow `#pragma nv_abi` are optional and may be provided in any order; however, at least one argument is required.
- ▶ The `preserve_n` arguments limit the number of registers preserved during a function call:
 - ▶ `preserve_n_data(EXPR)` limits the number of data registers.
 - ▶ `preserve_n_control(EXPR)` limits the number of control registers.

The `#pragma nv_abi` directive can be placed immediately before a device function declaration or definition.

```
#pragma nv_abi preserve_n_data(16)
__device__ void dev_func();

#pragma nv_abi preserve_n_data(16) preserve_n_control(8)
__device__ int dev_func() {
    return 0;
}
```

Alternatively, it can be placed directly before an indirect function call within a C++ expression statement inside a device function. Note that while indirect function calls to free functions are supported, indirect calls to function references or class member functions are not supported.

```
__device__ int dev_func1();

struct MyStruct {
    __device__ int member_func2();
};

__device__ void test() {
    auto* dev_func_ptr = &dev_func1; // type: int (*)(void)
    #pragma nv_abi preserve_n_control(8)
    int v1 = dev_func_ptr();           // CORRECT, indirect call

    #pragma nv_abi preserve_n_control(8)
    int v2 = dev_func1();              // WRONG, direct call; the pragma has no
    ↪effect
}
```

(continues on next page)

(continued from previous page)

```

// dev_func1 has type: int(void)

auto& dev_func_ref = &dev_func1; // type: int (&)(void)
#pragma nv_abi preserve_n_control(8)
int v3 = dev_func_ref();          // WRONG, call to a reference
                                   // the pragma has no effect

auto member_function_ptr = &MyStruct::member_func2; // type: int
→(MyStruct::*)(void)
#pragma nv_abi preserve_n_control(8)
int v4 = member_function_ptr();   // WRONG, indirect call to member function
                                   // the pragma has no effect
}

```

When applied to a device function's declaration or definition, the pragma modifies the custom ABI properties for any calls to that function. When placed at an indirect function call site, it affects the ABI properties only for that specific call. Note that the pragma only affects indirect function calls when placed at a call site; it has no effect on direct function calls.

```

#pragma nv_abi preserve_n_control(8)
__device__ int dev_func3();

__device__ int dev_func4();

__device__ void test() {
    int v1 = dev_func3();          // CORRECT, the pragma affects the direct
    →call

    auto* dev_func_ptr = &dev_func4; // type: int (*)(void)
    #pragma nv_abi preserve_n_control(8)
    int v2 = dev_func_ptr();       // CORRECT, the pragma affects the
    →indirect call

    int v3 = dev_func_ptr();       // WRONG, the pragma has no effect
}

```

Note that a program is ill-formed if the pragma arguments for a function declaration and its corresponding definition do not match.

5.4.9.7 MMA Throughput Pragma

The `nv_mma_throughput` pragma is a directive to enable compiler optimizations specifically tuned for matrix multiply-accumulate operations. It is valid only at entry-function scope, that is, on a `__global__` function. Effects may extend into `__device__` functions invoked from that entry function by optimizations propagated along call paths.

Example:

```

#pragma nv_mma_throughput
__global__ void kernel(){
    ...
}

```

Note

The `nv_mma_throughput` pragma is still experimental. The settings it enables were tuned on a limited set of internal NVIDIA workloads. Improved performance is not guaranteed for every kernel.

5.4.10. Debugging and Diagnostics

5.4.10.1 Assertion

```
#define assert(expression) /* unspecified */
```

The `assert()` macro stops kernel execution if `expression` is equal to zero. If the program is run within a debugger, a breakpoint is triggered, allowing the debugger to be used to inspect the current state of the device. Otherwise, each thread for which `expression` is equal to zero prints a message to `stderr` after synchronizing with the host via `cudaDeviceSynchronize()`, `cudaStreamSynchronize()`, or `cudaEventSynchronize()`. The format of this message is as follows:

```
<filename>:<line number>:<function>:
block: [blockIdx.x,blockIdx.y,blockIdx.z],
thread: [threadIdx.x,threadIdx.y,threadIdx.z]
Assertion `expression` failed.
```

Execution of the kernel is aborted, raising an interrupt in the host program. The `assert()` macro results in a corrupted CUDA context, causing any subsequent CUDA calls or kernel invocations to fail with `cudaErrorAssert`.

The kernel execution is unaffected if `expression` is different from zero.

For example, the following program from source file `test.cu`

```
#include <assert.h>

__global__ void testAssert(void) {
    int is_one = 1;
    int should_be_one = 0;

    // This will have no effect
    assert(is_one);

    // This will halt kernel execution
    assert(should_be_one);
}

int main(void) {
    testAssert<<<1,1>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

will output:

```
test.cu:11: void testAssert(): block: [0,0,0], thread: [0,0,0] Assertion
↪ `should_be_one` failed.
```

Assertions are intended for debugging purposes. Since they can affect performance, it is recommended that they be disabled in production code. They can be disabled at compile time by defining the `NDEBUG` preprocessor macro before including `assert.h` or `<cassert>`, or by using the compiler flag `-DNDEBUG`. Note that the expression should not have side effects; otherwise, disabling the assertion will affect the functionality of the code.

The `assert()` macro is available in both `__device__` and `__tile__` code.

5.4.10.2 Breakpoint Function

The execution of a kernel function can be suspended by calling the `__brkpt()` function from any device thread.

```
void __brkpt();
```

5.4.10.3 Diagnostic Pragmas

The following pragmas can be used to manage the severity of errors that are triggered when a specific diagnostic message is raised.

```
#pragma nv_diag_suppress
#pragma nv_diag_warning
#pragma nv_diag_error
#pragma nv_diag_default
#pragma nv_diag_once
```

The uses of these pragmas are as follows:

```
#pragma nv_diag_xxx <error_number1>, <error_number2> ...
```

The affected diagnostic is specified using the error number shown in the warning message. Any diagnostic can be changed to an error, but only warnings can have their severity suppressed or restored after being changed to an error. The `nv_diag_default` pragma returns the severity of a diagnostic to the severity that was in effect before any other pragmas were issued, namely, the normal severity of the message as modified by any command-line options. The following example suppresses the declared but never referenced warning of `foo()`:

```
#pragma nv_diag_suppress 177 // "declared but never referenced"
void foo() {
    int i = 0;
}

#pragma nv_diag_default 177
void bar() {
    int i = 0;
}
```

The following pragmas may be used to save and restore the current diagnostic pragma state:

```
#pragma nv_diagnostic push
#pragma nv_diagnostic pop
```

Examples:

```
#pragma nv_diagnostic push
#pragma nv_diag_suppress 177 // "declared but never referenced"
void foo() {
    int i = 0;
}

#pragma nv_diagnostic pop
void bar() {
    int i = 0; // raise a warning
}
```

Note that these directives only affect the nvcc CUDA front-end compiler. They have no effect on the host compiler.

nvcc defines the macro `__NVCC_DIAG_PRAGMA_SUPPORT__` when diagnostic pragmas are supported.

5.4.11. Warp Matrix Functions

C++ warp matrix operations leverage Tensor Cores to accelerate matrix problems of the form $D=A*B+C$. These operations are supported on mixed-precision floating point data for devices of compute capability 7.0 or higher. This requires co-operation from all threads in a *warp*. In addition, these operations are allowed in conditional code only if the condition evaluates identically across the entire *warp*, otherwise the code execution is likely to hang.

5.4.11.1 Description

All following functions and types are defined in the namespace `nvcuda::wmma`. Sub-byte operations are considered preview, i.e. the data structures and APIs for them are subject to change and may not be compatible with future releases. This extra functionality is defined in the `nvcuda::wmma::experimental` namespace.

```
template<typename Use, int m, int n, int k, typename T, typename Layout=void>
    class fragment;

void load_matrix_sync(fragment<...> &a, const T* mptr, unsigned ldm);
void load_matrix_sync(fragment<...> &a, const T* mptr, unsigned ldm, layout_t
    layout);
void store_matrix_sync(T* mptr, const fragment<...> &a, unsigned ldm, layout_t
    layout);
void fill_fragment(fragment<...> &a, const T& v);
void mma_sync(fragment<...> &d, const fragment<...> &a, const fragment<...> &
    b, const fragment<...> &c, bool satf=false);
```

fragment

An overloaded class containing a section of a matrix distributed across all threads in the warp. The mapping of matrix elements into fragment internal storage is unspecified and subject to change in future architectures.

Only certain combinations of template arguments are allowed. The first template parameter specifies how the fragment will participate in the matrix operation. Acceptable values for `Use` are:

- `matrix_a` when the fragment is used as the first multiplicand, `A`,
- `matrix_b` when the fragment is used as the second multiplicand, `B`, or

- accumulator when the fragment is used as the source or destination accumulators (C or D, respectively).

The `m`, `n` and `k` sizes describe the shape of the warp-wide matrix tiles participating in the multiply-accumulate operation. The dimension of each tile depends on its role. For `matrix_a` the tile takes dimension `m` $\times$ `k`; for `matrix_b` the dimension is `k` $\times$ `n`, and accumulator tiles are `m` $\times$ `n`.

The data type, `T`, may be `double`, `float`, `__half`, `__nv_bfloat16`, `char`, or `unsigned char` for multiplicands and `double`, `float`, `int`, or `__half` for accumulators. As documented in *Element Types and Matrix Sizes*, limited combinations of accumulator and multiplicand types are supported. The `Layout` parameter must be specified for `matrix_a` and `matrix_b` fragments. `row_major` or `col_major` indicate that elements within a matrix row or column are contiguous in memory, respectively. The `Layout` parameter for an accumulator matrix should retain the default value of `void`. A row or column layout is specified only when the accumulator is loaded or stored as described below.

load_matrix_sync

Waits until all warp lanes have arrived at `load_matrix_sync` and then loads the matrix fragment `a` from memory. `mptr` must be a 256-bit aligned pointer pointing to the first element of the matrix in memory. `ldm` describes the stride in elements between consecutive rows (for row major layout) or columns (for column major layout) and must be a multiple of 8 for `__half` element type or multiple of 4 for `float` element type. (i.e., multiple of 16 bytes in both cases). If the fragment is an accumulator, the `layout` argument must be specified as either `mem_row_major` or `mem_col_major`. For `matrix_a` and `matrix_b` fragments, the layout is inferred from the fragment's `layout` parameter. The values of `mptr`, `ldm`, `layout` and all template parameters for `a` must be the same for all threads in the warp. This function must be called by all threads in the warp, or the result is undefined.

store_matrix_sync

Waits until all warp lanes have arrived at `store_matrix_sync` and then stores the matrix fragment `a` to memory. `mptr` must be a 256-bit aligned pointer pointing to the first element of the matrix in memory. `ldm` describes the stride in elements between consecutive rows (for row major layout) or columns (for column major layout) and must be a multiple of 8 for `__half` element type or multiple of 4 for `float` element type. (i.e., multiple of 16 bytes in both cases). The layout of the output matrix must be specified as either `mem_row_major` or `mem_col_major`. The values of `mptr`, `ldm`, `layout` and all template parameters for `a` must be the same for all threads in the warp.

fill_fragment

Fill a matrix fragment with a constant value `v`. Because the mapping of matrix elements to each fragment is unspecified, this function is ordinarily called by all threads in the warp with a common value for `v`.

mma_sync

Waits until all warp lanes have arrived at `mma_sync`, and then performs the warp-synchronous matrix multiply-accumulate operation $D=A*B+C$. The in-place operation, $C=A*B+C$, is also supported. The value of `satf` and template parameters for each matrix fragment must be the same for all threads in the warp. Also, the template parameters `m`, `n` and `k` must match between fragments `A`, `B`, `C` and `D`. This function must be called by all threads in the warp, or the result is undefined.

If `satf` (saturate to finite value) mode is `true`, the following additional numerical properties apply for the destination accumulator:

- If an element result is +Infinity, the corresponding accumulator will contain `+MAX_NORM`
- If an element result is -Infinity, the corresponding accumulator will contain `-MAX_NORM`
- If an element result is NaN, the corresponding accumulator will contain `+0`

Because the map of matrix elements into each thread's fragment is unspecified, individual matrix elements must be accessed from memory (shared or global) after calling `store_matrix_sync`. In the special case where all threads in the warp will apply an element-wise operation uniformly to all fragment elements, direct element access can be implemented using the following fragment class members.

```
enum fragment<Use, m, n, k, T, Layout>::num_elements;
T fragment<Use, m, n, k, T, Layout>::x[num_elements];
```

As an example, the following code scales an accumulator matrix tile by half.

```
wmma::fragment<wmma::accumulator, 16, 16, 16, float> frag;
float alpha = 0.5f; // Same value for all threads in warp
/*...*/
for(int t=0; t<frag.num_elements; t++)
    frag.x[t] *= alpha;
```

5.4.11.2 Alternate Floating Point

Tensor Cores support alternate types of floating point operations on devices with compute capability 8.0 and higher.

`__nv_bfloat16`

This data format is an alternate fp16 format that has the same range as f32 but reduced precision (7 bits). You can use this data format directly with the `__nv_bfloat16` type available in `cuda_bf16.h`. Matrix fragments with `__nv_bfloat16` data types are required to be composed with accumulators of `float` type. The shapes and operations supported are the same as with `__half`.

`tf32`

This data format is a special floating-point format supported by Tensor Cores, with the same range as f32 and reduced precision (≥ 10 bits). The internal layout of this format is implementation-defined. To use this floating-point format with WMMA operations, the input matrices must be manually converted to tf32 precision.

To facilitate conversion, a new intrinsic `__float_to_tf32` is provided. While the input and output arguments to the intrinsic are of `float` type, the output will be `tf32` numerically. This new precision is intended to be used with Tensor Cores only, and if mixed with other floattype operations, the precision and range of the result will be undefined.

Once an input matrix (`matrix_a` or `matrix_b`) is converted to tf32 precision, the combination of a fragment with `precision::tf32` precision, and a data type of `float` to `load_matrix_sync` will take advantage of this new capability. Both the accumulator fragments must have `float` data types. The only supported matrix size is 16x16x8 (m-n-k).

The elements of the fragment are represented as `float`, hence the mapping from `element_type<T>` to `storage_element_type<T>` is:

```
precision::tf32 -> float
```

5.4.11.3 Double Precision

Tensor Cores support double-precision floating point operations on devices with compute capability 8.0 and higher. To use this new functionality, a fragment with the `double` type must be used. The `mma_sync` operation will be performed with the `.rn` (rounds to nearest even) rounding modifier.

5.4.11.4 Sub-byte Operations

Sub-byte WMMA operations provide a way to access the low-precision capabilities of Tensor Cores. They are considered a preview feature i.e. the data structures and APIs for them are subject to change and may not be compatible with future releases. This functionality is available via the `nvcuda::wmma::experimental` namespace:

```
namespace experimental {
    namespace precision {
        struct u4; // 4-bit unsigned
        struct s4; // 4-bit signed
        struct b1; // 1-bit
    }
    enum bmmaBitOp {
        bmmaBitOpXOR = 1, // compute_75 minimum
        bmmaBitOpAND = 2  // compute_80 minimum
    };
    enum bmmaAccumulateOp { bmmaAccumulateOpPOPC = 1 };
}
```

For 4 bit precision, the APIs available remain the same, but you must specify `experimental::precision::u4` or `experimental::precision::s4` as the fragment data type. Since the elements of the fragment are packed together, `num_storage_elements` will be smaller than `num_elements` for that fragment. The `num_elements` variable for a sub-byte fragment, hence returns the number of elements of sub-byte type `element_type<T>`. This is true for single bit precision as well, in which case, the mapping from `element_type<T>` to `storage_element_type<T>` is as follows:

```
experimental::precision::u4 -> unsigned (8 elements in 1 storage element)
experimental::precision::s4 -> int (8 elements in 1 storage element)
experimental::precision::b1 -> unsigned (32 elements in 1 storage element)
T -> T //all other types
```

The allowed layouts for sub-byte fragments is always `row_major` for `matrix_a` and `col_major` for `matrix_b`.

For sub-byte operations the value of `ldm` in `load_matrix_sync` should be a multiple of 32 for element type `experimental::precision::u4` and `experimental::precision::s4` or a multiple of 128 for element type `experimental::precision::b1` (i.e., multiple of 16 bytes in both cases).

Note

Support for the following variants for MMA instructions is deprecated and will be removed in `sm_90`:

- ▶ `experimental::precision::u4`
- ▶ `experimental::precision::s4`
- ▶ `experimental::precision::b1` with `bmmaBitOp` set to `bmmaBitOpXOR`

bmma_sync

Waits until all warp lanes have executed `bmma_sync`, and then performs the warp-synchronous bit matrix multiply-accumulate operation $D = (A \text{ op } B) + C$, where `op` consists of a logical operation `bmmaBitOp` followed by the accumulation defined by `bmmaAccumulateOp`. The available operations are:

`mmaBitOpXOR`, a 128-bit XOR of a row in `matrix_a` with the 128-bit column of `matrix_b`

`mmaBitOpAND`, a 128-bit AND of a row in `matrix_a` with the 128-bit column of `matrix_b`, available on devices with compute capability 8.0 and higher.

The accumulate op is always `mmaAccumulateOpPOPC` which counts the number of set bits.

5.4.11.5 Restrictions

The special format required by tensor cores may be different for each major and minor device architecture. This is further complicated by threads holding only a fragment (opaque architecture-specific ABI data structure) of the overall matrix, with the developer not allowed to make assumptions on how the individual parameters are mapped to the registers participating in the matrix multiply-accumulate.

Since fragments are architecture-specific, it is unsafe to pass them from function A to function B if the functions have been compiled for different link-compatible architectures and linked together into the same device executable. In this case, the size and layout of the fragment will be specific to one architecture and using WMMA APIs in the other will lead to incorrect results or potentially, corruption.

An example of two link-compatible architectures, where the layout of the fragment differs, is `sm_70` and `sm_75`.

```
fragA.cu: void foo() { wmma::fragment<...> mat_a; bar(&mat_a); }
fragB.cu: void bar(wmma::fragment<...> *mat_a) { // operate on mat_a }
```

```
// sm_70 fragment layout
$> nvcc -dc -arch=compute_70 -code=sm_70 fragA.cu -o fragA.o
// sm_75 fragment layout
$> nvcc -dc -arch=compute_75 -code=sm_75 fragB.cu -o fragB.o
// Linking the two together
$> nvcc -dlink -arch=sm_75 fragA.o fragB.o -o frag.o
```

This undefined behavior might also be undetectable at compilation time and by tools at runtime, so extra care is needed to make sure the layout of the fragments is consistent. This linking hazard is most likely to appear when linking with a legacy library that is both built for a different link-compatible architecture and expecting to be passed a WMMA fragment.

Note that in the case of weak linkages (for example, a CUDA C++ inline function), the linker may choose any available function definition which may result in implicit passes between compilation units.

To avoid these sorts of problems, the matrix should always be stored out to memory for transit through external interfaces (e.g. `wmma::store_matrix_sync(dst, ...)`;) and then it can be safely passed to `bar()` as a pointer type [e.g. `float *dst`].

Note that since `sm_70` can run on `sm_75`, the above example `sm_75` code can be changed to `sm_70` and correctly work on `sm_75`. However, it is recommended to have `sm_75` native code in your application when linking with other `sm_75` separately compiled binaries.

5.4.11.6 Element Types and Matrix Sizes

Tensor Cores support a variety of element types and matrix sizes. The following table presents the various combinations of `matrix_a`, `matrix_b` and accumulator matrix supported:

Matrix A	Matrix B	Accumulator	Matrix Size (m-n-k)
__half	__half	float	16x16x16
__half	__half	float	32x8x16
__half	__half	float	8x32x16
__half	__half	__half	16x16x16
__half	__half	__half	32x8x16
__half	__half	__half	8x32x16
unsigned char	unsigned char	int	16x16x16
unsigned char	unsigned char	int	32x8x16
unsigned char	unsigned char	int	8x32x16
signed char	signed char	int	16x16x16
signed char	signed char	int	32x8x16
signed char	signed char	int	8x32x16

Alternate floating-point support:

Matrix A	Matrix B	Accumulator	Matrix Size (m-n-k)
__nv_bfloat16	__nv_bfloat16	float	16x16x16
__nv_bfloat16	__nv_bfloat16	float	32x8x16
__nv_bfloat16	__nv_bfloat16	float	8x32x16
precision::tf32	precision::tf32	float	16x16x8

Double Precision Support:

Matrix A	Matrix B	Accumulator	Matrix Size (m-n-k)
double	double	double	8x8x4

Experimental support for sub-byte operations:

Matrix A	Matrix B	Accumulator	Matrix Size (m-n-k)
precision::u4	precision::u4	int	8x8x32
precision::s4	precision::s4	int	8x8x32
precision::b1	precision::b1	int	8x8x128

5.4.11.7 Example

The following code implements a 16x16x16 matrix multiplication in a single warp.

```
#include <mma.h>
using namespace nv_cuda;

__global__ void wmma_ker(half *a, half *b, float *c) {
    // Declare the fragments
    wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::col_major> a_frag;
    wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::row_major> b_frag;
    wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;

    // Initialize the output to zero
    wmma::fill_fragment(c_frag, 0.0f);

    // Load the inputs
    wmma::load_matrix_sync(a_frag, a, 16);
    wmma::load_matrix_sync(b_frag, b, 16);

    // Perform the matrix multiplication
    wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);

    // Store the output
    wmma::store_matrix_sync(c, c_frag, 16, wmma::mem_row_major);
}
```

5.5. Floating-Point Computation

5.5.1. Floating-Point Introduction

Since the adoption of the [IEEE-754 Standard](#) for Binary Floating-Point Arithmetic in 1985, virtually all mainstream computing systems, including NVIDIA's CUDA architectures, have implemented the standard. The IEEE-754 standard specifies how the results of floating-point arithmetic should be approximated.

To get accurate results and achieve the highest performance with the required precision, it is important to consider many aspects of floating-point behavior. This is particularly important in a heterogeneous computing environment where operations are performed on different types of hardware.

The following sections review the basic properties of floating-point computation and cover Fused Multiply-Add (FMA) operations and the dot product. These examples illustrate how different implementation choices affect accuracy.

5.5.1.1 Floating-Point Format

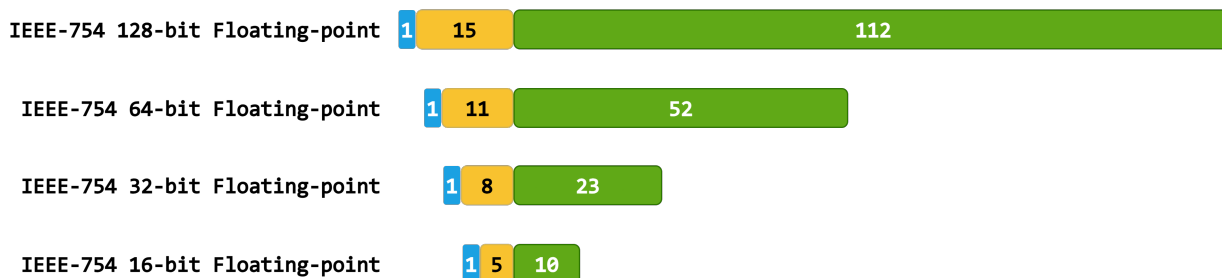
Floating-point format and functionality are defined in the [IEEE-754 Standard](#).

The standard mandates that binary floating-point data be encoded on three fields:

- **Sign:** one bit to indicate a positive or negative number.
- **Exponent:** encodes the base 2 exponent offset by a numeric bias.
- **Significand** (also called *mantissa* or *fraction*): encodes the fractional value of the number.



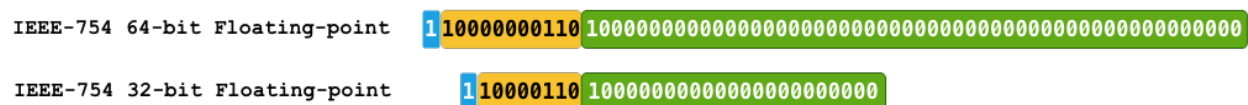
- ▶ 16-bit, also known as half-precision, corresponding to the `__half` data type in CUDA.
- ▶ 32-bit, also known as single-precision, corresponding to the `float` data type in C, C++, and CUDA.
- ▶ 64-bit, also known as double-precision, corresponding to the `double` data type in C, C++, and CUDA.
- ▶ 128-bit, also known as quad-precision, corresponding to the `__float128` or `_Float128` data types in CUDA.



$$(-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{\text{exponent}-\text{bias}}$$

$$(-1)^{\text{sign}} \times 0.\text{mantissa} \times 2^{1-\text{bias}}$$

For example, the value $-192 = (-1)^1 \times 2^7 \times 1.5$, and is encoded as a negative sign, an exponent of 7, and a fractional part 0.5. Hence the exponent 7 is represented by bit strings with values $7 + 127 = 134 = 10000110$ for float and $7 + 1023 = 1030 = 10000000110$ for double. The mantissa $0.5 = 2^{-1}$ is represented by a binary value with 1 in the first position. The binary encodings of -192 in single-precision and double-precision are shown in the following figure:



Since the fraction field uses a limited number of bits, not all real numbers can be represented exactly. For instance, the binary representation of the mathematical value of the fraction $2/3$ is $0.10101010 \dots$, which has an infinite number of bits after the binary point. Therefore, $2/3$ must be rounded before it can be represented as a floating-point number with limited precision. The rounding rules and modes are specified in IEEE-754. The most frequently used mode is *round-to-nearest-ties-to-even*, abbreviated round-to-nearest.

5.5.1.2 Normal and Subnormal Values

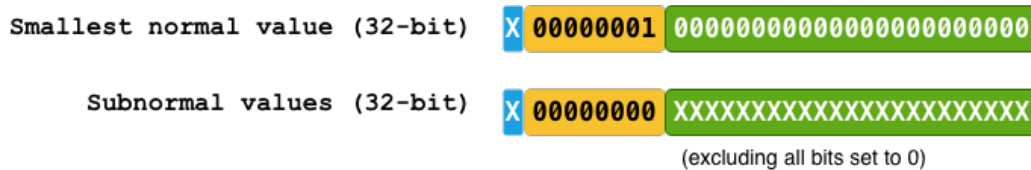
Any floating-point value with an exponent field that is neither all zeros nor all ones is called *normal*.

An important aspect of floating-point values is the wide gap between the smallest representable positive normal number, `FLT_MIN`, and zero. This gap is much wider than the gap between `FLT_MIN` and the second-smallest normal number.

Floating-point *subnormal* numbers, also called *denormals*, were introduced to address this issue. A subnormal floating-point value is represented with all bits in the exponent set to zero and at least one bit set in the significand. Subnormals are a required part of the IEEE-754 floating-point standard.

Subnormal numbers allow for a gradual loss of precision as an alternative to sudden rounding toward zero. However, subnormal numbers are computationally more expensive. Therefore, applications that don't require strict accuracy may choose to avoid them to improve performance. The `nvcc` compiler allows disabling subnormal numbers by setting the `-ftz=true` option (flush-to-zero), which is also included in `--use_fast_math`.

A simplified visualization of the encoding of the smallest normal value and subnormal values in single-precision is shown in the following figure:



where X represents both 0 and 1.

5.5.1.3 Special Values

The IEEE-754 standard defines three special values for floating-point numbers:

Zero:

- ▶ Mathematical zero.
- ▶ Note that there are two possible representations of floating-point zero: $+0$ and -0 . This differs from the representation of integer zero.
- ▶ $+0 == -0$ evaluates to `true`.
- ▶ Zero is encoded with all bits set to 0 in the exponent and significand.

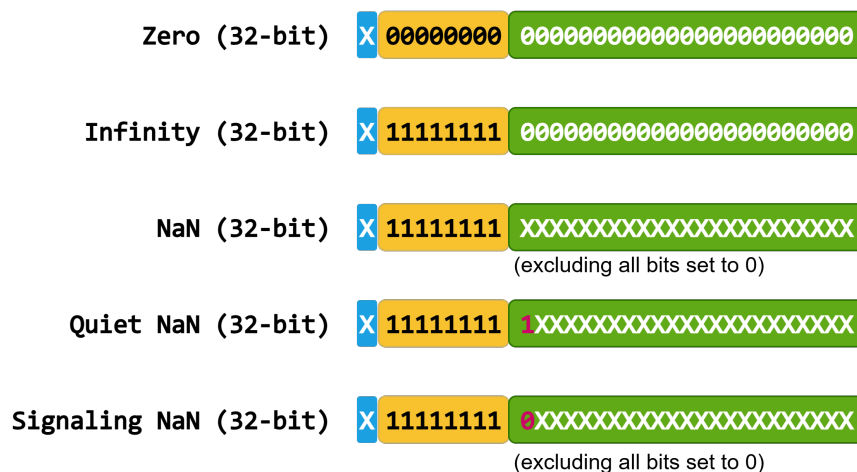
Infinity:

- ▶ Floating-point numbers behave according to saturation arithmetic, in which operations that overflow the representable range result in $+Infinity$ or $-Infinity$.
- ▶ Infinity is encoded with all bits in the exponent set to 1 and all bits in the significand set to 0. There are exactly two encodings for infinity values.
- ▶ Arithmetic operations involving infinity and finite nonzero values typically result in infinity. Indeterminate forms such as $Inf * 0.0$, $Inf - Inf$, Inf / Inf , and $0.0 / 0.0$ result in NaN.

Not-a-Number (NaN):

- ▶ NaN is a special symbol that represents an undefined or non-representable value. Common examples are $0.0 / 0.0$, $\sqrt{-1.0}$, or $+\text{Inf} - \text{Inf}$.
- ▶ NaN is encoded with all bits in the exponent set to 1 and any bit pattern in the significand, except for all bits set to 0. There are $2^{\text{mantissa}+1} - 2$ possible encodings.
- ▶ Any arithmetic operation involving a NaN will result in NaN.
- ▶ Any ordered comparison ($<$, $<=$, $>$, $>=$, $==$) involving a NaN will result in `false`, including `NaN == NaN` (non-reflexive). The unordered comparison `NaN != NaN` returns `true`.
- ▶ NaNs are provided in two forms:
 - ▶ Quiet NaNs `qNaN` are used to propagate errors resulting from invalid operations or values. Invalid arithmetic operations generally produce a quiet NaN. They are encoded with the most significant bit of the significand set to 1.
 - ▶ Signaling NaNs `sNaN` are designed to raise an invalid-operation exception. Signaling NaNs are generally explicitly created. They are encoded with the most significant bit of the significand set to 0.
- ▶ The exact bit patterns for Quiet and Signaling NaNs are implementation-defined. CUDA provides the `cuda::std::numeric_limits<T>::quiet_NaN` and `cuda::std::numeric_limits<T>::signaling_NaN` constants to get their special values.

A simplified visualization of the encodings of special values is shown in the following figure:



where X represents both 0 and 1.

5.5.1.4 Associativity

It is important to note that the rules and properties of mathematical arithmetic do not directly apply to floating-point arithmetic due to its limited precision. The example below shows single-precision values A, B, and C and the exact mathematical value of their sum computed using different associativity.

$$\begin{aligned}
 A &= 2^1 \times 1.000000000000000000000001 \\
 B &= 2^0 \times 1.000000000000000000000001 \\
 C &= 2^3 \times 1.000000000000000000000001 \\
 (A + B) + C &= 2^3 \times 1.011000000000000000000001011 \\
 A + (B + C) &= 2^3 \times 1.011000000000000000000001011
 \end{aligned}$$

Mathematically, $(A + B) + C$ is equal to $A + (B + C)$.

Let $\text{rn}(x)$ denote one rounding step on x . Performing the same computations in single-precision floating-point arithmetic in round-to-nearest mode according to IEEE-754, we obtain:

$$\begin{aligned} A + B &= 2^1 \times 1.10000000000000000000110000\dots \\ \text{rn}(A + B) &= 2^1 \times 1.100000000000000000000010 \\ B + C &= 2^3 \times 1.0010000000000000000000100100\dots \\ \text{rn}(B + C) &= 2^3 \times 1.001000000000000000000001 \\ A + B + C &= 2^3 \times 1.01100000000000000000000101100\dots \\ \text{rn}(\text{rn}(A + B) + C) &= 2^3 \times 1.0110000000000000000000010 \\ \text{rn}(A + \text{rn}(B + C)) &= 2^3 \times 1.0110000000000000000000001 \end{aligned}$$

For reference, the exact mathematical results are also computed above. The results computed according to IEEE-754 differ from the exact mathematical results. Additionally, the results corresponding to the sums $\text{rn}(\text{rn}(A + B) + C)$ and $\text{rn}(A + \text{rn}(B + C))$ differ from each other. In this case, $\text{rn}(A + \text{rn}(B + C))$ is closer to the correct mathematical result than $\text{rn}(\text{rn}(A + B) + C)$.

This example shows that seemingly identical computations can produce different results, even when all basic operations comply with IEEE-754.

5.5.1.5 Fused Multiply-Add (FMA)

The Fused Multiply-Add (FMA) operation computes the result with only one rounding step. Without the FMA, the result would require two rounding steps: one for multiplication and one for addition. Because the FMA uses only one rounding step, it produces a more accurate result.

The Fused Multiply-Add operation can affect the propagation of NaNs differently than two separate operations. However, FMA NaN handling is not universally identical across all targets. Different implementations with multiple NaN operands may prefer a quiet NaN or propagate one operand's payload. Additionally, IEEE-754 does not strictly mandate a deterministic payload selection order when multiple NaN operands are present. NaNs may also occur in intermediate computations, for example, $\infty \times 0 + 1$ or $1 \times \infty - \infty$, resulting in an implementation-defined NaN payload.

For clarity, first consider an example using decimal arithmetic to illustrate how the FMA operation works. We will compute $x^2 - 1$ using five total digits of precision, with four digits after the decimal point.

- For $x = 1.0008$, the correct mathematical result is $x^2 - 1 = 1.60064 \times 10^{-4}$. The closest number using only four digits after the decimal point is 1.6006×10^{-4} .
- The Fused Multiply-Add operation achieves the correct result using only one rounding step $\text{rn}(x \times x - 1) = 1.6006 \times 10^{-4}$.
- The alternative is to compute the multiply and add steps separately. $x^2 = 1.00160064$ translates to $\text{rn}(x \times x) = 1.0016$. The final result is $\text{rn}(\text{rn}(x \times x) - 1) = 1.6000 \times 10^{-4}$.

Rounding the multiply and add separately yields a result that is off by 0.00064. The corresponding FMA computation is wrong by only 0.00004 and its result is closest to the correct mathematical answer. The

- ▶ x86 platforms with the AVX2 ISA, for example, code compiled with the `-mavx2` flag using GCC or Clang, and `/arch:AVX2` with Microsoft Visual Studio.
- ▶ Arm64 (AArch64) platforms with Advanced SIMD (Neon) ISA.
- ▶ `fma(x, y, z)` and `fmaf(x, y, z)` [C Standard Library functions](#).
- ▶ `std::fma(x, y, z)` and `std::fmaf(x, y, z)` [C++ Standard Library functions](#).
- ▶ `cuda::std::fma(x, y, z)` and `cuda::std::fmaf(x, y, z)` [CUDA C++ Standard Library functions](#).

5.5.1.6 Dot Product Example

Consider the problem of finding the dot product of two short vectors $\vec{a}$ and $\vec{b}$ both with four elements.

$$\vec{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{bmatrix} \quad \vec{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix} \quad \vec{a} \cdot \vec{b} = a_1b_1 + a_2b_2 + a_3b_3 + a_4b_4$$

Although this operation is easy to write down mathematically, implementing it in software involves several alternatives that could lead to slightly different results. All of the strategies presented here use operations that are fully compliant with IEEE-754.

Example Algorithm 1: The simplest way to compute the dot product is to use a sequential sum of products, keeping the multiplications and additions separate.

The final result can be represented as $((((a_1 \times b_1) + (a_2 \times b_2)) + (a_3 \times b_3)) + (a_4 \times b_4))$.

Example Algorithm 2: Compute the dot product sequentially using fused multiply-add.

The final result can be represented as $(a_4 \times b_4) + ((a_3 \times b_3) + ((a_2 \times b_2) + (a_1 \times b_1 + 0)))$.

Example Algorithm 3: Compute the dot product using a divide-and-conquer strategy. First, we find the dot products of the first and second halves of the vectors. Then, we combine these results using addition. This algorithm is called the “parallel algorithm” because the two subproblems can be computed in parallel since they are independent of each other. However, the algorithm does not require a parallel implementation; it can be implemented with a single thread.

The final result can be represented as $((a_1 \times b_1) + (a_2 \times b_2)) + ((a_3 \times b_3) + (a_4 \times b_4))$.

5.5.1.7 Rounding

The IEEE-754 standard requires support for several operations. These include arithmetic operations such as addition, subtraction, multiplication, division, square root, fused multiply-add, finding the remainder, conversion, scaling, sign, and comparison operations. The results of these operations are guaranteed to be consistent across all implementations of the standard for a given format and rounding mode.

Rounding Modes

The IEEE-754 standard defines four rounding modes: *round-to-nearest*, *round towards positive*, *round towards negative*, and *round towards zero*. CUDA supports all four modes. By default, operations use *round-to-nearest*. [Intrinsic mathematical functions](#) can be used to select other rounding modes for individual operations.

Rounding Mode	Interpretation
rn	Round to nearest, ties to even
rz	Round towards zero
ru	Round towards ∞
rd	Round towards $-\infty$

5.5.1.8 Notes on Host/Device Computation Accuracy

The accuracy of a floating-point computation result is affected by several factors. This section summarizes important considerations for achieving reliable results in floating-point computations. Some of these aspects have been described in greater detail in previous sections.

These aspects are also important when comparing the results between CPU and GPU. Differences between host and device execution must be interpreted carefully. The presence of differences does not necessarily mean the GPU's result is incorrect or that there is a problem with the GPU.

Associativity:

Floating-point addition and multiplication in finite precision are not *associative* because they often result in mathematical values that cannot be directly represented in the target format, requiring rounding. The order in which these operations are evaluated affects how rounding errors accumulate and can significantly alter the final result.

Fused Multiply-Add:

Fused Multiply-Add computes $a \times b + c$ in a single operation, resulting in greater accuracy and a faster execution time. The accuracy of the final result can be affected by its use. Fused Multiply-Add relies on hardware support and can be enabled either explicitly by calling the related function or implicitly through compiler optimization flags.

Precision:

Increasing the floating-point precision can potentially improve the accuracy of the results. Higher precision reduces loss of significance and enables the representation of a wider range of values. However, higher precision types have lower throughput and consume more registers. Additionally, using them to explicitly store input and output increases memory usage and data movement.

Compiler Flags and Optimizations:

All major compilers provide a variety of optimization flags to control the behavior of floating-point operations.

- ▶ The highest optimization level for GCC (-O3), Clang (-O3), nvcc (-O3), and Microsoft Visual Studio (/O2) does not affect floating-point semantics. However, inlining, loop unrolling, vectorization, and common subexpression elimination could affect the results. The NVCC++ compiler also requires the flags -Kieee -Mnofma for IEEE-754-compliant semantics.
- ▶ Refer to the [GCC](#), [Clang](#), [Microsoft Visual Studio Compiler](#), [nvcc++](#), and [Arm C/C++ compiler](#) documentation for detailed information about options that affect floating-point behavior.
- ▶ See also the nvcc [User Manual](#) for detailed descriptions of compiler flags that specifically affect floating-point behavior in CUDA device code: -ftz, -prec-div,

`-prec-sqrt`, `-fmad`, `--use_fast_math`. Besides these floating-point options, it is also important to verify the effects of other compiler optimizations in the context of the user program. Users are encouraged to verify the correctness of their results with extensive testing and compare results obtained with optimizations enabled versus all device code optimizations disabled; see also the `-G` compiler flag.

Library Implementations:

Functions defined outside the IEEE-754 standard are not guaranteed to be correctly rounded and depend on implementation-defined behavior. Therefore, the results may differ across different platforms, including between host, device, and different device architectures.

Deterministic Results:

A deterministic result refers to computing the same bit-wise numerical outputs every time when run with the same inputs under the same specified conditions. Such conditions include:

- ▶ Hardware dependencies, such as execution on the same CPU processor or GPU device.
- ▶ Compiler aspects, such as the version of the compiler and the *Compiler Flags and Optimizations*.
- ▶ Run-time conditions that affect the computation, such as *rounding mode* or environment variables.
- ▶ Identical inputs to the computation.
- ▶ Thread configuration, including the number of threads involved in the computation and their organization, for example block and grid size.
- ▶ The ordering of *arithmetic atomic operations* depends on hardware scheduling which can vary between runs.

Taking Advantage of the CUDA Libraries:

The *CUDA Math Libraries*, *C Standard Library Mathematical functions*, and *C++ Standard Library Mathematical functions* are designed to boost developer productivity for common functionalities, particularly for floating-point math and numerics-intensive routines. These functionalities provide a consistent high-level interface, are optimized, and are widely tested across platforms and edge cases. Users are encouraged to take full advantage of these libraries and avoid tedious manual reimplementations.

5.5.2. Floating-Point Data Types

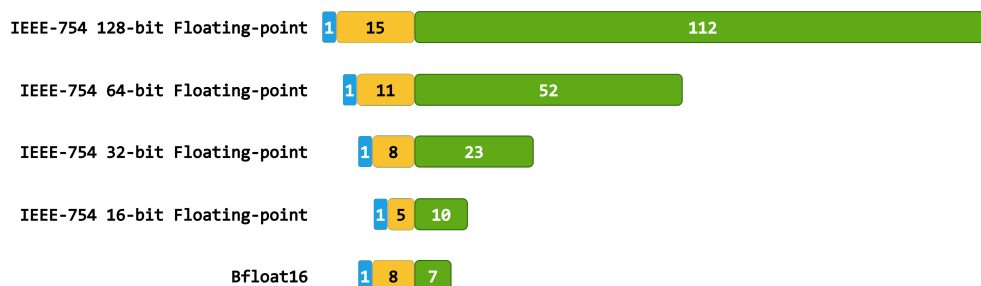
CUDA supports the Bfloat16, half-, single-, double-, and quad-precision floating-point data types. The following table summarizes the supported floating-point data types in CUDA and their requirements.

Table 44: Supported Floating-Point Types

Precision / Name	Data Type	IEEE-754	Header / Built-in	Requirements
Bfloat16	<code>__nv_bfloat16</code>	×	<code><cuda_bf16.h></code>	Compute Capability 8.0 or higher.
Half Precision	<code>__half</code>	✓	<code><cuda_fp16.h></code>	
Single Precision	<code>float</code>	✓	Built-in	
Double Precision	<code>double</code>	✓	Built-in	
Quad Precision	<code>__float128</code>	✓	Built-in <code><rt/device_fp128_functions.h></code> for mathematical functions	Host compiler support and Compute Capability 10.0 or higher. The C or C++ spelling, <code>_Float128</code> and <code>__float128</code> respectively, also depends on the host compiler support.

CUDA also supports [TensorFloat-32 \(TF32\)](#), [microscaling \(MX\)](#) floating-point types, and other [lower precision numerical formats](#) that are not intended for general-purpose computation, but rather for specialized purposes involving tensor cores. These include 4-, 6-, and 8-bit floating-point types. See the [CUDA Math API](#) for more details.

The following figure reports the mantissa and exponent sizes of the supported floating-point data types.



The following table reports the ranges of the supported floating-point data types.

Table 45: Supported Floating-Point Types Properties

Precision Name	/	Largest Value	Smallest Positive Value	Smallest Positive De-normal	Positive De-normal	Ep-silon
Bfloat16		$\approx 2^{128}$ $\approx 3.39 \cdot 10^{38}$	2^{-126} $\approx 1.18 \cdot 10^{-38}$	2^{-133}		2^{-7}
Half Precision		$\approx 2^{16}$ 65504	2^{-14} $\approx 6.1 \cdot 10^{-5}$	2^{-24}		2^{-10}
Single Precision		$\approx 2^{128}$ $\approx 3.40 \cdot 10^{38}$	2^{-126} $\approx 1.18 \cdot 10^{-38}$	2^{-149}		2^{-23}
Double Precision		$\approx 2^{1024}$ $\approx 1.8 \cdot 10^{308}$	2^{-1022} $\approx 2.22 \cdot 10^{-308}$	2^{-1074}		2^{-52}
Quad Precision		$\approx 2^{16384}$ $\approx 1.19 \cdot 10^{4932}$	2^{-16382} $\approx 3.36 \cdot 10^{-4932}$	2^{-16494}		2^{-112}

Hint

The *CUDA C++ Standard Library* provides `cuda::std::numeric_limits` in the `<cuda/std/limits>` header to query the properties and the ranges of the supported floating-point types, including *microscaling formats (MX)*. See the *C++ reference* for the list of queryable properties.

Complex numbers support:

- The *CUDA C++ Standard Library* supports complex numbers with the `cuda::std::complex` type in the `<cuda/std/complex>` header. See also the *libcu++ documentation* for more details.
- CUDA also provides basic support for complex numbers with the `cuComplex` and `cuDoubleComplex` types in the `cuComplex.h` header.

5.5.3. CUDA and IEEE-754 Compliance

All GPU devices follow the *IEEE 754-2019* standard for binary floating-point arithmetic with the following limitations:

- There is no dynamically configurable rounding mode; however, most of the operations support multiple constant IEEE rounding modes, selectable via specifically named *device intrinsics functions*.
- There is no mechanism to detect floating-point exceptions, so all operations behave as if IEEE-754 exceptions are always masked. If there is an exceptional event, the default masked response defined by IEEE-754 is delivered. For this reason, although signaling NaN SNaN encodings are supported, they are not signaling and are handled as quiet exceptions.
- Floating-point operations may alter the bit patterns of input NaN payloads. Operations such as absolute value and negation may also not comply with the IEEE 754 requirement, which could result in the sign of a NaN being updated in an implementation-defined manner.

To maximize the portability of results, users are recommended to use the default settings of the `nvcc` compiler's floating-point options: `-ftz=false`, `-prec-div=true`, and `-prec-sqrt=true`, and not

use the `--use_fast_math` option. Note that floating-point expression re-associations and contractions are allowed by default, similarly to the `--fmad=true` option. See also the [nvcc User Manual](#) for a detailed description of these compilation flags.

The IEEE-754 and C/C++ language standards do not explicitly address the conversion of a floating-point value to an integer value in cases where the rounded-to-integer value falls outside the range of the target integer format. The clamping behavior to the range of GPU devices is delineated in the [PTX ISA conversion instructions](#) section. However, compiler optimizations may leverage the unspecified behavior clause when out-of-range conversion is not invoked directly via a PTX instruction, consequently resulting in undefined behavior and an invalid CUDA program. The CUDA Math documentation issues warnings to users on a per-function/intrinsic basis. For instance, consider the `__double2int_rz()` instruction. This may differ from how host compilers and library implementations behave.

Atomic Functions Denormals Behavior:

Atomic operations have the following behavior regarding floating-point denormals, regardless of the setting of the compiler flag `-ftz`:

- ▶ Atomic single-precision floating-point adds on global memory always operate in flush-to-zero mode, namely behave equivalent to PTX `add.rn.ftz.f32` semantic.
- ▶ Atomic single-precision floating-point adds on shared memory always operate with denormal support, namely behave equivalent to PTX `add.rn.f32` semantic.

5.5.4. CUDA and C/C++ Compliance

Floating-Point Exceptions:

Unlike the host implementation, the mathematical operators and functions supported in device code do not set the global `errno` variable nor report [floating-point exceptions](#) to indicate errors. Thus, if error diagnostic mechanisms are required, users should implement additional input and output screening for the functions.

Undefined Behavior with Floating-Point Operations:

Common conditions of undefined behavior for mathematical operations include:

- ▶ Invalid arguments to mathematical operators and functions:
 - ▶ Using an uninitialized floating-point variable.
 - ▶ Using a floating-point variable outside its lifetime.
 - ▶ Signed integer overflow.
 - ▶ Dereferencing an invalid pointer.
- ▶ Floating-point specific undefined behavior:
 - ▶ Converting a floating-point value to an integer type for which the result is not representable is undefined behavior. This also includes NaN and infinity.

Users are responsible for ensuring the validity of a CUDA program. Invalid arguments may result in undefined behavior and be subject to compiler optimizations.

Contrary to integer division by zero, floating-point division by zero is not undefined behavior and not subject to compiler optimizations; rather, it is implementation-specific behavior. C++ implementations that conform to [IEC-60559](#) (IEEE-754), including CUDA, produce infinity. Note that invalid floating-point operations produce NaN and should not be misinterpreted as undefined behavior. Examples include zero divided by zero and infinity divided by infinity.

Floating-Point Literals Portability:

Both C and C++ allow for the representation of floating-point values in either decimal or hexadecimal notation. Hexadecimal floating-point literals, which are supported in C99 and C++17, denote a real value in scientific notation that can be precisely expressed in base-2. However, this does not guarantee that the literal will map to an actual value stored in a target variable (see the next paragraph). Conversely, a decimal floating-point literal may represent a numeric value that cannot be expressed in base-2.

According to the C++ standard rules, hexadecimal and decimal floating-point literals are rounded to the nearest representable value, larger or smaller, chosen in an implementation-defined manner. This rounding behavior may differ between the host and the device.

```
float f1 = 0.5f;      // 0.5, '0.5f' is a decimal floating-point literal
float f2 = 0x1p-1f;   // 0.5, '0x1p-1f' is a hexadecimal floating-point literal
float f3 = 0.1f;
// f1, f2 are represented as 0 01111110 000000000000000000000000
// f3 is represented as 0 01111011 10011001100110011001101
```

The run-time and compile-time evaluations of the same floating-point expression are subject to the following portability issues:

- ▶ The run-time evaluation of a floating-point expression may be affected by the selected rounding mode, floating-point contraction (FMA) and reassociation compiler settings, as well as floating-point exceptions. Note that CUDA does not support floating-point exceptions and the *rounding mode* is set to *round-to-nearest-ties-to-even* by default. Other rounding modes can be selected using *intrinsic functions*.
- ▶ The compiler may use a higher-precision internal representation for constant expressions.
- ▶ The compiler may perform optimizations, such as constant folding, constant propagation, and common subexpression elimination, which can lead to a different final value or comparison result.

C Standard Math Library Notes:

The host implementations of common mathematical functions are mapped to C Standard Math Library functions in a platform-specific way. These functions are provided by the host compiler and the respective host `libm`, if available.

- ▶ Functions not available from the host compilers are implemented in the `crt/math_functions.h` header file. For example, `erfinv()` is implemented there.
- ▶ Less common functions, such as `rhypot()` and `cyl_bessel_i0()`, are only available in the device code.

As previously mentioned, the host and device implementations of mathematical functions are independent. For more details on the behavior of these functions, please refer to the host implementation's documentation.

5.5.5. Floating-Point Functionality Exposure

The mathematical functions supported by CUDA are exposed through the following methods:

Built-in C/C++ language arithmetic operators:

- ▶ `x + y`, `x - y`, `x * y`, `x / y`, `x++`, `x--`, `x += y`, `x -= y`, `x *= y`, `x /= y`.

- ▶ Support single-, double-, and quad-precision types, `float`, `double`, and `__float128/_Float128` respectively.
 - ▶ `__half` and `__nv_bfloat16` types are also supported by including the `<cuda_fp16.h>` and `<cuda_bf16.h>` headers, respectively.
 - ▶ `__float128/_Float128` type support relies on the host compiler and device compute capability, see the *Supported Floating-Point Types* table.
- ▶ They are available in both host and device code.
- ▶ Their behavior is affected by the nvcc *optimization flags*.

CUDA C++ Standard Library Mathematical functions:

- ▶ Expose the full set of C++ `<cmath>` *header functions* through the `<cuda/std/cmath>` header and the `cuda::std::` namespace.
- ▶ Support IEEE-754 standard floating-point types, `__half`, `float`, `double`, `__float128`, as well as `Bfloat16` `__nv_bfloat16`.
 - ▶ `__float128` support relies on the host compiler and device compute capability, see the *Supported Floating-Point Types* table.
- ▶ They are available in both host and device code.
- ▶ They often rely on the *CUDA Math API functions*. Therefore, there could be different levels of accuracy between the host and device code.
- ▶ Their behavior is affected by the nvcc *optimization flags*.
- ▶ A subset of functionalities is also supported in constant expressions, such as `constexpr` functions, in accordance with the C++23 and C++26 standard specifications.

CUDA C Standard Library Mathematical functions (CUDA Math API):

- ▶ Expose a subset of the C `<math.h>` *header functions*.
- ▶ Support single and double-precision types, `float` and `double` respectively.
 - ▶ They are available in both host and device code.
 - ▶ They don't require additional headers.
 - ▶ Their behavior is affected by the nvcc *optimization flags*.
- ▶ A subset of the `<math.h>` header functionalities is also available for `__half`, `__nv_bfloat16`, and `__float128/_Float128` types. These functions have names that resemble those of the C Standard Library.
 - ▶ `__half` and `__nv_bfloat16` types require the `<cuda_fp16.h>` and `<cuda_bf16.h>` headers, respectively. Their host and device code availability is defined on a per-function basis.
 - ▶ `__float128/_Float128` type support relies on the host compiler and device compute capability, see the *Supported Floating-Point Types* table. The related functions require the `crt/device_fp128_functions.h` header and they are only available in device code.
- ▶ They can have a different accuracy between host and device code.

Non-standard CUDA Mathematical functions (CUDA Math API):

- ▶ Expose mathematical functionalities that are not part of the C/C++ Standard Library.
- ▶ Mainly support single- and double-precision types, `float` and `double` respectively.
 - ▶ Their host and device code availability is defined on a per-function basis.

- ▶ They don't require additional headers.
- ▶ They can have a different accuracy between host and device code.
- ▶ `__nv_bfloat16`, `__half`, `__float128`/`_Float128` are supported for a limited set of functions.
 - ▶ `__half` and `__nv_bfloat16` types require the `<cuda_fp16.h>` and `<cuda_bf16.h>` headers, respectively.
 - ▶ `__float128`/`_Float128` type support relies on the host compiler and device compute capability, see the *Supported Floating-Point Types* table. The related functions require the `crt/device_fp128_functions.h` header.
 - ▶ They are only available in device code.
- ▶ Their behavior is affected by the nvcc *optimization flags*.

Intrinsic Mathematical functions (CUDA Math API):

- ▶ Support single- and double-precision types, `float` and `double` respectively.
- ▶ They are only available in device code.
- ▶ They are faster but less accurate than the respective *CUDA Math API functions*.
- ▶ Their behavior is not affected by the nvcc *floating-point optimization flags* `-prec-div=false`, `-prec-sqrt=false`, and `-fmad=true`. The only exception is `-ftz=true`, which is also included in `-use_fast_math`.

Table 46: Summary of Math Functionality Features

Functionality	Supported Types	Host	De- vice	Affected by Floating-Point Optimization Flags (only for float and double)
<i>Built-in C/C++ language arithmetic operators</i>	float, double, __half, __nv_bfloat16, __float128/_Float128, cuda::std::complex	✓	✓	✓
<i>CUDA C++ Standard Library Mathematical functions</i>	float, double, __half, __nv_bfloat16, __float128, cuda::std::complex __nv_fp8_e4m3, __nv_fp8_e5m2, __nv_fp8_e8m0, __nv_fp6_e2m3, __nv_fp6_e3m2, __nv_fp4_e2m1*	✓	✓	✓
<i>CUDA C Standard Library Mathematical functions</i>	float, double __nv_bfloat16, __half with limited support and similar names __float128/_Float128 with limited support and similar names	✓ On a per-function basis ×	✓ ✓	✓
<i>Non-standard CUDA Mathematical functions</i>	float, double __nv_bfloat16, __float128/_Float128, __half, with limited support	 ×	 ✓	✓
<i>Intrinsic functions</i>	float, double	×	✓	Only with -ftz=true, also included in -use_fast_math

* The *CUDA C++ Standard Library functions* support queries for small floating-point types, such as `numeric_limits<T>`, `fpclassify()`, `isfinite()`, `isnormal()`, `isinf()`, and `isnan()`.

The following sections provide accuracy information for some of these functions, when applicable. It uses ULP for quantification. For more information on the definition of the [Unit in the Last Place \(ULP\)](#), please see Jean-Michel Muller's paper [On the definition of ulp\(x\)](#).

5.5.6. Built-In Arithmetic Operators

The built-in C/C++ language operators, such as $x + y$, $x - y$, $x * y$, x / y , $x++$, $x--$, and reciprocal $1 / x$, for single-, double-, and quad-precision types comply with the IEEE-754 standard. They guarantee a maximum ULP error of zero using a *round-to-nearest-ties-to-even* rounding mode. They are available in both host and device code.

The `nvcc` compilation flag `-fmad=true`, also included in `--use_fast_math`, enables contraction of

floating-point multiplies and adds/subtracts into floating-point multiply-add operations and has the following effect on the maximum ULP error for the single-precision type `float`:

- $x * y + z \rightarrow \text{__fmaf_rn}(x, y, z)$: 0 ULP

The `nvcc` compilation flag `-prec-div=false`, also included in `--use_fast_math`, has the following effect on the maximum ULP error for the division operator `/` for the single-precision type `float`:

- $x / y \rightarrow \text{__fdividef}(x, y)$: 2 ULP
- $1 / x$: 1 ULP

5.5.7. CUDA C++ Mathematical Standard Library Functions

CUDA provides comprehensive support for C++ Standard Library mathematical functions through the `cuda::std::` namespace. The functionalities are part of the `<cuda/std/cmath>` header. They are available in both host and device code.

The following sections specify the mapping with the CUDA Math APIs and the error bounds of each function when executed on the device.

- The maximum ULP error is stated as the maximum observed absolute value of the difference in ULPs between the value returned by the function and a correctly rounded result of the corresponding precision obtained according to the *round-to-nearest ties-to-even* rounding mode.
- The error bounds are derived from extensive, though not exhaustive, testing. Therefore, they are not guaranteed.

5.5.7.1 Basic Operations

CUDA Math API for basic operations are available in both host and device code, except for `__float128`.

All the following functions have a maximum ULP error of zero.

Table 47: C++ Mathematical Standard Library Functions C
Math API Mapping **Basic Operations**

cuda::std Function	Meaning	__nv_bfloat16	__half	float	double	__float128
<code>fabs(x)</code>	$ x $	<code>__habs(x)</code>	<code>__habs(x)</code>	<code>fabsf(x)</code>	<code>fabs(x)</code>	<code>__nv_fp128_fabs(x)</code>
<code>fmod(x, y)</code>	Remainder of $\frac{x}{y}$, computed as $x - \text{trunc}\left(\frac{x}{y}\right) \cdot y$	N/A	N/A	<code>fmodf(x, y)</code>	<code>fmod(x, y)</code>	<code>__nv_fp128_fmod(x, y)</code>
<code>remainder(x, y)</code>	Remainder of $\frac{x}{y}$, computed as $x - \text{rint}\left(\frac{x}{y}\right) \cdot y$	N/A	N/A	<code>remainderf(x, y)</code>	<code>remainder(x, y)</code>	<code>__nv_fp128_remainder(x, y)</code>
<code>remquo(x, y, iptr)</code>	Remainder and quotient of $\frac{x}{y}$	N/A	N/A	<code>remquof(x, y, iptr)</code>	<code>remquo(x, y, iptr)</code>	N/A
<code>fma(x, y, z)</code>	$x \cdot y + z$	<code>__hfma(x, y, z)</code> , device-only	<code>__hfma(x, y, z)</code> , device-only	<code>fmaf(x, y, z)</code>	<code>fma(x, y, z)</code>	<code>__nv_fp128_fma(x, y, z)</code>
<code>fmax(x, y)</code>	$\max(x, y)$	<code>__hmax(x, y)</code>	<code>__hmax(x, y)</code>	<code>fmaxf(x, y)</code>	<code>fmax(x, y)</code>	<code>__nv_fp128_fmax(x, y)</code>
<code>fmin(x, y)</code>	$\min(x, y)$	<code>__hmin(x, y)</code>	<code>__hmin(x, y)</code>	<code>fminf(x, y)</code>	<code>fmin(x, y)</code>	<code>__nv_fp128_fmin(x, y)</code>
<code>fdim(x, y)</code>	$\max(x - y, 0)$	N/A	N/A	<code>fdimf(x, y)</code>	<code>fdim(x, y)</code>	<code>__nv_fp128_fdim(x, y)</code>
<code>nan(str)</code>	NaN value from string representation	N/A	N/A	<code>nanf(str)</code>	<code>nan(str)</code>	N/A

* Mathematical functions marked with "N/A" are not natively available for CUDA-extended floating-point types, such as `__half` and `__nv_bfloat16`. In these cases, the function is implemented in terms of the corresponding `float` or `double` function.

5.5.7.2 Exponential Functions

CUDA Math API for exponential functions are available in both host and device code only for `float` and `double` types.

Table 48: C++ Mathematical Standard Library Functions C
Math API Mapping and Accuracy (Maximum ULP) **Exponential Functions**

cuda::std Function	Mean- ing	__nv_bf16	__half	float	double	__float128		
exp(x)	e^x	hexp(x) ULP	0	hexp(x) ULP	0	expf(x) 2 ULP	exp(x) 1 ULP	__nv_fp128_exp(x) 1 ULP
exp2(x)	2^x	hexp2(x) ULP	0	hexp2(x) ULP	0	exp2f(x) 2 ULP	exp2(x) 1 ULP	__nv_fp128_exp2(x) 1 ULP
expm1(x)	$e^x - 1$	N/A	N/A	expm1f(x) 1 ULP	expm1(x) 1 ULP	__nv_fp128_expm1(x) 1 ULP		
log(x)	$\ln(x)$	hlog(x) ULP	0	hlog(x) ULP	0	logf(x) 1 ULP	log(x) 1 ULP	__nv_fp128_log(x) 1 ULP
log10(x)	$\log_{10}(x)$	hlog10(x) ULP	0	hlog10(x) ULP	0	log10f(x) 2 ULP	log10(x) 1 ULP	__nv_fp128_log10(x) 1 ULP
log2(x)	$\log_2(x)$	hlog2(x) ULP	0	hlog2(x) ULP	0	log2f(x) 1 ULP	log2(x) 1 ULP	__nv_fp128_log2(x) 1 ULP
log1p(x)	$\ln(1 + x)$	N/A	N/A	log1pf(x) ULP	1	log1p(x) 1 ULP	__nv_fp128_log1p(x) 1 ULP	

* Mathematical functions marked with “N/A” are not natively available for CUDA-extended floating-point types, such as __half and __nv_bfloat16. In these cases, the function is implemented using a software emulation.

5.5.7.3 Power Functions

CUDA Math API for power functions are available in both host and device code only for float and double types.

Table 49: C++ Mathematical Standard Library Functions C
Math API Mapping and Accuracy (Maximum ULP) **Power Functions**

cuda::std Function	Mean- ing	__nv_bf16	__half	float	double	__float128
pow(x, y)	x^y	N/A	N/A	powf(x, y) 4 ULP	pow(x, y) 2 ULP	__nv_fp128_pow(x, y) 1 ULP
sqrt(x)	$\sqrt{x}$	hsqrt(x) 0 ULP	hsqrt(x) 0 ULP	sqrtof(x) 0 ULP with --use_fast_math	sqrt(x) 0 ULP	__nv_fp128_sqrt(x) 0 ULP
cbrt(x)	$\sqrt[3]{x}$	N/A	N/A	cbrtof(x) 1 ULP	cbrt(x) 1 ULP	N/A
hypot(x, y)	$\sqrt{x^2 + y^2}$	N/A	N/A	hypotf(x, y) 3 ULP	hypot(x, y) 2 ULP	__nv_fp128_hypot(x, y) 1 ULP

* Mathematical functions marked with “N/A” are not natively available for CUDA-extended floating-point types, such as __half and __nv_bfloat16. In these cases, the function is implemented using a software emulation.

5.5.7.4 Trigonometric Functions

CUDA Math API for trigonometric functions are available in both host and device code only for `float` and `double` types.

Table 50: C++ Mathematical Standard Library Functions C Math API Mapping and Accuracy (Maximum ULP) **Trigonometric Functions**

cuda::std Function	Mean- ing	__nv_bfloat16	__half	float	double	__float128
<code>sin(x)</code>	$\sin(x)$	<code>hsin(x)</code> ULP	0	<code>hsin(x)</code> ULP	<code>sinf(x)</code> 2 ULP	<code>sin(x)</code> 2 ULP <code>__nv_fp128_sin(x)</code> 1 ULP
<code>cos(x)</code>	$\cos(x)$	<code>hcos(x)</code> ULP	0	<code>hcos(x)</code> ULP	<code>cosf(x)</code> 2 ULP	<code>cos(x)</code> 2 ULP <code>__nv_fp128_cos(x)</code> 1 ULP
<code>tan(x)</code>	$\tan(x)$	N/A	N/A	<code>tanf(x)</code> ULP	4 ULP <code>tan(x)</code> 2 ULP	<code>__nv_fp128_tan(x)</code> 1 ULP
<code>asin(x)</code>	$\sin^{-1}(x)$	N/A	N/A	<code>asinf(x)</code> ULP	2 ULP <code>asin(x)</code> 2 ULP	<code>__nv_fp128_asin(x)</code> 1 ULP
<code>acos(x)</code>	$\cos^{-1}(x)$	N/A	N/A	<code>acosf(x)</code> ULP	2 ULP <code>acos(x)</code> 2 ULP	<code>__nv_fp128_acos(x)</code> 1 ULP
<code>atan(x)</code>	$\tan^{-1}(x)$	N/A	N/A	<code>atanf(x)</code> ULP	2 ULP <code>atan(x)</code> 2 ULP	<code>__nv_fp128_atan(x)</code> 1 ULP
<code>atan2(y, x)</code>	$\tan^{-1}\left(\frac{y}{x}\right)$	N/A	N/A	<code>atan2f(y, x)</code> 3 ULP	2 ULP <code>atan2(y, x)</code> 2 ULP	N/A

* Mathematical functions marked with “N/A” are not natively available for CUDA-extended floating-point types, such as `__half` and `__nv_bfloat16`. In these cases, the functions are implemented using a software emulation.

5.5.7.5 Hyperbolic Functions

CUDA Math API for hyperbolic functions are available in both host and device code only for `float` and `double` types.

Table 51: C++ Mathematical Standard Library Functions C
Math API Mapping and Accuracy (Maximum ULP) **Hyperbolic Functions**

cuda::std Function	Mean- ing	__nv_bf16	__half	float	double	__float128
sinh(x)	$\sinh(x)$	N/A	N/A	sinhf(x) ULP	3 ULP	sinh(x) 2 ULP
cosh(x)	$\cosh(x)$	N/A	N/A	coshf(x) ULP	2 ULP	cosh(x) 1 ULP
tanh(x)	$\tanh(x)$	htanh(x) ULP	0 ULP	tanhf(x) ULP	2 ULP	tanh(x) 1 ULP
asinh(x)	$\sinh^{-1}(x)$	N/A	N/A	asinhf(x) ULP	3 ULP	asinh(x) 3 ULP
acosh(x)	$\cosh^{-1}(x)$	N/A	N/A	acoshf(x) ULP	4 ULP	acosh(x) 3 ULP
atanh(x)	$\tanh^{-1}(x)$	N/A	N/A	atanhf(x) ULP	3 ULP	atanh(x) 2 ULP

* Mathematical functions marked with “N/A” are not natively available for CUDA-extended floating-point types, such as __half and __nv_bfloat16. In these cases, the functions are emulated by converting to a float type and then converting the result back.

5.5.7.6 Error and Gamma Functions

CUDA Math API for error and gamma functions are available in both host and device code for float and double types.

Error and Gamma functions are not natively available for CUDA-extended floating-point types, such as __half and __nv_bfloat16. In these cases, the functions are emulated by converting to a float type and then converting the result back.

Table 52: C++ Mathematical Standard Library Functions C
Math API Mapping and Accuracy (Maximum ULP) **Error and Gamma Functions**

cuda::std Function	Mean- ing	float	double
erf(x)	$\frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$	erff(x) 2 ULP	erf(x) 2 ULP
erfc(x)	$1 - \text{erf}(x)$	erfcf(x) 4 ULP	erfc(x) 5 ULP
tgamma(x)	$\Gamma(x)$	tgammaf(x) 5 ULP	tgamma(x) 10 ULP
lgamma(x)	$\ln \Gamma(x) $	lgammaf(x) • 6 ULP for $x \notin [-10.001, -2.264]$ • larger otherwise	lgamma(x) • 4 ULP for $x \notin [-23.0001, -2.2637]$ • larger otherwise

5.5.7.7 Nearest Integer Floating-Point Operations

CUDA Math API for nearest integer floating-point operations are available in both host and device code only for float and double types.

All the following functions have a maximum ULP error of zero.

Table 53: C++ Mathematical Standard Library Functions C Math API Mapping **Nearest Integer Floating-Point Operations**

cuda::std Function	Meaning	__nv_bfloat16	__half	float	double	__float128
ceil(x)	$\lceil x \rceil$	hceil(x)	hceil(x)	ceilf(x)	ceil(x)	__nv_fp128_ceil(x)
floor(x)	$\lfloor x \rfloor$	hfloor(x)	hfloor(x)	floorf(x)	floor(x)	__nv_fp128_floor(x)
trunc(x)	Truncate to integer	htrunc(x)	htrunc(x)	truncf(x)	trunc(x)	__nv_fp128_trunc(x)
round(x)	Round to nearest integer, ties away from zero	N/A	N/A	roundf(x)	round(x)	__nv_fp128_round(x)
nearbyint(x)	Round to nearest integer, ties to even	N/A	N/A	nearbyintf(x)	nearbyint(x)	N/A
rint(x)	Round to nearest integer, ties to even	hrint(x)	hrint(x)	rintf(x)	rint(x)	__nv_fp128_rint(x)
lrint(x)	Round to nearest integer, ties to even (returns long int)	N/A	N/A	lrintf(x)	lrint(x)	N/A
llrint(x)	Round to nearest integer, ties to even (returns long long int)	N/A	N/A	llrintf(x)	llrint(x)	N/A
lround(x)	Round to nearest integer, ties away from zero (returns long int)	N/A	N/A	lroundf(x)	lround(x)	N/A
llround(x)	Round to nearest integer, ties away from zero (returns long long int)	N/A	N/A	llroundf(x)	llround(x)	N/A

* Mathematical functions marked with “N/A” are not natively available for CUDA-extended floating-point types, such as __half and __nv_bfloat16. In these cases, the functions are emulated by converting to a float type and then converting the result back.

Performance Considerations

The recommended way to round a single- or double-precision floating-point operand to an integer is to use the functions rintf() and rint(), not roundf() and round(). This is because roundf() and round() map to multiple instructions in device code, whereas rintf() and rint() map to a single instruction. truncf(), trunc(), ceilf(), ceil(), floorf(), and floor() each map to a single instruction as well.

5.5.7.8 Floating-Point Manipulation Functions

CUDA Math API for floating-point manipulation functions are available in both host and device code, except for __float128.

Floating-point manipulation functions are not natively available for CUDA-extended floating-point types, such as __half and __nv_bfloat16. In these cases, the functions are emulated by converting to a float type and then converting the result back.

All the following functions have a maximum ULP error of zero.

Table 54: C++ Mathematical Standard Library Functions C
Math API Mapping **Floating-Point Manipulation Functions**

<code>cuda::std</code> Function	Meaning	<code>float</code>	<code>double</code>	<code>__float128</code>
<code>frexp(x, exp)</code>	Extract mantissa and exponent	<code>frexpf(x, exp)</code>	<code>frexp(x, exp)</code>	<code>__nv_fp128_frexp(x, nptr)</code>
<code>ldexp(x, n)</code>	$x \cdot 2^n$	<code>ldexpf(x, n)</code>	<code>ldexp(x, n)</code>	<code>__nv_fp128_ldexp(x, n)</code>
<code>modf(x, iptr)</code>	Extract integer and fractional parts	<code>modff(x, iptr)</code>	<code>modf(x, iptr)</code>	<code>__nv_fp128_modf(x, iptr)</code>
<code>scalbn(x, n)</code>	$x \cdot 2^n$	<code>scalbnf(x, n)</code>	<code>scalbn(x, n)</code>	N/A
<code>scalbln(x, n)</code>	$x \cdot 2^n$	<code>scalblnf(x, n)</code>	<code>scalbln(x, n)</code>	N/A
<code>ilogb(x)</code>	$\lfloor \log_2(x) \rfloor$	<code>ilogbf(x)</code>	<code>ilogb(x)</code>	<code>__nv_fp128_ilogb(x)</code>
<code>logb(x)</code>	$\lfloor \log_2(x) \rfloor$	<code>logbf(x)</code>	<code>logb(x)</code>	N/A
<code>nextafter(x, y)</code>	Next representable value toward y	<code>nextafterf(x, y)</code>	<code>nextafter(x, y)</code>	N/A
<code>copysign(x, y)</code>	Copy sign of y to x	<code>copysignf(x, y)</code>	<code>copysign(x, y)</code>	<code>__nv_fp128_copysign(x, y)</code>

5.5.7.9 Classification and Comparison

CUDA Math API for classification and comparison functions are available in both host and device code, except for `__float128`.

All the following functions have a maximum ULP error of zero.

Table 55: C++ Mathematical Standard Library Functions C
Math API Mapping **Classification and Comparison Functions**

cuda::std Function	Meaning	__nv_bfloat16	__half	float	double	__float128
fpclassify(x)	Classify x	N/A	N/A	N/A	N/A	N/A
isfinite(x)	Check if x is finite	N/A	N/A	isfinite(x)	isfinite(x)	N/A
isinf(x)	Check if x is infinite	__hisinf(x)	__hisinf(x)	isinf(x)	isinf(x)	N/A
isnan(x)	Check if x is NaN	__hisnan(x)	__hisnan(x)	isnan(x)	isnan(x)	__nv_fp128_isnan(x)
isnormal(x)	Check if x is normal	N/A	N/A	N/A	N/A	N/A
signbit(x)	Check if sign bit is set	N/A	N/A	signbit(x)	signbit(x)	N/A
isgreater(x, y)	Check if $x > y$	__hgt(x, y)	__hgt(x, y)	N/A	N/A	N/A
is-greater-equal(x, y)	Check if $x \geq y$	__hge(x, y)	__hge(x, y)	N/A	N/A	N/A
isless(x, y)	Check if $x < y$	__hlt(x, y)	__hlt(x, y)	N/A	N/A	N/A
islessequal(x, y)	Check if $x \leq y$	__hle(x, y)	__hle(x, y)	N/A	N/A	N/A
isless-greater(x, y)	Check if $x < y$ or $x > y$	__hne(x, y)	__hne(x, y)	N/A	N/A	N/A
isunordered(x, y)	Check if x, y , or both are NaN	N/A	N/A	N/A	N/A	__nv_fp128_isunordered(x, y)

* Mathematical functions marked with “N/A” are not natively available for CUDA-extended floating-point types, such as __half and __nv_bfloat16.

5.5.8. Non-Standard CUDA Mathematical Functions

CUDA provides mathematical functions that are not part of the C/C++ Standard Library and are instead offered as extensions. For single- and double-precision functions, host and device code availability is defined on a per-function basis.

This section specifies the error bounds of each function when executed on the device.

- The maximum ULP error is stated as the maximum observed absolute value of the difference in ULPs between the value returned by the function and a correctly rounded result of the corresponding precision obtained according to the *round-to-nearest ties-to-even* rounding mode.
- The error bounds are derived from extensive, though not exhaustive, testing. Therefore, they are not guaranteed.

Table 56: **Non-standard CUDA Mathematical functions** float and double Mapping and Accuracy (Maximum ULP)

Meaning	float	double
$\frac{x}{y}$	<code>fdivdef(x, y)</code> , device-only 0 ULP, same as <code>x / y</code>	N/A
10^x	<code>exp10f(x)</code> 2 ULP	<code>exp10(x)</code> 1 ULP
$\sqrt{x^2 + y^2 + z^2}$	<code>norm3df(x, y, z)</code> , device-only 3 ULP	<code>norm3d(x, y, z)</code> , device-only 2 ULP
$\sqrt{x^2 + y^2 + z^2 + t^2}$	<code>norm4df(x, y, z, t)</code> , device-only 3 ULP	<code>norm4d(x, y, z, t)</code> , device-only 2 ULP
$\sqrt{\sum_{i=0}^{\text{dim}-1} p_i^2}$	<code>normf(dim, p)</code> , device-only An error bound cannot be provided because a fast algorithm is used with accuracy loss due to round-off	<code>norm(dim, p)</code> , device-only An error bound cannot be provided because a fast algorithm is used with accuracy loss due to round-off
$\frac{1}{\sqrt{x}}$	<code>rsqrtf(x)</code> 2 ULP	<code>rsqrt(x)</code> 1 ULP
$\frac{1}{\sqrt[3]{x}}$	<code>rcbrtf(x)</code> 1 ULP	<code>rcbrt(x)</code> 1 ULP
$\frac{1}{\sqrt{x^2 + y^2}}$	<code>rhypotf(x, y)</code> , device-only 2 ULP	<code>rhypot(x, y)</code> , device-only 1 ULP
$\frac{1}{\sqrt{x^2 + y^2 + z^2}}$	<code>rnorm3df(x, y, z)</code> , device-only 2 ULP	<code>rnorm3d(x, y, z)</code> , device-only 1 ULP
$\frac{1}{\sqrt{x^2 + y^2 + z^2 + t^2}}$	<code>rnorm4df(x, y, z, t)</code> , device-only 2 ULP	<code>rnorm4d(x, y, z, t)</code> , device-only 1 ULP
$\frac{1}{\sqrt{\sum_{i=0}^{\text{dim}-1} p_i^2}}$	<code>rnormf(dim, p)</code> , device-only An error bound cannot be provided because a fast algorithm is used with accuracy loss due to round-off	<code>rnorm(dim, p)</code> , device-only An error bound cannot be provided because a fast algorithm is used with accuracy loss due to round-off
$\cos(\pi x)$	<code>cospif(x)</code> 1 ULP	<code>cospi(x)</code> 2 ULP
$\sin(\pi x)$	<code>sinpif(x)</code> 1 ULP	<code>sinpi(x)</code> 2 ULP
$\sin(\pi x), \cos(\pi x)$	<code>sincospif(x, sptr, cptr)</code> 1 ULP	<code>sincospi(x, sptr, cptr)</code> 2 ULP
$\Phi(x)$	<code>normcdf(x)</code> 5 ULP	<code>normcdf(x)</code> 5 ULP
$\Phi^{-1}(x)$	<code>normcdfinvf(x)</code> 5 ULP	<code>normcdfinv(x)</code> 8 ULP
$\text{erfc}^{-1}(x)$	<code>erfcinvf(x)</code> 4 ULP	<code>erfcinv(x)</code> 6 ULP
$e^{x^2} \text{erfc}(x)$	<code>erfcxf(x)</code> 4 ULP	<code>erfcx(x)</code> 4 ULP
$\text{erf}^{-1}(x)$	<code>erfinvf(x)</code> 2 ULP	<code>erfinv(x)</code> 5 ULP
$I_0(x)$	<code>cyl_bessel_i0f(x)</code> , device-only 6 ULP	<code>cyl_bessel_i0(x)</code> , device-only 6 ULP
$I_1(x)$	<code>cyl_bessel_i1f(x)</code> , device-only 6 ULP	<code>cyl_bessel_i1(x)</code> , device-only 6 ULP
$J_0(x)$	<code>j0f(x)</code> • 9 ULP for $ x < 8$ • the maximum absolute error = $2.2 \cdot 10^{-6}$, otherwise	<code>j0(x)</code> • 7 ULP for $ x < 8$ • the maximum absolute error = $5 \cdot 10^{-12}$, otherwise
$J_1(x)$	<code>j1f(x)</code> • 9 ULP for $ x < 8$ • the maximum absolute error = $2.2 \cdot 10^{-6}$, otherwise	<code>j1(x)</code> • 7 ULP for $ x < 8$ • the maximum absolute error = $5 \cdot 10^{-12}$, otherwise
$J_n(x)$	<code>jnf(n, x)</code> For $n = 128$, the maximum absolute error = $2.2 \cdot 10^{-6}$	<code>jn(n, x)</code> For $n = 128$, the maximum absolute error = $5 \cdot 10^{-12}$
$Y_0(x)$	<code>y0f(x)</code> • 9 ULP for $ x < 8$ • the maximum absolute error = $2.2 \cdot 10^{-6}$, otherwise	<code>y0(x)</code> • 7 ULP for $ x < 8$ • the maximum absolute error = $5 \cdot 10^{-12}$, otherwise

5.5. Floating-Point Computation

$Y_1(x)$	<code>y1f(x)</code> • 9 ULP for $ x < 8$ • the maximum absolute error = $2.2 \cdot 10^{-6}$, otherwise	<code>y1(x)</code> • 7 ULP for $ x < 8$ • the maximum absolute error = $5 \cdot 10^{-12}$, otherwise
----------	--	--

Non-standard CUDA Mathematical functions for `__half`, `__nv_bfloat16`, and `__float128/_Float128` are only available in device code.

Table 57: **Non-standard CUDA Mathematical functions**
`__nv_bfloat16`, `__half`, `__float128/_Float128` Mapping
 and Accuracy (Maximum ULP)

Meaning	<code>__nv_bfloat16</code>	<code>__half</code>	<code>__float128/_Float128</code>
$\frac{1}{x}$	<code>hrcp(x)</code> 0 ULP	<code>hrcp(x)</code> 0 ULP	N/A
10^x	<code>hexp10(x)</code> 0 ULP	<code>hexp10(x)</code> 0 ULP	<code>__nv_fp128_exp10(x)</code> 1 ULP
$\frac{1}{\sqrt{x}}$	<code>hrsqrt(x)</code> 0 ULP	<code>hrsqrt(x)</code> 0 ULP	N/A
$\tanh(x)$ (approximate)	<code>htanh_approx(x)</code> 1 ULP	<code>htanh_approx(x)</code> 1 ULP	N/A

5.5.9. Intrinsic Functions

Intrinsic mathematical functions are faster and less accurate versions of their corresponding [CUDA C Standard Library Mathematical functions](#).

- ▶ They have the same name prefixed with `__`, such as `__sinf(x)`.
- ▶ They are only available in device code.
- ▶ They are faster because they map to fewer native instructions.
- ▶ The flag `--use_fast_math` automatically translates the corresponding [CUDA Math API functions](#) into intrinsic functions. See the [-use_fast_math Effect](#) section for the full list of affected functions.

5.5.9.1 Basic Intrinsic Functions

A subset of mathematical intrinsic functions allow specifying the rounding mode:

- ▶ Functions suffixed with `_rn` operate using the *round to nearest even* rounding mode.
- ▶ Functions suffixed with `_rz` operate using the *round towards zero* rounding mode.
- ▶ Functions suffixed with `_ru` operate using the *round up* (toward positive infinity) rounding mode.
- ▶ Functions suffixed with `_rd` operate using the *round down* (toward negative infinity) rounding mode.

The `__fadd_[rn,rz,ru,rd]()`, `__dadd_[rn,rz,ru,rd]()`, `__fmul_[rn,rz,ru,rd]()`, and `__dmul_[rn,rz,ru,rd]()` functions map to addition and multiplication operations that the compiler never merges into the FFMA or DFMA instructions. In contrast, additions and multiplications generated from the `*` and `+` operators are often combined into FFMA or DFMA.

The following table lists the single- and double-precision floating-point intrinsic functions. All of them have a maximum ULP error of 0 and are IEEE-compliant.

Table 58: Single- and Double-Precision Floating-Point Intrinsic Functions

Meaning	float	double
$x + y$	<code>__fadd_[rn,rz,ru,rd](x, y)</code>	<code>__dadd_[rn,rz,ru,rd](x, y)</code>
$x - y$	<code>__fsub_[rn,rz,ru,rd](x, y)</code>	<code>__dsub_[rn,rz,ru,rd](x, y)</code>
$x \cdot y$	<code>__fmul_[rn,rz,ru,rd](x, y)</code>	<code>__dmul_[rn,rz,ru,rd](x, y)</code>
$x \cdot y + z$	<code>__fmaf_[rn,rz,ru,rd](x, y, z)</code>	<code>__fma_[rn,rz,ru,rd](x, y, z)</code>
$\frac{x}{y}$	<code>__fdiv_[rn,rz,ru,rd](x, y)</code>	<code>__ddiv_[rn,rz,ru,rd](x, y)</code>
$\frac{1}{x}$	<code>__frcp_[rn,rz,ru,rd](x)</code>	<code>__drcp_[rn,rz,ru,rd](x)</code>
$\sqrt{x}$	<code>__fsqrt_[rn,rz,ru,rd](x)</code>	<code>__dsqrt_[rn,rz,ru,rd](x)</code>

5.5.9.2 Single-Precision-Only Intrinsic Functions

The following table lists the single-precision floating-point intrinsic functions with their maximum ULP error.

- The maximum ULP error is stated as the maximum observed absolute value of the difference in ULPs between the value returned by the function and a correctly rounded result of the corresponding precision obtained according to the *round-to-nearest ties-to-even* rounding mode.
- The error bounds are derived from extensive, though not exhaustive, testing. Therefore, they are not guaranteed.

Table 59: **Single-Precision Only Floating-Point Intrinsic Functions** Mapping and Accuracy (Maximum ULP)

Function	Meaning	Maximum ULP Error
<code>__fdividef(x, y)</code>	$\frac{x}{y}$	2 for $ y \in [2^{-126}, 2^{126}]$
<code>__frsqrt_rn(x)</code>	$\frac{1}{\sqrt{x}}$	0 ULP
<code>__expf(x)</code>	e^x	$2 + \lfloor 1.173 \cdot x \rfloor$
<code>__exp10f(x)</code>	10^x	$2 + \lfloor 2.97 \cdot x \rfloor$
<code>__powf(x, y)</code>	x^y	Derived from <code>exp2f(y * __log2f(x))</code>
<code>__logf(x)</code>	$\ln(x)$	• $2^{-21.41}$ abs error for $x \in [0.5, 2]$ • 3 ULP, otherwise
<code>__log2f(x)</code>	$\log_2(x)$	• 2^{-22} abs error for $x \in [0.5, 2]$ • 2 ULP, otherwise
<code>__log10f(x)</code>	$\log_{10}(x)$	• 2^{-24} abs error for $x \in [0.5, 2]$ • 3 ULP, otherwise
<code>__sinf(x)</code>	$\sin(x)$	• $2^{-21.41}$ abs error for $x \in [-\pi, \pi]$ • larger otherwise
<code>__cosf(x)</code>	$\cos(x)$	• $2^{-21.41}$ abs error for $x \in [-\pi, \pi]$ • larger otherwise
<code>__sincosf(x, sptr, cptr)</code>	$\sin(x), \cos$	Component-wise, the same as <code>__sinf(x)</code> and <code>__cosf(x)</code>
<code>__tanf(x)</code>	$\tan(x)$	Derived from <code>__sinf(x) * (1 / __cosf(x))</code>
<code>__tanhf(x)</code>	$\tanh(x)$	• Max relative error: 2^{-11} • Subnormal results are not flushed to zero even under <code>-ftz=true</code> compiler flag.

5.5.9.3 --use_fast_math Effect

The nvcc compiler flag `--use_fast_math` translates a subset of [CUDA Math API functions](#) called in device code into their intrinsic counterpart. Note that the [CUDA C++ Standard Library functions](#) are also affected by this flag. See the [Intrinsic Functions](#) section for more details on the implications of using intrinsic functions instead of CUDA Math API functions.

A more robust approach is to selectively replace mathematical function calls with intrinsic versions only where the performance gains justify it and where the changed properties, such as reduced accuracy and different special-case handling, are acceptable.

Table 60: Functions Directly Affected by `--use_fast_math`

Device Function	Intrinsic Function
<code>x/y, fdividef(x, y)</code>	<code>__fdividef(x, y)</code>
<code>sinf(x)</code>	<code>__sinf(x)</code>
<code>cosf(x)</code>	<code>__cosf(x)</code>
<code>tanf(x)</code>	<code>__tanf(x)</code>
<code>sincosf(x, sptr, cptr)</code>	<code>__sincosf(x, sptr, cptr)</code>
<code>logf(x)</code>	<code>__logf(x)</code>
<code>log2f(x)</code>	<code>__log2f(x)</code>
<code>log10f(x)</code>	<code>__log10f(x)</code>
<code>expf(x)</code>	<code>__expf(x)</code>
<code>exp10f(x)</code>	<code>__exp10f(x)</code>
<code>powf(x,y)</code>	<code>__powf(x,y)</code>
<code>tanhf(x)</code>	<code>__tanhf(x)</code>

5.5.10. References

1. IEEE 754-2019 Standard for Floating-Point Arithmetic.
2. Jean-Michel Muller. *On the definition of ulp(x)*. INRIA/LIP research report, 2005.
3. Nathan Whitehead, Alex Fit-Florea. *Precision & Performance: Floating Point and IEEE 754 Compliance for NVIDIA GPUs*. Nvidia Report, 2011.
4. David Goldberg. *What every computer scientist should know about floating-point arithmetic*. ACM Computing Surveys, March 1991.
5. David Monniaux. *The pitfalls of verifying floating-point computations*. ACM Transactions on Programming Languages and Systems, May 2008.
6. Peter Dinda, Conor Hetland. *Do Developers Understand IEEE Floating Point?*. IEEE International Parallel and Distributed Processing Symposium (IPDPS), 2018.

5.6. Device-Callable APIs and Intrinsics

This chapter contains reference material and API documentation for APIs and intrinsics which can be called from CUDA kernels and device code.

5.6.1. Memory Barrier Primitives Interface

The primitives API is a C-like interface to `cuda::barrier` functionality. These primitives are available by including the `<cuda_awbarrier_primitives.h>` header.

5.6.1.1 Data Types

```
typedef /* implementation defined */ __mbarrier_t;  
typedef /* implementation defined */ __mbarrier_token_t;
```

5.6.1.2 Memory Barrier Primitives API

```
uint32_t __mbarrier_maximum_count();  
void __mbarrier_init(__mbarrier_t* bar, uint32_t expected_count);
```

- ▶ `bar` must be a pointer to `__shared__` memory.
- ▶ `expected_count <= __mbarrier_maximum_count()`
- ▶ Initialize `*bar` expected arrival count for the current and next phase to `expected_count`.

```
void __mbarrier_inval(__mbarrier_t* bar);
```

- ▶ `bar` must be a pointer to the barrier object residing in shared memory.
- ▶ Invalidation of `*bar` is required before the corresponding shared memory can be repurposed.

```
__mbarrier_token_t __mbarrier_arrive(__mbarrier_t* bar);
```

- ▶ Initialization of `*bar` must happen before this call.
- ▶ Pending count must not be zero.
- ▶ Atomically decrement the pending count for the current phase of the barrier.
- ▶ Return an arrival token associated with the barrier state immediately prior to the decrement.

```
__mbarrier_token_t __mbarrier_arrive_and_drop(__mbarrier_t* bar);
```

- ▶ Initialization of `*bar` must happen before this call.
- ▶ Pending count must not be zero.
- ▶ Atomically decrement the pending count for the current phase and expected count for the next phase of the barrier.
- ▶ Return an arrival token associated with the barrier state immediately prior to the decrement.

```
bool __mbarrier_test_wait(__mbarrier_t* bar, __mbarrier_token_t token);
```

- ▶ `token` must be associated with the immediately preceding phase or current phase of `*bar`.
- ▶ Returns `true` if `token` is associated with the immediately preceding phase of `*bar`, otherwise returns `false`.

```
bool __mbarrier_test_wait_parity(__mbarrier_t* bar, bool phase_parity);
```

- ▶ `phase_parity` must indicate the parity of either the current phase or the immediately preceding phase of `*bar`. A value of `true` corresponds to odd-numbered phases and a value of `false` corresponds to even-numbered phases.
- ▶ Returns `true` if `phase_parity` indicates the integer parity of the immediately preceding phase of `*bar`, otherwise returns `false`.

```
bool __mbarrier_try_wait(__mbarrier_t* bar, __mbarrier_token_t token, uint32_t
    ↪ max_sleep_nanosec);
```

- ▶ token must be associated with the immediately preceding phase or current phase of *bar.
- ▶ Returns true if token is associated with the immediately preceding phase of *bar. Otherwise, the executing thread may be suspended. Suspended thread resumes execution when the specified phase completes (returns true) OR before the phase completes following a system-dependent time limit (returns false).
- ▶ max_sleep_nanosec specifies the time limit, in nanoseconds, that may be used for the time limit instead of the system-dependent limit.

```
bool __mbarrier_try_wait_parity(__mbarrier_t* bar, bool phase_parity, uint32_t
    ↪ max_sleep_nanosec);
```

- ▶ phase_parity must indicate the parity of either the current phase or the immediately preceding phase of *bar. A value of true corresponds to odd-numbered phases and a value of false corresponds to even-numbered phases.
- ▶ Returns true if phase_parity indicates the integer parity of the immediately preceding phase of *bar. Otherwise, the executing thread may be suspended. Suspended thread resumes execution when the specified phase completes (returns true) OR before the phase completes following a system-dependent time limit (returns false).
- ▶ max_sleep_nanosec specifies the time limit, in nanoseconds, that may be used for the time limit instead of the system-dependent limit.

5.6.2. Pipeline Primitives Interface

Pipeline primitives provide a C-like interface for the functionality available in <cuda/pipeline>. The pipeline primitives interface is available by including the <cuda_pipeline.h> header. When compiling without ISO C++ 2011 compatibility, include the <cuda_pipeline_primitives.h> header.

Note

The pipeline primitives API only supports tracking asynchronous copies from global memory to shared memory with specific size and alignment requirements. It provides equivalent functionality to a `cuda::pipeline` object with `cuda::thread_scope_thread`.

5.6.2.1 memcpy_async Primitive

```
void __pipeline_memcpy_async(void* __restrict__ dst_shared,
    const void* __restrict__ src_global,
    size_t size_and_align,
    size_t zfill=0);
```

- ▶ Request that the following operation be submitted for asynchronous evaluation:

```
size_t i = 0;
for (; i < size_and_align - zfill; ++i) ((char*)dst_shared)[i] =
    ↪((char*)src_global)[i]; /* copy */
```

(continues on next page)

(continued from previous page)

```
for (; i < size_and_align; ++i) ((char*)dst_shared)[i] = 0; /* zero-fill
→ */
```

- ▶ Requirements:
 - ▶ `dst_shared` must be a pointer to the shared memory destination for the `memcpy_async`.
 - ▶ `src_global` must be a pointer to the global memory source for the `memcpy_async`.
 - ▶ `size_and_align` must be 4, 8, or 16.
 - ▶ `zfill <= size_and_align`.
 - ▶ `size_and_align` must be the alignment of `dst_shared` and `src_global`.
- ▶ It is a race condition for any thread to modify the source memory or observe the destination memory prior to waiting for the `memcpy_async` operation to complete. Between submitting a `memcpy_async` operation and waiting for its completion, any of the following actions introduces a race condition:
 - ▶ Loading from `dst_shared`.
 - ▶ Storing to `dst_shared` or `src_global`.
 - ▶ Applying an atomic update to `dst_shared` or `src_global`.

5.6.2.2 Commit Primitive

```
void __pipeline_commit();
```

- ▶ Commit submitted `memcpy_async` to the pipeline as the current batch.

5.6.2.3 Wait Primitive

```
void __pipeline_wait_prior(size_t N);
```

- ▶ Let {0, 1, 2, ..., L} be the sequence of indices associated with invocations of `__pipeline_commit()` by a given thread.
- ▶ Wait for completion of batches *at least* up to and including L-N.

5.6.2.4 Arrive On Barrier Primitive

```
void __pipeline_arrive_on(__mbarrier_t* bar);
```

- ▶ `bar` points to a barrier in shared memory.
- ▶ Increments the barrier arrival count by one, when all `memcpy_async` operations sequenced before this call have completed, the arrival count is decremented by one and hence the net effect on the arrival count is zero. It is user's responsibility to make sure that the increment on the arrival count does not exceed `__mbarrier_maximum_count()`.

5.6.3. Cooperative Groups API

5.6.3.1 cooperative_groups.h

5.6.3.1.1 class thread_block

Any CUDA programmer is already familiar with a certain group of threads: the thread block. The Cooperative Groups extension introduces a new datatype, `thread_block`, to explicitly represent this concept within the kernel.

```
class thread_block;
```

Constructed via:

```
thread_block g = this_thread_block();
```

Public Member Functions:

`static void sync():` Synchronize the threads named in the group, equivalent to `g.barrier_wait(g.barrier_arrive())`

`thread_block::arrival_token barrier_arrive():` Arrive on the `thread_block` barrier, returns a token that needs to be passed into `barrier_wait()`.

`void barrier_wait(thread_block::arrival_token&& t):` Wait on the `thread_block` barrier, takes arrival token returned from `barrier_arrive()` as an rvalue reference.

`static unsigned int thread_rank():` Rank of the calling thread within `[0, num_threads)`

`static dim3 group_index():` 3-Dimensional index of the block within the launched grid

`static dim3 thread_index():` 3-Dimensional index of the thread within the launched block

`static dim3 dim_threads():` Dimensions of the launched block in units of threads

`static unsigned int num_threads():` Total number of threads in the group

Legacy member functions (aliases):

`static unsigned int size():` Total number of threads in the group (alias of `num_threads()`)

`static dim3 group_dim():` Dimensions of the launched block (alias of `dim_threads()`)

Example:

```
/// Loading an integer from global into shared memory
__global__ void kernel(int *globalInput) {
    __shared__ int x;
    thread_block g = this_thread_block();
    // Choose a leader in the thread block
    if (g.thread_rank() == 0) {
        // load from global into shared for all threads to work with
        x = (*globalInput);
    }
    // After loading data into shared memory, you want to synchronize
    // if all threads in your thread block need to see it
    g.sync(); // equivalent to __syncthreads();
}
```

Note: that all threads in the group must participate in collective operations, or the behavior is undefined.

Related: The `thread_block` datatype is derived from the more generic `thread_group` datatype, which can be used to represent a wider class of groups.

5.6.3.1.2 class `cluster_group`

This group object represents all the threads launched in a single cluster. The APIs are available on all hardware with Compute Capability 9.0+. In such cases, when a non-cluster grid is launched, the APIs assume a 1x1x1 cluster.

`class cluster_group;`

Constructed via:

```
cluster_group g = this_cluster();
```

Public Member Functions:

`static void sync():` Synchronize the threads named in the group, equivalent to `g.barrier_wait(g.barrier_arrive())`

`static cluster_group::arrival_token barrier_arrive():` Arrive on the cluster barrier, returns a token that needs to be passed into `barrier_wait()`.

`static void barrier_wait(cluster_group::arrival_token&& t):` Wait on the cluster barrier, takes arrival token returned from `barrier_arrive()` as a rvalue reference.

`static unsigned int thread_rank():` Rank of the calling thread within `[0, num_threads)`

`static unsigned int block_rank():` Rank of the calling block within `[0, num_blocks)`

`static unsigned int num_threads():` Total number of threads in the group

`static unsigned int num_blocks():` Total number of blocks in the group

`static dim3 dim_threads():` Dimensions of the launched cluster in units of threads

`static dim3 dim_blocks():` Dimensions of the launched cluster in units of blocks

`static dim3 block_index():` 3-Dimensional index of the calling block within the launched cluster

`static unsigned int query_shared_rank(const void *addr):` Obtain the block rank to which a shared memory address belongs

`static T* map_shared_rank(T *addr, int rank):` Obtain the address of a shared memory variable of another block in the cluster

Legacy member functions (aliases):

`static unsigned int size():` Total number of threads in the group (alias of `num_threads()`)

5.6.3.1.3 class `grid_group`

This group object represents all the threads launched in a single grid. APIs other than `sync()` are available at all times, but to be able to synchronize across the grid, you need to use the cooperative launch API.

`class grid_group;`

Constructed via:

```
grid_group g = this_grid();
```

Public Member Functions:

`bool is_valid() const`: Returns whether the `grid_group` can synchronize

`void sync() const`: Synchronize the threads named in the group, equivalent to `g.barrier_wait(g.barrier_arrive())`

`grid_group::arrival_token barrier_arrive()`: Arrive on the grid barrier, returns a token that needs to be passed into `barrier_wait()`.

`void barrier_wait(grid_group::arrival_token&& t)`: Wait on the grid barrier, takes arrival token returned from `barrier_arrive()` as a rvalue reference.

`static unsigned long long thread_rank()`: Rank of the calling thread within `[0, num_threads)`

`static unsigned long long block_rank()`: Rank of the calling block within `[0, num_blocks)`

`static unsigned long long cluster_rank()`: Rank of the calling cluster within `[0, num_clusters)`

`static unsigned long long num_threads()`: Total number of threads in the group

`static unsigned long long num_blocks()`: Total number of blocks in the group

`static unsigned long long num_clusters()`: Total number of clusters in the group

`static dim3 dim_blocks()`: Dimensions of the launched grid in units of blocks

`static dim3 dim_clusters()`: Dimensions of the launched grid in units of clusters

`static dim3 block_index()`: 3-Dimensional index of the block within the launched grid

`static dim3 cluster_index()`: 3-Dimensional index of the cluster within the launched grid

Legacy member functions (aliases):

`static unsigned long long size()`: Total number of threads in the group (alias of `num_threads()`)

`static dim3 group_dim()`: Dimensions of the launched grid (alias of `dim_blocks()`)

5.6.3.1.4 class thread_block_tile

A templated version of a tiled group, where a template parameter is used to specify the size of the tile - with this known at compile time there is the potential for more optimal execution.

```
template <unsigned int Size, typename ParentT = void>
class thread_block_tile;
```

Constructed via:

```
template <unsigned int Size, typename ParentT>
_CG_QUALIFIER thread_block_tile<Size, ParentT> tiled_partition(const ParentT&
↪g)
```

Size must be a power of 2 and less than or equal to 1024. Notes section describes extra steps needed to create tiles of size larger than 32 on hardware with Compute Capability 7.5 or lower.

ParentT is the parent-type from which this group was partitioned. It is automatically inferred, but a value of `void` will store this information in the group handle rather than in the type.

Public Member Functions:

`void sync() const`: Synchronize the threads named in the group

`unsigned long long num_threads() const`: Total number of threads in the group

`unsigned long long thread_rank()` const: Rank of the calling thread within `[0, num_threads)`
`unsigned long long meta_group_size()` const: Returns the number of groups created when the parent group was partitioned.

`unsigned long long meta_group_rank()` const: Linear rank of the group within the set of tiles partitioned from a parent group (bounded by `meta_group_size`)

`T shfl(T var, unsigned int src_rank)` const: Refer to [Warp Shuffle Functions](#), **Note: For sizes larger than 32 all threads in the group have to specify the same `src_rank`, otherwise the behavior is undefined.**

`T shfl_up(T var, int delta)` const: Refer to [Warp Shuffle Functions](#), available only for sizes lower or equal to 32.

`T shfl_down(T var, int delta)` const: Refer to [Warp Shuffle Functions](#), available only for sizes lower or equal to 32.

`T shfl_xor(T var, int delta)` const: Refer to [Warp Shuffle Functions](#), available only for sizes lower or equal to 32.

`int any(int predicate)` const: Refer to [Warp Vote Functions](#)

`int all(int predicate)` const: Refer to [Warp Vote Functions](#)

`unsigned int ballot(int predicate)` const: Refer to [Warp Vote Functions](#), available only for sizes lower or equal to 32.

`unsigned int match_any(T val)` const: Refer to [Warp Match Functions](#), available only for sizes lower or equal to 32.

`unsigned int match_all(T val, int &pred)` const: Refer to [Warp Match Functions](#), available only for sizes lower or equal to 32.

Legacy member functions (aliases):

`unsigned long long size()` const: Total number of threads in the group (alias of `num_threads()`)

Notes:

- ▶ `thread_block_tile` templated data structure is being used here, the size of the group is passed to the `tiled_partition` call as a template parameter rather than an argument.
- ▶ `shfl`, `shfl_up`, `shfl_down`, and `shfl_xor` functions accept objects of any type when compiled with C++11 or later. This means it's possible to shuffle non-integral types as long as they satisfy the below constraints:
 - ▶ Qualifies as trivially copyable i.e., `is_trivially_copyable<T>::value == true`
 - ▶ `sizeof(T) <= 32` for tile sizes lower or equal 32, `sizeof(T) <= 8` for larger tiles
- ▶ On hardware with Compute Capability 7.5 or lower tiles of size larger than 32 need small amount of memory reserved for them. This can be done using `cooperative_groups::block_tile_memory` struct template that has to reside in either shared or global memory.

```
template <unsigned int MaxBlockSize = 1024>
struct block_tile_memory;
```

`MaxBlockSize` Specifies the maximal number of threads in the current thread block. This parameter can be used to minimize the shared memory usage of `block_tile_memory` in kernels launched only with smaller thread counts.

This `block_tile_memory` needs be then passed into `cooperative_groups::this_thread_block`, allowing the resulting `thread_block` to be partitioned into tiles of sizes larger than 32. Overload of `this_thread_block` accepting `block_tile_memory` argument is a collective operation and has to be called with all threads in the `thread_block`.

`block_tile_memory` can be used on hardware with Compute Capability 8.0 or higher in order to be able to write one source targeting multiple different Compute Capabilities. It should consume no memory when instantiated in shared memory in cases where its not required.

Examples:

```
/// The following code will create two sets of tiled groups, of size 32 and 4
↪respectively:
/// The latter has the provenance encoded in the type, while the first stores
↪it in the handle
thread_block block = this_thread_block();
thread_block_tile<32> tile32 = tiled_partition<32>(block);
thread_block_tile<4, thread_block> tile4 = tiled_partition<4>(block);
```

```
/// The following code will create tiles of size 128 on all Compute
↪Capabilities.
/// block_tile_memory can be omitted on Compute Capability 8.0 or higher.
__global__ void kernel(...) {
    // reserve shared memory for thread_block_tile usage,
    // specify that block size will be at most 256 threads.
    __shared__ block_tile_memory<256> shared;
    thread_block thb = this_thread_block(shared);

    // Create tiles with 128 threads.
    auto tile = tiled_partition<128>(thb);

    // ...
}
```

5.6.3.1.5 class `coalesced_group`

In CUDA's SIMT architecture, at the hardware level the multiprocessor executes threads in groups of 32 called warps. If there exists a data-dependent conditional branch in the application code such that threads within a warp diverge, then the warp serially executes each branch disabling threads not on that path. The threads that remain active on the path are referred to as coalesced. Cooperative Groups has functionality to discover, and create, a group containing all coalesced threads.

Constructing the group handle via `coalesced_threads()` is opportunistic. It returns the set of active threads at that point in time, and makes no guarantee about which threads are returned (as long as they are active) or that they will stay coalesced throughout execution (they will be brought back together for the execution of a collective but can diverge again afterwards).

```
class coalesced_group;
```

Constructed via:

```
coalesced_group active = coalesced_threads();
```

Public Member Functions:

```
void sync() const: Synchronize the threads named in the group
```

`unsigned long long num_threads()` const: Total number of threads in the group

`unsigned long long thread_rank()` const: Rank of the calling thread within `[0, num_threads)`

`unsigned long long meta_group_size()` const: Returns the number of groups created when the parent group was partitioned. If this group was created by querying the set of active threads, for example `coalesced_threads()` the value of `meta_group_size()` will be 1.

`unsigned long long meta_group_rank()` const: Linear rank of the group within the set of tiles partitioned from a parent group (bounded by `meta_group_size`). If this group was created by querying the set of active threads, e.g. `coalesced_threads()` the value of `meta_group_rank()` will always be 0.

`T shfl(T var, unsigned int src_rank)` const: Refer to [Warp Shuffle Functions](#)

`T shfl_up(T var, int delta)` const: Refer to [Warp Shuffle Functions](#)

`T shfl_down(T var, int delta)` const: Refer to [Warp Shuffle Functions](#)

`int any(int predicate)` const: Refer to [Warp Vote Functions](#)

`int all(int predicate)` const: Refer to [Warp Vote Functions](#)

`unsigned int ballot(int predicate)` const: Refer to [Warp Vote Functions](#)

`unsigned int match_any(T val)` const: Refer to [Warp Match Functions](#)

`unsigned int match_all(T val, int &pred)` const: Refer to [Warp Match Functions](#)

Legacy member functions (aliases):

`unsigned long long size()` const: Total number of threads in the group (alias of `num_threads()`)

Notes:

`shfl`, `shfl_up`, and `shfl_down` functions accept objects of any type when compiled with C++11 or later. This means it's possible to shuffle non-integral types as long as they satisfy the below constraints:

- Qualifies as trivially copyable i.e. `is_trivially_copyable<T>::value == true`
- `sizeof(T) <= 32`

Example:

```
/// Consider a situation whereby there is a branch in the
/// code in which only the 2nd, 4th and 8th threads in each warp are
/// active. The coalesced_threads() call, placed in that branch, will create
→ (for each
/// warp) a group, active, that has three threads (with
/// ranks 0-2 inclusive).
__global__ void kernel(int *globalInput) {
    // Lets say globalInput says that threads 2, 4, 8 should handle the data
    if (threadIdx.x == *globalInput) {
        coalesced_group active = coalesced_threads();
        // active contains 0-2 inclusive
        active.sync();
    }
}
```

5.6.3.2 cooperative_groups/async.h

5.6.3.2.1 memcpy_async

`memcpy_async` is a group-wide collective memcpy that utilizes hardware accelerated support for non-blocking memory transactions from global to shared memory. Given a set of threads named in the group, `memcpy_async` will move specified amount of bytes or elements of the input type through a single pipeline stage. Additionally for achieving best performance when using the `memcpy_async` API, an alignment of 16 bytes for both shared memory and global memory is required. It is important to note that while this is a memcpy in the general case, it is only asynchronous if the source is global memory and the destination is shared memory and both can be addressed with 16, 8, or 4 byte alignments. Asynchronously copied data should only be read following a call to `wait` or `wait_prior` which signals that the corresponding stage has completed moving data to shared memory.

Having to wait on all outstanding requests can lose some flexibility (but gain simplicity). In order to efficiently overlap data transfer and execution, its important to be able to kick off an **N+1** `memcpy_async` request while waiting on and operating on request **N**. To do so, use `memcpy_async` and wait on it using the collective stage-based `wait_prior` API. See [wait and wait_prior](#) for more details.

Usage 1

```
template <typename TyGroup, typename TyElem, typename TyShape>
void memcpy_async(
    const TyGroup &group,
    TyElem *__restrict__ _dst,
    const TyElem *__restrict__ _src,
    const TyShape &shape
);
```

Performs a copy of ``shape`` bytes.

Usage 2

```
template <typename TyGroup, typename TyElem, typename TyDstLayout, typename
↳ TySrcLayout>
void memcpy_async(
    const TyGroup &group,
    TyElem *__restrict__ dst,
    const TyDstLayout &dstLayout,
    const TyElem *__restrict__ src,
    const TySrcLayout &srcLayout
);
```

Performs a copy of ``min(dstLayout, srcLayout)`` elements. If layouts are of type `cuda::aligned_size_t<N>`, both must specify the same alignment.

Errata The `memcpy_async` API introduced in CUDA 11.1 with both src and dst input layouts, expects the layout to be provided in elements rather than bytes. The element type is inferred from `TyElem` and has the size `sizeof(TyElem)`. If `cuda::aligned_size_t<N>` type is used as the layout, the number of elements specified times `sizeof(TyElem)` must be a multiple of N and it is recommended to use `std::byte` or `char` as the element type.

If specified shape or layout of the copy is of type `cuda::aligned_size_t<N>`, alignment will be guaranteed to be at least `min(16, N)`. In that case both `dst` and `src` pointers need to be aligned to N bytes and the number of bytes copied needs to be a multiple of N.

Codegen Requirements: Compute Capability 5.0 minimum, Compute Capability 8.0 for asynchronicity, C++11

cooperative_groups/memcpy_async.h header needs to be included.

Example:

```
/// This example streams elementsPerThreadBlock worth of data from global memory
/// into a limited sized shared memory (elementsInShared) block to operate on.
#include <cooperative_groups.h>
#include <cooperative_groups/memcpy_async.h>

namespace cg = cooperative_groups;

__global__ void kernel(int* global_data) {
    cg::thread_block tb = cg::this_thread_block();
    const size_t elementsPerThreadBlock = 16 * 1024;
    const size_t elementsInShared = 128;
    __shared__ int local_smem[elementsInShared];

    size_t copy_count;
    size_t index = 0;
    while (index < elementsPerThreadBlock) {
        cg::memcpy_async(tb, local_smem, elementsInShared, global_data +
↪index, elementsPerThreadBlock - index);
        copy_count = min(elementsInShared, elementsPerThreadBlock - index);
        cg::wait(tb);
        // Work with local_smem
        index += copy_count;
    }
}
```

5.6.3.2.2 wait and wait_prior

```
template <typename TyGroup>
void wait(TyGroup & group);

template <unsigned int NumStages, typename TyGroup>
void wait_prior(TyGroup & group);
```

wait and wait_prior collectives allow to wait for memcpy_async copies to complete. wait blocks calling threads until all previous copies are done. wait_prior allows that the latest NumStages are still not done and waits for all the previous requests. So with N total copies requested, it waits until the first N-NumStages are done and the last NumStages might still be in progress. Both wait and wait_prior will synchronize the named group.

Codegen Requirements: Compute Capability 5.0 minimum, Compute Capability 8.0 for asynchronicity, C++11

cooperative_groups/memcpy_async.h header needs to be included.

Example:

```
/// This example streams elementsPerThreadBlock worth of data from global memory
/// into a limited sized shared memory (elementsInShared) block to operate on in
/// multiple (two) stages. As stage N is kicked off, we can wait on and operate
↪on stage N-1.
#include <cooperative_groups.h>
```

(continues on next page)

(continued from previous page)

```

#include <cooperative_groups/memcpy_async.h>

namespace cg = cooperative_groups;

__global__ void kernel(int* global_data) {
    cg::thread_block tb = cg::this_thread_block();
    const size_t elementsPerThreadBlock = 16 * 1024 + 64;
    const size_t elementsInShared = 128;
    __align__(16) __shared__ int local_smem[2][elementsInShared];
    int stage = 0;
    // First kick off an extra request
    size_t copy_count = elementsInShared;
    size_t index = copy_count;
    cg::memcpy_async(tb, local_smem[stage], elementsInShared, global_data,
    ↪elementsPerThreadBlock - index);
    while (index < elementsPerThreadBlock) {
        // Now we kick off the next request...
        cg::memcpy_async(tb, local_smem[stage ^ 1], elementsInShared, global_
    ↪data + index, elementsPerThreadBlock - index);
        // ... but we wait on the one before it
        cg::wait_prior<1>(tb);

        // Its now available and we can work with local_smem[stage] here
        // (...)
        //

        // Calculate the amount of data that was actually copied, for the next
    ↪iteration.
        copy_count = min(elementsInShared, elementsPerThreadBlock - index);
        index += copy_count;

        // A cg::sync(tb) might be needed here depending on whether
        // the work done with local_smem[stage] can release threads to race
    ↪ahead or not
        // Wrap to the next stage
        stage ^= 1;
    }
    cg::wait(tb);
    // The last local_smem[stage] can be handled here
}

```

5.6.3.3 cooperative_groups/partition.h

5.6.3.3.1 tiled_partition

```

template <unsigned int Size, typename ParentT>
thread_block_tile<Size, ParentT> tiled_partition(const ParentT& g);

```

```

thread_group tiled_partition(const thread_group& parent, unsigned int tilesz);

```

The `tiled_partition` method is a collective operation that partitions the parent group into a one-dimensional, row-major, tiling of subgroups. A total of $((\text{size}(\text{parent})/\text{tilesz}))$ subgroups will be created,

therefore the parent group size must be evenly divisible by the `Size`. The allowed parent groups are `thread_block` or `thread_block_tile`.

The implementation may cause the calling thread to wait until all the members of the parent group have invoked the operation before resuming execution. Functionality is limited to native hardware sizes, 1/2/4/8/16/32 and the `cg::size(parent)` must be greater than the `Size` parameter. The templated version of `tiled_partition` supports 64/128/256/512 sizes as well, but some additional steps are required on Compute Capability 7.5 or lower, refer to [class `thread\_block\_tile`](#) for details.

Codegen Requirements: Compute Capability 5.0 minimum, C++11 for sizes larger than 32

5.6.3.3.2 `labeled_partition`

```
template <typename Label>
coalesced_group labeled_partition(const coalesced_group& g, Label label);
```

```
template <unsigned int Size, typename Label>
coalesced_group labeled_partition(const thread_block_tile<Size>& g, Label
→label);
```

The `labeled_partition` method is a collective operation that partitions the parent group into one-dimensional subgroups within which the threads are coalesced. The implementation will evaluate a condition label and assign threads that have the same value for label into the same group.

`Label` can be any integral type.

The implementation may cause the calling thread to wait until all the members of the parent group have invoked the operation before resuming execution.

Note: This functionality is still being evaluated and may slightly change in the future.

Codegen Requirements: Compute Capability 7.0 minimum, C++11

5.6.3.3.3 `binary_partition`

```
coalesced_group binary_partition(const coalesced_group& g, bool pred);
```

```
template <unsigned int Size>
coalesced_group binary_partition(const thread_block_tile<Size>& g, bool pred);
```

The `binary_partition()` method is a collective operation that partitions the parent group into one-dimensional subgroups within which the threads are coalesced. The implementation will evaluate a predicate and assign threads that have the same value into the same group. This is a specialized form of `labeled_partition()`, where the label can only be 0 or 1.

The implementation may cause the calling thread to wait until all the members of the parent group have invoked the operation before resuming execution.

Example:

```
/// This example divides a 32-sized tile into a group with odd
/// numbers and a group with even numbers
_global__ void oddEven(int *inputArr) {
    auto block = cg::this_thread_block();
    auto tile32 = cg::tiled_partition<32>(block);
```

(continues on next page)

(continued from previous page)

```

// inputArr contains random integers
int elem = inputArr[block.thread_rank()];
// after this, tile32 is split into 2 groups,
// a subtile where elem&1 is true and one where its false
auto subtile = cg::binary_partition(tile32, (elem & 1));
}

```

5.6.3.4 cooperative_groups/reduce.h

5.6.3.4.1 Reduce Operators

Below are the prototypes of function objects for some of the basic operations that can be done with reduce.

```

namespace cooperative_groups {
    template <typename Ty>
    struct cg::plus;

    template <typename Ty>
    struct cg::less;

    template <typename Ty>
    struct cg::greater;

    template <typename Ty>
    struct cg::bit_and;

    template <typename Ty>
    struct cg::bit_xor;

    template <typename Ty>
    struct cg::bit_or;
}

```

Reduce is limited to the information available to the implementation at compile time. Thus in order to make use of intrinsics introduced in CC 8.0, the `cg::` namespace exposes several functional objects that mirror the hardware. These objects appear similar to those presented in the C++ STL, with the exception of `less/greater`. The reason for any difference from the STL is that these function objects are designed to actually mirror the operation of the hardware intrinsics.

Functional description:

- `cg::plus`: Accepts two values and returns the sum of both using operator+.
- `cg::less`: Accepts two values and returns the lesser using operator<. This differs in that the **lower value is returned** rather than a Boolean.
- `cg::greater`: Accepts two values and returns the greater using operator<. This differs in that the **greater value is returned** rather than a Boolean.
- `cg::bit_and`: Accepts two values and returns the result of operator&.
- `cg::bit_xor`: Accepts two values and returns the result of operator^.
- `cg::bit_or`: Accepts two values and returns the result of operator|.

Example:

```

{
    // cg::plus<int> is specialized within cg::reduce and calls __reduce_add_
    →sync(...) on CC 8.0+
    cg::reduce(tile, (int)val, cg::plus<int>());

    // cg::plus<float> fails to match with an accelerator and instead performs
    →a standard shuffle based reduction
    cg::reduce(tile, (float)val, cg::plus<float>());

    // While individual components of a vector are supported, reduce will not
    →use hardware intrinsics for the following
    // It will also be necessary to define a corresponding operator for vector
    →and any custom types that may be used
    int4 vec = {...};
    cg::reduce(tile, vec, cg::plus<int4>());

    // Finally lambdas and other function objects cannot be inspected for
    →dispatch
    // and will instead perform shuffle based reductions using the provided
    →function object.
    cg::reduce(tile, (int)val, [](int l, int r) -> int {return l + r;});
}

```

5.6.3.4.2 reduce

```

template <typename TyGroup, typename TyArg, typename TyOp>
auto reduce(const TyGroup& group, TyArg&& val, TyOp&& op) -> decltype(op(val,
    →val));

```

reduce performs a reduction operation on the data provided by each thread named in the group passed in. This takes advantage of hardware acceleration (on compute 80 and higher devices) for the arithmetic add, min, or max operations and the logical AND, OR, or XOR, as well as providing a software fallback on older generation hardware. Only 4B types are accelerated by hardware.

group: Valid group types are `coalesced_group` and `thread_block_tile`.

val: Any type that satisfies the below requirements:

- ▶ Qualifies as trivially copyable i.e. `is_trivially_copyable<TyArg>::value == true`
- ▶ `sizeof(T) <= 32` for `coalesced_group` and tiles of size lower or equal 32, `sizeof(T) <= 8` for larger tiles
- ▶ Has suitable arithmetic or comparative operators for the given function object.

Note: Different threads in the group can pass different values for this argument.

op: Valid function objects that will provide hardware acceleration with integral types are `plus()`, `less()`, `greater()`, `bit_and()`, `bit_xor()`, `bit_or()`. These must be constructed, hence the `TyVal` template argument is required, i.e. `plus<int>()`. Reduce also supports lambdas and other function objects that can be invoked using operator()

Asynchronous reduce

```

template <typename TyGroup, typename TyArg, typename TyAtomic, typename TyOp>
void reduce_update_async(const TyGroup& group, TyAtomic& atomic, TyArg&& val,

```

(continues on next page)

(continued from previous page)

```

    ↪TyOp&& op);

template <typename TyGroup, typename TyArg, typename TyAtomic, typename TyOp>
void reduce_store_async(const TyGroup& group, TyAtomic& atomic, TyArg&& val,
    ↪TyOp&& op);

template <typename TyGroup, typename TyArg, typename TyOp>
void reduce_store_async(const TyGroup& group, TyArg* ptr, TyArg&& val, TyOp&&
    ↪op);

```

*_async variants of the API are asynchronously calculating the result to either store to or update a specified destination by one of the participating threads, instead of returning it by each thread. To observe the effect of these asynchronous calls, calling group of threads or a larger group containing them need to be synchronized.

- In case of the atomic store or update variant, atomic argument can be either of `cuda::atomic` or `cuda::atomic_ref` available in [CUDA C++ Standard Library](#). This variant of the API is available only on platforms and devices, where these types are supported by the CUDA C++ Standard Library. Result of the reduction is used to atomically update the atomic according to the specified op, eg. the result is atomically added to the atomic in case of `cg::plus()`. Type held by the atomic must match the type of TyArg. Scope of the atomic must include all the threads in the group and if multiple groups are using the same atomic concurrently, scope must include all threads in all groups using it. Atomic update is performed with relaxed memory ordering.
- In case of the pointer store variant, result of the reduction will be weakly stored into the dst pointer.

5.6.3.5 cooperative_groups/scan.h

5.6.3.5.1 inclusive_scan and exclusive_scan

```

template <typename TyGroup, typename TyVal, typename TyFn>
auto inclusive_scan(const TyGroup& group, TyVal&& val, TyFn&& op) ->
    ↪decltype(op(val, val));

template <typename TyGroup, typename TyVal>
TyVal inclusive_scan(const TyGroup& group, TyVal&& val);

template <typename TyGroup, typename TyVal, typename TyFn>
auto exclusive_scan(const TyGroup& group, TyVal&& val, TyFn&& op) ->
    ↪decltype(op(val, val));

template <typename TyGroup, typename TyVal>
TyVal exclusive_scan(const TyGroup& group, TyVal&& val);

```

`inclusive_scan` and `exclusive_scan` performs a scan operation on the data provided by each thread named in the group passed in. Result for each thread is a reduction of data from threads with lower thread_rank than that thread in case of `exclusive_scan`. `inclusive_scan` result also includes the calling thread data in the reduction.

group: Valid group types are `coalesced_group` and `thread_block_tile`.

val: Any type that satisfies the below requirements:

- Qualifies as trivially copyable i.e. `is_trivially_copyable<TyArg>::value == true`

- `sizeof(T) <= 32` for `coalesced_group` and tiles of size lower or equal 32, `sizeof(T) <= 8` for larger tiles
- Has suitable arithmetic or comparative operators for the given function object.

Note: Different threads in the group can pass different values for this argument.

op: Function objects defined for convenience are `plus()`, `less()`, `greater()`, `bit_and()`, `bit_xor()`, `bit_or()` described in [cooperative_groups/reduce.h](#). These must be constructed, hence the `TyVal` template argument is required, i.e. `plus<int>()`. `inclusive_scan` and `exclusive_scan` also supports lambdas and other function objects that can be invoked using `operator()`. Overloads without this argument use `cg::plus<TyVal>()`.

Scan update

```
template <typename TyGroup, typename TyAtomic, typename TyVal, typename TyFn>
auto inclusive_scan_update(const TyGroup& group, TyAtomic& atomic, TyVal&&
    ↪ val, TyFn&& op) -> decltype(op(val, val));

template <typename TyGroup, typename TyAtomic, typename TyVal>
TyVal inclusive_scan_update(const TyGroup& group, TyAtomic& atomic, TyVal&&
    ↪ val);

template <typename TyGroup, typename TyAtomic, typename TyVal, typename TyFn>
auto exclusive_scan_update(const TyGroup& group, TyAtomic& atomic, TyVal&&
    ↪ val, TyFn&& op) -> decltype(op(val, val));

template <typename TyGroup, typename TyAtomic, typename TyVal>
TyVal exclusive_scan_update(const TyGroup& group, TyAtomic& atomic, TyVal&&
    ↪ val);
```

*_scan_update collectives take an additional argument `atomic` that can be either of `cuda::atomic` or `cuda::atomic_ref` available in [CUDA C++ Standard Library](#). These variants of the API are available only on platforms and devices, where these types are supported by the CUDA C++ Standard Library. These variants will perform an update to the `atomic` according to `op` with value of the sum of input values of all threads in the group. Previous value of the `atomic` will be combined with the result of scan by each thread and returned. Type held by the `atomic` must match the type of `TyVal`. Scope of the `atomic` must include all the threads in the group and if multiple groups are using the same `atomic` concurrently, scope must include all threads in all groups using it. Atomic update is performed with relaxed memory ordering.

Following pseudocode illustrates how the update variant of scan works:

```
/*
    inclusive_scan_update behaves as the following block,
    except both reduce and inclusive_scan is calculated simultaneously.
    auto total = reduce(group, val, op);
    TyVal old;
    if (group.thread_rank() == selected_thread) {
        atomically {
            old = atomic.load();
            atomic.store(op(old, total));
        }
    }
    old = group.shfl(old, selected_thread);
    return op(inclusive_scan(group, val, op), old);
*/
```

cooperative_groups/scan.h header needs to be included.

Example of stream compaction using exclusive_scan:

```
#include <cooperative_groups.h>
#include <cooperative_groups/scan.h>
namespace cg = cooperative_groups;

// put data from input into output only if it passes test_fn predicate
template<typename Group, typename Data, typename TyFn>
__device__ int stream_compaction(Group &g, Data *input, int count, TyFn&&
    test_fn, Data *output) {
    int per_thread = count / g.num_threads();
    int thread_start = min(g.thread_rank() * per_thread, count);
    int my_count = min(per_thread, count - thread_start);

    // get all passing items from my part of the input
    // into a contiguous part of the array and count them.
    int i = thread_start;
    while (i < my_count + thread_start) {
        if (test_fn(input[i])) {
            i++;
        }
        else {
            my_count--;
            input[i] = input[my_count + thread_start];
        }
    }

    // scan over counts from each thread to calculate my starting
    // index in the output
    int my_idx = cg::exclusive_scan(g, my_count);

    for (i = 0; i < my_count; ++i) {
        output[my_idx + i] = input[thread_start + i];
    }
    // return the total number of items in the output
    return g.shfl(my_idx + my_count, g.num_threads() - 1);
}
```

Example of dynamic buffer space allocation using exclusive_scan_update:

```
#include <cooperative_groups.h>
#include <cooperative_groups/scan.h>
namespace cg = cooperative_groups;

// Buffer partitioning is static to make the example easier to follow,
// but any arbitrary dynamic allocation scheme can be implemented by replacing
// this function.
__device__ int calculate_buffer_space_needed(cg::thread_block_tile<32>& tile)
{
    return tile.thread_rank() % 2 + 1;
}
```

(continues on next page)

(continued from previous page)

```

__device__ int my_thread_data(int i) {
    return i;
}

__global__ void kernel() {
    __shared__ extern int buffer[];
    __shared__ cuda::atomic<int, cuda::thread_scope_block> buffer_used;

    auto block = cg::this_thread_block();
    auto tile = cg::tiled_partition<32>(block);
    buffer_used = 0;
    block.sync();

    // each thread calculates buffer size it needs
    int buf_needed = calculate_buffer_space_needed(tile);

    // scan over the needs of each thread, result for each thread is an offset
    // of that thread's part of the buffer. buffer_used is atomically updated
    →with
    // the sum of all thread's inputs, to correctly offset other tile's
    →allocations
    int buf_offset =
        cg::exclusive_scan_update(tile, buffer_used, buf_needed);

    // each thread fills its own part of the buffer with thread specific data
    for (int i = 0 ; i < buf_needed ; ++i) {
        buffer[buf_offset + i] = my_thread_data(i);
    }

    block.sync();
    // buffer_used now holds total amount of memory allocated
    // buffer is {0, 0, 1, 0, 0, 1 ...};
}

```

5.6.3.6 cooperative_groups/sync.h

5.6.3.6.1 barrier_arrive and barrier_wait

```

T::arrival_token T::barrier_arrive();
void T::barrier_wait(T::arrival_token&&);

```

`barrier_arrive` and `barrier_wait` member functions provide a synchronization API similar to `cuda::barrier` ([read more](#)). Cooperative Groups automatically initializes the group barrier, but arrive and wait operations have an additional restriction resulting from collective nature of those operations: All threads in the group must arrive and wait at the barrier once per phase. When `barrier_arrive` is called with a group, result of calling any collective operation or another barrier arrival with that group is undefined until completion of the barrier phase is observed with `barrier_wait` call. Threads blocked on `barrier_wait` might be released from the synchronization before other threads call `barrier_wait`, but only after all threads in the group called `barrier_arrive`. Group type `T` can be any of the [implicit groups](#). This allows threads to do independent work after they arrive and before they wait for the synchronization to resolve, allowing to hide some of the synchronization latency. `barrier_arrive` returns an `arrival_token` object that must be passed into the corresponding `bar-`

rier_wait. Token is consumed this way and can not be used for another barrier_wait call.

Example of barrier_arrive and barrier_wait used to synchronize initialization of shared memory across the cluster:

```
#include <cooperative_groups.h>

using namespace cooperative_groups;

void __device__ init_shared_data(const thread_block& block, int *data);
void __device__ local_processing(const thread_block& block);
void __device__ process_shared_data(const thread_block& block, int *data);

__global__ void cluster_kernel() {
    extern __shared__ int array[];
    auto cluster = this_cluster();
    auto block = this_thread_block();

    // Use this thread block to initialize some shared state
    init_shared_data(block, &array[0]);

    auto token = cluster.barrier_arrive(); // Let other blocks know this block
    ↪is running and data was initialized

    // Do some local processing to hide the synchronization latency
    local_processing(block);

    // Map data in shared memory from the next block in the cluster
    int *dsmem = cluster.map_shared_rank(&array[0], (cluster.block_rank() +
    ↪1) % cluster.num_blocks());

    // Make sure all other blocks in the cluster are running and initialized
    ↪shared data before accessing dsmem
    cluster.barrier_wait(std::move(token));

    // Consume data in distributed shared memory
    process_shared_data(block, dsmem);
    cluster.sync();
}
```

5.6.3.6.2 sync

```
static void T::sync();

template <typename T>
void sync(T& group);
```

sync synchronizes the threads named in the group. Group type T can be any of the existing group types, as all of them support synchronization. Its available as a member function in every group type or as a free function taking a group as parameter.

5.6.3.6.2.1 Grid Synchronization

Prior to the introduction of Cooperative Groups, the CUDA programming model only allowed synchronization between thread blocks at a kernel completion boundary. The kernel boundary carries with it an implicit invalidation of state, and with it, potential performance implications.

For example, in certain use cases, applications have a large number of small kernels, with each kernel representing a stage in a processing pipeline. The presence of these kernels is required by the current CUDA programming model to ensure that the thread blocks operating on one pipeline stage have produced data before the thread block operating on the next pipeline stage is ready to consume it. In such cases, the ability to provide global inter thread block synchronization would allow the application to be restructured to have persistent thread blocks, which are able to synchronize on the device when a given stage is complete.

To synchronize across the grid, from within a kernel, you would simply use the `grid.sync()` function:

```
grid_group grid = this_grid();
grid.sync();
```

And when launching the kernel it is necessary to use, instead of the `<<<...>>>` execution configuration syntax, the `cudaLaunchCooperativeKernel` CUDA runtime launch API or the CUDA driver equivalent.

Example:

To guarantee co-residency of the thread blocks on the GPU, the number of blocks launched needs to be carefully considered. For example, as many blocks as there are SMs can be launched as follows:

```
int dev = 0;
cudaDeviceProp deviceProp;
cudaGetDeviceProperties(&deviceProp, dev);
// initialize, then launch
cudaLaunchCooperativeKernel((void*)my_kernel, deviceProp.multiProcessorCount,
    ↪ numThreads, args);
```

Alternatively, you can maximize the exposed parallelism by calculating how many blocks can fit simultaneously per-SM using the occupancy calculator as follows:

```
/// This will launch a grid that can maximally fill the GPU, on the default
    ↪ stream with kernel arguments
int numBlocksPerSm = 0;
// Number of threads my_kernel will be launched with
int numThreads = 128;
cudaDeviceProp deviceProp;
cudaGetDeviceProperties(&deviceProp, dev);
cudaOccupancyMaxActiveBlocksPerMultiprocessor(&numBlocksPerSm, my_kernel,
    ↪ numThreads, 0);
// launch
void *kernelArgs[] = { /* add kernel args */ };
dim3 dimBlock(numThreads, 1, 1);
dim3 dimGrid(deviceProp.multiProcessorCount*numBlocksPerSm, 1, 1);
cudaLaunchCooperativeKernel((void*)my_kernel, dimGrid, dimBlock, kernelArgs);
```

It is good practice to first ensure the device supports cooperative launches by querying the device attribute `cudaDevAttrCooperativeLaunch`:

```
int dev = 0;
int supportsCoopLaunch = 0;
cudaDeviceGetAttribute(&supportsCoopLaunch, cudaDevAttrCooperativeLaunch,
    dev);
```

which will set `supportsCoopLaunch` to 1 if the property is supported on device 0. Only devices with compute capability of 6.0 and higher are supported. In addition, you need to be running on either of these:

- ▶ The Linux platform without MPS
- ▶ The Linux platform with MPS and on a device with compute capability 7.0 or higher
- ▶ The latest Windows platform

5.6.4. CUDA Device Runtime

The CUDA device runtime is an API available within kernel code which provides many of the same capabilities as the CUDA Runtime API on the host. These APIs are used most often in the contexts of *CUDA Dynamic Parallelism* or *Device Graph Launch*.

5.6.4.1 Including Device Runtime API in CUDA Code

Similar to the host-side runtime API, prototypes for the CUDA device runtime API are included automatically during program compilation. There is no need to include `cuda_device_runtime_api.h` explicitly.

5.6.4.2 Memory in the CUDA Device Runtime

5.6.4.2.1 Configuration Options

Resource allocation for the device runtime system software is controlled via the `cudaDeviceSetLimit()` API from the host program. Limits must be set before any kernel is launched, and may not be changed while the GPU is actively running programs.

The following named limits may be set:

Limit	Behavior
<code>cudaLimitDevRuntimePendingLaunchCount</code>	Controls the amount of memory set aside for buffering kernel launches and events which have not yet begun to execute, due either to unresolved dependencies or lack of execution resources. When the buffer is full, an attempt to allocate a launch slot during a device side kernel launch will fail and return <code>cudaErrorLaunchOutOfResources</code> , while an attempt to allocate an event slot will fail and return <code>cudaErrorMemoryAllocation</code> . The default number of launch slots is 2048. Applications may increase the number of launch and/or event slots by setting <code>cudaLimitDevRuntimePendingLaunchCount</code> . The number of event slots allocated is twice the value of that limit.
<code>cudaLimitStackSize</code>	Controls the stack size in bytes of each GPU thread. The CUDA driver automatically increases the per-thread stack size for each kernel launch as needed. This size isn't reset back to the original value after each launch. To set the per-thread stack size to a different value, <code>cudaDeviceSetLimit()</code> can be called to set this limit. The stack will be immediately resized, and if necessary, the device will block until all preceding requested tasks are complete. <code>cudaDeviceGetLimit()</code> can be called to get the current per-thread stack size.

5.6.4.2.2 Allocation and Lifetime

`cudaMalloc()` and `cudaFree()` have distinct semantics between the host and device environments. When invoked from the host, `cudaMalloc()` allocates a new region from unused device memory. When invoked from the device runtime these functions map to device-side `malloc()` and `free()`. This implies that within the device environment the total allocatable memory is limited to the device `malloc()` heap size, which may be smaller than the available unused device memory. Also, it is an error to invoke `cudaFree()` from the host program on a pointer which was allocated by `cudaMalloc()` on the device or vice-versa.

	<code>cudaMalloc()</code> on Host	<code>cudaMalloc()</code> on Device
<code>cudaFree()</code> on Host	Supported	Not Supported
<code>cudaFree()</code> on Device	Not Supported	Supported
Allocation limit	Available device memory	<code>cudaLimitMallocHeapSize</code>

5.6.4.2.2.1 Memory Declarations

5.6.4.2.2.2 Device and Constant Memory

Memory declared at file scope with `__device__` or `__constant__` memory space specifiers behaves identically when using the device runtime. All kernels may read or write device variables, whether the kernel was initially launched by the host or device runtime. Equivalently, all kernels will have the same view of `__constant__`s as declared at the module scope.

5.6.4.2.2.3 Textures and Surfaces

The device runtime does not allow creation or destruction of texture or surface objects from within device code. Texture and surface objects created from the host may be used and passed around freely on the device. Regardless of where they are created, dynamically created texture objects are always valid and may be passed to child kernels from a parent.

Note

The device runtime does not support legacy module-scope (i.e., compute capability 2.0 or Fermi-style) textures and surfaces within a kernel launched from the device. Module-scope (legacy) textures may be created from the host and used in device code as for any kernel, but may only be used by a top-level kernel (i.e., the one which is launched from the host).

5.6.4.2.2.4 Shared Memory Variable Declarations

In CUDA C++ shared memory can be declared either as a statically sized file-scope or function-scoped variable, or as an extern variable with the size determined at runtime by the kernel's caller via a launch configuration argument. Both types of declarations are valid under the device runtime.

```
__global__ void permute(int n, int *data) {
    extern __shared__ int smem[ ];
    if (n <= 1)
        return;

    smem[threadIdx.x] = data[threadIdx.x];
    __syncthreads();

    permute_data(smem, n);
    __syncthreads();

    // Write back to GMEM since we can't pass SMEM to children.
    data[threadIdx.x] = smem[threadIdx.x];
    __syncthreads();

    if (threadIdx.x == 0) {
        permute<<< 1, 256, n/2*sizeof(int) >>>(n/2, data);
        permute<<< 1, 256, n/2*sizeof(int) >>>(n/2, data+n/2);
    }
}

void host_launch(int *data) {
    permute<<< 1, 256, 256*sizeof(int) >>>(256, data);
}
```

5.6.4.2.2.5 Constant Memory

Constants may not be modified from the device. They may only be modified from the host, but the behavior of modifying a constant from the host while there is a concurrent grid that access that constant at any point during its lifetime is undefined.

5.6.4.2.2.6 Symbol Addresses

Device-side symbols (i.e., those marked `__device__`) may be referenced from within a kernel simply via the `&` operator, as all global-scope device variables are in the kernel's visible address space. This also applies to `__constant__` symbols, although in this case the pointer will reference read-only data.

Since device-side symbols can be referenced directly, those CUDA runtime APIs which reference symbols (e.g., `cudaMemcpyToSymbol()` or `cudaGetSymbolAddress()`) are unnecessary and are not supported by the device runtime. This implies that constant data cannot be altered from within a running kernel, even ahead of a child kernel launch, as references to `__constant__` space are read-only.

5.6.4.3 SM Id and Warp Id

Note that in PTX `%smid` and `%warpid` are defined as volatile values. The device runtime may reschedule thread blocks onto different SMs in order to more efficiently manage resources. As such, it is unsafe to rely upon `%smid` or `%warpid` remaining unchanged across the lifetime of a thread or thread block.

5.6.4.4 Launch Setup APIs

Device-Side Kernel Launch describes the syntax for launching kernels from device code using the same triple chevron launch notation as the host CUDA Runtime API.

Kernel launch is a system-level mechanism exposed through the device runtime library. It is also available directly from PTX via `cudaGetParameterBuffer()` and `cudaLaunchDevice()` APIs. It is permitted for a CUDA application to call these APIs itself, with the same requirements as for PTX. In both cases, the user is then responsible for correctly populating all necessary data structures in the correct format according to specification. Backwards compatibility is guaranteed in these data structures.

As with host-side launch, the device-side operator `<<<>>>` maps to underlying kernel launch APIs. This allows users targeting PTX will to perform a launch. The NVCC compiler front-end translates `<<<>>>` into these calls.

Table 61: New Device-only Launch Implementation Functions

Runtime Functions	API Launch	Description of Difference From Host Runtime Behavior (behavior is identical if no description)
<code>cudaGetParameterBuffer</code>		Generated automatically from <code><<<>>></code> . Note different API to host equivalent.
<code>cudaLaunchDevice</code>		Generated automatically from <code><<<>>></code> . Note different API to host equivalent.

The APIs for these launch functions are different to those of the CUDA Runtime API, and are defined as follows:

```
extern device cudaError_t cudaGetParameterBuffer(void **params);
extern __device__ cudaError_t cudaLaunchDevice(void *kernel,
                                                void *params, dim3 gridDim,
                                                dim3 blockDim,
                                                unsigned int sharedMemSize = 0,
                                                cudaStream_t stream = 0);
```

5.6.4.5 Device Management

There is no multi-GPU support from the device runtime; the device runtime is only capable of operating on the device upon which it is currently executing. It is permitted, however, to query properties for any CUDA capable device in the system.

5.6.4.6 API Reference

The portions of the CUDA Runtime API supported in the device runtime are detailed here. Host and device runtime APIs have identical syntax; semantics are the same except where indicated. The following table provides an overview of the API relative to the version available from the host.

Table 62: Supported API Functions

Runtime API Functions	Details
cudaDeviceGetCacheConfig	
cudaDeviceGetLimit	
cudaGetLastError	Last error is per-thread state, not per-block state
cudaPeekAtLastError	
cudaGetErrorString	
cudaGetDeviceCount	
cudaDeviceGetAttribute	Will return attributes for any device
cudaGetDevice	Always returns current device ID as would be seen from host
cudaStreamCreateWithFlags	Must pass cudaStreamNonBlocking flag
cudaStreamDestroy	
cudaStreamWaitEvent	
cudaEventCreateWithFlags	Must pass cudaEventDisableTiming flag
cudaEventRecord	
cudaEventDestroy	
cudaFuncGetAttributes	
cudaMemcpyAsync	Notes about all memcpy/memset functions: ▶ Only async memcpy/set functions are supported ▶ Only device-to-device memcpy is permitted ▶ May not pass in local or shared memory pointers
cudaMemcpy2DAsync	
cudaMemcpy3DAsync	
cudaMemsetAsync	
cudaMemset2DAsync	
cudaMemset3DAsync	
cudaRuntimeGetVersion	
cudaMalloc	May not call cudaFree on the device on a pointer created on the host, and vice-versa
cudaFree	
cudaOccupancyMaxActiveBlocksPerMulti-processor	
cudaOccupancyMaxPotentialBlockSize	
cudaOccupancyMaxPotentialBlockSize-VariableSMem	

5.6.4.7 API Errors and Launch Failures

As usual for the CUDA runtime, any function may return an error code. The last error code returned is recorded and may be retrieved via the `cudaGetLastError()` call. Errors are recorded per-thread, so that each thread can identify the most recent error that it has generated. The error code is of type `cudaError_t`.

Similar to a host-side launch, device-side launches may fail for many reasons (invalid arguments, etc). The user must call `cudaGetLastError()` to determine if a launch generated an error, however lack of an error after launch does not imply the child kernel completed successfully.

For device-side exceptions, e.g., access to an invalid address, an error in a child grid will be returned to the host.

5.6.4.8 Device Runtime Streams

The CUDA device runtime exposes special named streams which provide specific behaviors for kernels and graphs launched from the device. The named streams relevant to device graph launch are documented in [Device Launch](#). Two other named streams which can be used for kernels and memcpy operations in the CUDA device runtime are `cudaStreamTailLaunch` and `cudaStreamTailLaunch`. The specific behaviors of these named streams are documented in this section.

Both named and unnamed (NULL) streams are available from the device runtime. Stream handles may not be passed to parent or child grids. In other words, a stream should be treated as private to the grid in which it is created.

The host-side NULL stream's cross-stream barrier semantic is not supported on the device (see below for details). In order to retain semantic compatibility with the host runtime, all device streams must be created using the `cudaStreamCreateWithFlags()` API, passing the `cudaStreamNonBlocking` flag. The `cudaStreamCreate()` API is not available in the CUDA device runtime.

As `cudaStreamSynchronize()` and `cudaStreamQuery()` are unsupported by the device runtime. A kernel launched into the `cudaStreamTailLaunch` stream should be used instead when the application needs to know that stream-launched child kernels have completed.

5.6.4.8.1 The Implicit (NULL) Stream

Within a host program, the unnamed (NULL) stream has additional barrier synchronization semantics with other streams (see [Blocking and non-blocking streams and the default stream](#) for details). The device runtime offers a single implicit, unnamed stream shared between all threads in a thread block, but as all named streams must be created with the `cudaStreamNonBlocking` flag, work launched into the NULL stream will not insert an implicit dependency on pending work in any other streams (including NULL streams of other thread blocks).

5.6.4.8.2 The Fire-and-Forget Stream

The fire-and-forget named stream (`cudaStreamFireAndForget`) allows the user to launch fire-and-forget work with less boilerplate and without stream tracking overhead. It is functionally identical to, but faster than, creating a new stream per launch, and launching into that stream.

Fire-and-forget launches are immediately scheduled for launch without any dependency on the completion of previously launched grids. No other grid launches can depend on the completion of a fire-and-forget launch, except through the implicit synchronization at the end of the parent grid. So a tail launch or the next grid in parent grid's stream won't launch before a parent grid's fire-and-forget work has completed.

```
// In this example, C2's launch will not wait for C1's completion
__global__ void P( ... ) {
    C1<<< ... , cudaStreamFireAndForget >>>( ... );
    C2<<< ... , cudaStreamFireAndForget >>>( ... );
}
```

The fire-and-forget stream cannot be used to record or wait on events. Attempting to do so results in `cudaErrorInvalidValue`. The fire-and-forget stream is not supported when compiled with `CUDA_FORCE_CDP1_IF_SUPPORTED` defined. Fire-and-forget stream usage requires compilation to be in 64-bit mode.

5.6.4.8.3 The Tail Launch Stream

The tail launch named stream (`cudaStreamTailLaunch`) allows a grid to schedule a new grid for launch after its completion. It should be possible to use a tail launch to achieve the same functionality as a `cudaDeviceSynchronize()` in most cases.

Each grid has its own tail launch stream. All non-tail launch work launched by a grid is implicitly synchronized before the tail stream is kicked off. I.e. A parent grid's tail launch does not launch until the parent grid and all work launched by the parent grid to ordinary streams or per-thread or fire-and-forget streams have completed. If two grids are launched to the same grid's tail launch stream, the later grid does not launch until the earlier grid and all its descendent work has completed.

```
// In this example, C2 will only launch after C1 completes.
__global__ void P( ... ) {
    C1<<< ... , cudaStreamTailLaunch >>>( ... );
    C2<<< ... , cudaStreamTailLaunch >>>( ... );
}
```

Grids launched into the tail launch stream will not launch until the completion of all work by the parent grid, including all other grids (and their descendants) launched by the parent in all non-tail launched streams, including work executed or launched after the tail launch.

```
// In this example, C will only launch after all X, F and P complete.
__global__ void P( ... ) {
    C<<< ... , cudaStreamTailLaunch >>>( ... );
    X<<< ... , cudaStreamPerThread >>>( ... );
    F<<< ... , cudaStreamFireAndForget >>>( ... )
}
```

The next grid in the parent grid's stream will not be launched before a parent grid's tail launch work has completed. In other words, the tail launch stream behaves as if it were inserted between its parent grid and the next grid in its parent grid's stream.

```
// In this example, P2 will only launch after C completes.
__global__ void P1( ... ) {
    C<<< ... , cudaStreamTailLaunch >>>( ... );
}

__global__ void P2( ... ) {
}

int main ( ... ) {
    ...
}
```

(continues on next page)

(continued from previous page)

```

P1<<< ... >>>( ... );
P2<<< ... >>>( ... );
...
}

```

Each grid only gets one tail launch stream. To tail launch concurrent grids, it can be done like the example below.

```

// In this example, C1 and C2 will launch concurrently after P's completion
__global__ void T( ... ) {
    C1<<< ... , cudaStreamFireAndForget >>>( ... );
    C2<<< ... , cudaStreamFireAndForget >>>( ... );
}

__global__ void P( ... ) {
    ...
    T<<< ... , cudaStreamTailLaunch >>>( ... );
}

```

The tail launch stream cannot be used to record or wait on events. Attempting to do so results in `cudaErrorInvalidValue`. The tail launch stream is not supported when compiled with `CUDA_FORCE_CDP1_IF_SUPPORTED` defined. Tail launch stream usage requires compilation to be in 64-bit mode.

5.6.4.9 ECC Errors

No notification of ECC errors is available to code within a CUDA kernel. ECC errors are reported at the host side once the entire launch tree has completed. Any ECC errors which arise during execution of a nested program will either generate an exception or continue execution (depending upon error and configuration).

5.7. CUDA C++ Memory Model

Standard C++ presents a view that the cost to synchronize threads is uniform and low.

CUDA C++ is different: the cost to synchronize threads grows as threads are further apart. It is low across threads within a block, but high across arbitrary threads in the system running on multiple GPUs and CPUs.

To account for non-uniform thread synchronization costs that are not always low, CUDA C++ extends the standard C++ memory model and concurrency facilities in the `cuda::` namespace with **thread scopes**, retaining the syntax and semantics of standard C++ by default.

5.7.1. Thread Scopes

A **thread scope** specifies the kind of threads that can synchronize with each other using a synchronization primitive such as `cuda::atomic` or `cuda::barrier`.

```
namespace cuda {
```

(continues on next page)

(continued from previous page)

```
enum thread_scope {
    thread_scope_system,
    thread_scope_device,
    thread_scope_block,
    thread_scope_thread
};

} // namespace cuda
```

5.7.1.1 Scope Relationships

Each program thread is related to each other program thread by one or more thread scope relations:

- ▶ Each thread in the system is related to each other thread in the system by the *system* thread scope: `cuda::thread_scope_system`.
- ▶ Each GPU thread is related to each other GPU thread in the same CUDA device and within the same *memory synchronization domain* by the *device* thread scope: `cuda::thread_scope_device`.
- ▶ Each GPU thread is related to each other GPU thread in the same CUDA thread block by the *block* thread scope: `cuda::thread_scope_block`.
- ▶ Each thread is related to itself by the *thread* thread scope: `cuda::thread_scope_thread`.

5.7.2. Synchronization primitives

Types in namespaces `std::` and `cuda::std::` have the same behavior as corresponding types in namespace `cuda::` when instantiated with a scope of `cuda::thread_scope_system`.

5.7.3. Atomicity

An atomic operation is atomic at the scope it specifies if:

- ▶ it specifies a scope other than `cuda::thread_scope_system`, **or**
- ▶ the scope is `cuda::thread_scope_system` **and**:
 - ▶ it affects an object in *system allocated memory* and `pageableMemoryAccess` is 1 [0], **or**
 - ▶ it affects an object in *managed memory* and `concurrentManagedAccess` is 1, **or**
 - ▶ it affects an object in *mapped memory* and `hostNativeAtomicSupported` is 1, **or**
 - ▶ it is a load or store that affects a naturally-aligned object of sizes 1, 2, 4, 8, or 16 bytes on *mapped memory* [1], **or**
 - ▶ it affects an object in GPU memory, only GPU threads access it, **and**
 - ▶ `cudaDeviceGetP2PAttribute` (&val, `cudaDevP2PAttrNativeAtomicSupported`, `srcDev`, `dstDev`) between each accessing `srcDev` and the GPU where the object resides, `dstDev`, is 1, **or**
 - ▶ only GPU threads from a single GPU concurrently access it.

Note

- ▶ [0] If `PageableMemoryAccessUsesHostPagetable` is 0 then atomic operations to memory mapped file or `hugetlbfs` allocations are not atomic.
- ▶ [1] If `hostNativeAtomicSupported` is 0, atomic load or store operations at system scope that affect a naturally-aligned 16-byte wide object in *system allocated memory* or *mapped memory* require system support. NVIDIA is not aware of any system that lacks this support and there is no CUDA API query available to detect such systems.

For more information on *system allocated memory*, *managed memory*, *mapped memory*, CPU memory, and GPU memory, see the relevant sections of this guide.

5.7.4. Data Races

Modify `intro.races` paragraph 21 of ISO/IEC IS 14882 (the C++ Standard) as follows:

The execution of a program contains a data race if it contains two potentially concurrent conflicting actions, at least one of which is not atomic **at a scope that includes the thread that performed the other operation**, and neither happens before the other, except for the special case for signal handlers described below. Any such data race results in undefined behavior. [...]

Modify `thread.barrier.class` paragraph 4 of ISO/IEC IS 14882 (the C++ Standard) as follows

4. Concurrent invocations of the member functions of `barrier`, other than its destructor, do not introduce data races **as if they were atomic operations**. [...]

Modify `thread.latch.class` paragraph 2 of ISO/IEC IS 14882 (the C++ Standard) as follows:

2. Concurrent invocations of the member functions of `latch`, other than its destructor, do not introduce data races **as if they were atomic operations**. [...]

Modify `thread.sema.cnt` paragraph 3 of ISO/IEC IS 14882 (the C++ Standard) as follows:

3. Concurrent invocations of the member functions of `counting_semaphore`, other than its destructor, do not introduce data races **as if they were atomic operations**.

Modify `thread.stoptoken.intro` paragraph 5 of ISO/IEC IS 14882 (the C++ Standard) as follows:

Calls to the functions `request_stop`, `stop_requested`, and `stop_possible` do not introduce data races **as if they were atomic operations**. [...]

Modify `atomics.fences` paragraph 2 through 4 of ISO/IEC IS 14882 (the C++ Standard) as follows:

A release fence A synchronizes with an acquire fence B if there exist atomic operations X and Y, both operating on some atomic object M, such that A is sequenced before X, X modifies M, Y is sequenced before B, and Y reads the value written by X or a value written by any side effect in the hypothetical release sequence X would head if it were a release operation, **and each operation (A, B, X, and Y) specifies a scope that includes the thread that performed each other operation**.

A release fence A synchronizes with an atomic operation B that performs an acquire operation on an atomic object M if there exists an atomic operation X such that A is sequenced before X, X modifies M, and B reads the value written by X or a value written by any side effect in the hypothetical release sequence X would head if it were a release operation, **and each operation (A, B, and X) specifies a scope that includes the thread that performed each other operation**.

An atomic operation A that is a release operation on an atomic object M synchronizes with an acquire fence B if there exists some atomic operation X on M such that X is sequenced before B and reads the value written by A or a value written by any side effect in the release sequence headed by A, **and each operation (A, B, and X) specifies a scope that includes the thread that performed each other operation.**

5.7.5. Example: Message Passing

The following example passes a message stored to the x variable by a thread in block 0 to a thread in block 1 via the flag f:

Initially

```
int x = 0, f = 0;
```

Thread 0 Block 0

```
x = 42;
cuda::atomic_ref<int, cuda::thread_scope_device> flag(f);
flag.store(1, memory_order_release);
```

Thread 0 Block 1

```
cuda::atomic_ref<int, cuda::thread_scope_device> flag(f);
while(flag.load(memory_order_acquire) != 1);
assert(x == 42);
```

In the following variation of the previous example, two threads concurrently access the f object without synchronization, which leads to a **data race**, and exhibits **undefined behavior**:

Initially

```
int x = 0, f = 0;
```

Thread 0 Block 0

```
x = 42;
cuda::atomic_ref<int, cuda::thread_scope_block> flag(f);
flag.store(1, memory_order_release); // UB: data race
```

Thread 0 Block 1

```
cuda::atomic_ref<int, cuda::thread_scope_device> flag(f);
while(flag.load(memory_order_acquire) != 1); // UB: data race
assert(x == 42);
```

While the memory operations on `f` - the store and the loads - are atomic, the scope of the store operation is “block scope”. Since the store is performed by Thread 0 of Block 0, it only includes all other threads of Block 0. However, the thread doing the loads is in Block 1, i.e., it is not in a scope included by the store operation performed in Block 0, causing the store and the load to not be “atomic”, and introducing a data-race.

For more examples see the [PTX memory consistency model litmus tests](#).

5.8. CUDA C++ Execution model

CUDA C++ aims to provide [parallel forward progress \[intro.progress.9\]](#) for all device threads of execution, facilitating the parallelization of pre-existing C++ applications with CUDA C++.

[intro.progress]

- [\[intro.progress.7\]](#): For a thread of execution providing [concurrent forward progress guarantees](#), the implementation ensures that the thread will eventually make progress for as long as it has not terminated.

[Note 5: This applies regardless of whether or not other threads of execution (if any) have been or are making progress. To eventually fulfill this requirement means that this will happen in an unspecified but finite amount of time. — end note]

- [\[intro.progress.9\]](#): For a thread of execution providing [parallel forward progress guarantees](#), the implementation is not required to ensure that the thread will eventually make progress if it has not yet executed any execution step; once this thread has executed a step, it provides [concurrent forward progress guarantees](#).

[Note 6: This does not specify a requirement for when to start this thread of execution, which will typically be specified by the entity that creates this thread of execution. For example, a thread of execution that provides concurrent forward progress guarantees and executes tasks from a set of tasks in an arbitrary order, one after the other, satisfies the requirements of parallel forward progress for these tasks. — end note]

The CUDA C++ Programming Language is an extension of the C++ Programming Language. This section documents the modifications and extensions to the [intro.progress] section of the current ISO International Standard ISO/IEC 14882 – Programming Language C++ draft. Modified sections are called out explicitly and their diff is shown in **bold**. All other sections are additions.

5.8.1. Host threads

The forward progress provided by threads of execution created by the host implementation to execute `main`, `std::thread`, and `std::jthread` is implementation-defined behavior of the host implementation [intro.progress]. General-purpose host implementations should provide concurrent forward progress.

If the host implementation provides **concurrent forward progress** [intro.progress.7], then CUDA C++ provides **parallel forward progress** [intro.progress.9] for device threads.

5.8.2. Device threads

Once a device thread makes progress:

- ▶ If it is part of a **Cooperative Grid**, all device threads in its grid shall eventually make progress.
- ▶ Otherwise, all device threads in its **thread-block cluster** shall eventually make progress.

[Note: Threads in other thread-block clusters are not guaranteed to eventually make progress. - end note.]

[Note: This implies that all device threads within its thread block shall eventually make progress. - end note.]

Modify [intro.progress.1] as follows (modifications in **bold**):

The implementation may assume that any **host** thread will eventually do one of the following:

1. terminate,
2. invoke the function `std::this_thread::yield` ([thread.thread.this]),
3. make a call to a library I/O function,
4. perform an access through a volatile glvalue,
5. perform a synchronization operation or an atomic operation, or
6. continue execution of a trivial infinite loop ([stmt.iter.general]).

The implementation may assume that any device thread will eventually do one of the following:

1. **terminate,**
2. **make a call to a library I/O function,**
3. **perform an access through a volatile glvalue except if the designated object has automatic storage duration, or**
4. **perform a synchronization operation or an atomic read operation except if the designated object has automatic storage duration.**

[Note: Some current limitations of device threads relative to host threads are implementation defects known to us, that we may fix over time. Examples include the undefined behavior that arises from device threads that eventually only perform volatile or atomic operations on automatic storage duration objects. However, other limitations of device

threads relative to host threads are intentional choices. They enable performance optimizations that would not be possible if device threads followed the C++ Standard strictly. For example, providing forward progress to programs that eventually only perform atomic writes or fences would degrade overall performance for little practical benefit. - end note.]

Examples of forward progress guarantee differences between host and device threads due to modifications to [intro.progress.1].

The following examples refer to the itemized sub-clauses of the implementation assumptions for host and device threads above using “host.threads.<id>” and “device.threads.<id>”, respectively.

```
1 // Example: Execution.Model.Device.0
2 // Outcome: grid eventually terminates per device.threads.4 because the atomic
  ↳ object does not have automatic storage duration.
3 __global__ void ex0(cuda::atomic_ref<int, cuda::thread_scope_device> atom) {
4     if (threadIdx.x == 0) {
5         while(atom.load(cuda::memory_order_relaxed) == 0);
6     } else if (threadIdx.x == 1) {
7         atom.store(1, cuda::memory_order_relaxed);
8     }
9 }
```

```
1 // Example: Execution.Model.Device.1
2 // Allowed outcome: No thread makes progress because device threads don't
  ↳ support host.threads.2.
3 __global__ void ex1() {
4     while(true) cuda::std::this_thread::yield();
5 }
```

```
1 // Example: Execution.Model.Device.2
2 // Allowed outcome: No thread makes progress because device threads don't
  ↳ support host.threads.4
3 // for objects with automatic storage duration (see exception in device.threads.
  ↳ 3).
4 __global__ void ex2() {
5     volatile bool True = true;
6     while(True);
7 }
```

```
1 // Example: Execution.Model.Device.3
2 // Allowed outcome: No thread makes progress because device threads don't
  ↳ support host.threads.5
3 // for objects with automatic storage duration (see exception in device.threads.
  ↳ 4).
4 __global__ void ex3() {
5     cuda::atomic<bool, cuda::thread_scope_thread> True = true;
6     while(True.load());
7 }
```

```
1 // Example: Execution.Model.Device.4
2 // Allowed outcome: No thread makes progress because device threads don't
  ↳ support host.thread.6.
3 __global void ex4() {
```

(continues on next page)

(continued from previous page)

```

4   while(true) { /* empty */ }
5 }

```

5.8.3. CUDA APIs

A CUDA API call shall eventually either return or ensure at least one device thread makes progress.

CUDA query functions (e.g. `cudaStreamQuery`, `cudaEventQuery`, etc.) shall not consistently return `cudaErrorNotReady` without a device thread making progress.

[Note: The device thread need not be “related” to the API call, e.g., an API operating on one stream or process may ensure progress of a device thread on another stream or process. - end note.]

[Note: A simple but not sufficient method to test a program for CUDA API Forward Progress conformance is to run them with following environment variables set: `CUDA_DEVICE_MAX_CONNECTIONS=1` `CUDA_LAUNCH_BLOCKING=1`, and then check that the program still terminates. If it does not, the program has a bug. This method is not sufficient because it does not catch all Forward Progress bugs, but it does catch many such bugs. - end note.]

Examples of CUDA API forward progress guarantees.

```

1  // Example: Execution.Model.API.1
2  // Outcome: if no other device threads (e.g., from other processes) are making
   // progress,
3  // this program terminates and returns cudaSuccess.
4  // Rationale: CUDA guarantees that if the device is empty:
5  // - `cudaDeviceSynchronize` eventually ensures that at least one device-thread
   // makes progress, which implies that eventually `hello_world` grid and one of
   // its device-threads start.
6  // - All thread-block threads eventually start (due to "if a device thread
   // makes progress, all other threads in its thread-block cluster eventually
   // make progress").
7  // - Once all threads in thread-block arrive at `__syncthreads` barrier, all
   // waiting threads are unblocked.
8  // - Therefore all device threads eventually exit the `hello_world` grid.
9  // - And `cudaDeviceSynchronize` eventually unblocks.
10 __global__ void hello_world() { __syncthreads(); }
11 int main() {
12     hello_world<<<1,2>>>();
13     return (int)cudaDeviceSynchronize();
14 }

```

```

1  // Example: Execution.Model.API.2
2  // Allowed outcome: eventually, no thread makes progress.
3  // Rationale: the `cudaDeviceSynchronize` API below is only called if a device
   // thread eventually makes progress and sets the flag.
4  // However, CUDA only guarantees that `producer` device thread eventually
   // starts if the synchronization API is called.
5  // Therefore, the host thread may never be unblocked from the flag spin-loop.

```

(continues on next page)

(continued from previous page)

```

6  cuda::atomic<int, cuda::thread_scope_system> flag = 0;
7  __global__ void producer() { flag.store(1); }
8  int main() {
9      cudaHostRegister(&flag, sizeof(flag));
10     producer<<<1,1>>>();
11     while (flag.load() == 0);
12     return cudaDeviceSynchronize();
13 }

```

```

1  // Example: Execution.Model.API.3
2  // Allowed outcome: eventually, no thread makes progress.
3  // Rationale: same as Example.Model.API.2, with the addition that a single CUDA
4  // → query API call does not guarantee
5  // the device thread eventually starts, only repeated CUDA query API calls do
6  // → (see Execution.Model.API.4).
7  cuda::atomic<int, cuda::thread_scope_system> flag = 0;
8  __global__ void producer() { flag.store(1); }
9  int main() {
10     cudaHostRegister(&flag, sizeof(flag));
11     producer<<<1,1>>>();
12     (void)cudaStreamQuery(0);
13     while (flag.load() == 0);
14     return cudaDeviceSynchronize();
15 }

```

```

1  // Example: Execution.Model.API.4
2  // Outcome: terminates.
3  // Rationale: same as Execution.Model.API.3, but this example repeatedly calls
4  // a CUDA query API in within the flag spin-loop, which guarantees that the
5  // → device thread
6  // eventually makes progress.
7  cuda::atomic<int, cuda::thread_scope_system> flag = 0;
8  __global__ void producer() { flag.store(1); }
9  int main() {
10     cudaHostRegister(&flag, sizeof(flag));
11     producer<<<1,1>>>();
12     while (flag.load() == 0) {
13         (void)cudaStreamQuery(0);
14     }
15     return cudaDeviceSynchronize();
16 }

```

5.8.3.1 Dependencies

A device thread shall not start until all its dependencies have completed.

[Note: Dependencies that prevent device threads from starting to make progress can be created, for example, via [CUDA Stream Commands](#). These may include dependencies on the completion of, among others, [CUDA Events](#) and [CUDA Kernels](#). - end note.]

Examples of CUDA API forward progress guarantees due to dependencies

```

1 // Example: Execution.Model.Stream.0
2 // Allowed outcome: eventually, no thread makes progress.
3 // Rationale: while CUDA guarantees that one device thread makes progress,
4   ↳ since there
5 // is no dependency between `first` and `second`, it does not guarantee which
6   ↳ thread,
7 // and therefore it could always pick the device thread from `second`, which
8   ↳ then never
9 // unblocks from the spin-loop.
10 // That is, `second` may starve `first`.
11 cuda::atomic<int, cuda::thread_scope_system> flag = 0;
12 __global__ void first() { flag.store(1, cuda::memory_order_relaxed); }
13 __global__ void second() { while(flag.load(cuda::memory_order_relaxed) == 0)
14   ↳ {} }
15 int main() {
16     cudaHostRegister(&flag, sizeof(flag));
17     cudaStream_t s0, s1;
18     cudaStreamCreate(&s0);
19     cudaStreamCreate(&s1);
20     first<<<1,1,0,s0>>>();
21     second<<<1,1,0,s1>>>();
22     return cudaDeviceSynchronize();
23 }

```

```

1 // Example: Execution.Model.Stream.1
2 // Outcome: terminates.
3 // Rationale: same as Execution.Model.Stream.0, but this example has a stream
4   ↳ dependency
5 // between first and second, which requires CUDA to run the grids in order.
6 cuda::atomic<int, cuda::thread_scope_system> flag = 0;
7 __global__ void first() { flag.store(1, cuda::memory_order_relaxed); }
8 __global__ void second() { while(flag.load(cuda::memory_order_relaxed) == 0)
9   ↳ {} }
10 int main() {
11     cudaHostRegister(&flag, sizeof(flag));
12     cudaStream_t s0;
13     cudaStreamCreate(&s0);
14     first<<<1,1,0,s0>>>();
15     second<<<1,1,0,s0>>>();
16     return cudaDeviceSynchronize();
17 }

```

Chapter 6. Notices

6.1. Notice

This document is provided for information purposes only and shall not be regarded as a warranty of a certain functionality, condition, or quality of a product. NVIDIA Corporation (“NVIDIA”) makes no representations or warranties, expressed or implied, as to the accuracy or completeness of the information contained in this document and assumes no responsibility for any errors contained herein. NVIDIA shall have no liability for the consequences or use of such information or for any infringement of patents or other rights of third parties that may result from its use. This document is not a commitment to develop, release, or deliver any Material (defined below), code, or functionality.

NVIDIA reserves the right to make corrections, modifications, enhancements, improvements, and any other changes to this document, at any time without notice.

Customer should obtain the latest relevant information before placing orders and should verify that such information is current and complete.

NVIDIA products are sold subject to the NVIDIA standard terms and conditions of sale supplied at the time of order acknowledgement, unless otherwise agreed in an individual sales agreement signed by authorized representatives of NVIDIA and customer (“Terms of Sale”). NVIDIA hereby expressly objects to applying any customer general terms and conditions with regards to the purchase of the NVIDIA product referenced in this document. No contractual obligations are formed either directly or indirectly by this document.

NVIDIA products are not designed, authorized, or warranted to be suitable for use in medical, military, aircraft, space, or life support equipment, nor in applications where failure or malfunction of the NVIDIA product can reasonably be expected to result in personal injury, death, or property or environmental damage. NVIDIA accepts no liability for inclusion and/or use of NVIDIA products in such equipment or applications and therefore such inclusion and/or use is at customer’s own risk.

NVIDIA makes no representation or warranty that products based on this document will be suitable for any specified use. Testing of all parameters of each product is not necessarily performed by NVIDIA. It is customer’s sole responsibility to evaluate and determine the applicability of any information contained in this document, ensure the product is suitable and fit for the application planned by customer, and perform the necessary testing for the application in order to avoid a default of the application or the product. Weaknesses in customer’s product designs may affect the quality and reliability of the NVIDIA product and may result in additional or different conditions and/or requirements beyond those contained in this document. NVIDIA accepts no liability related to any default, damage, costs, or problem which may be based on or attributable to: (i) the use of the NVIDIA product in any manner that is contrary to this document or (ii) customer product designs.

No license, either expressed or implied, is granted under any NVIDIA patent right, copyright, or other NVIDIA intellectual property right under this document. Information published by NVIDIA regarding third-party products or services does not constitute a license from NVIDIA to use such products or

services or a warranty or endorsement thereof. Use of such information may require a license from a third party under the patents or other intellectual property rights of the third party, or a license from NVIDIA under the patents or other intellectual property rights of NVIDIA.

Reproduction of information in this document is permissible only if approved in advance by NVIDIA in writing, reproduced without alteration and in full compliance with all applicable export laws and regulations, and accompanied by all associated conditions, limitations, and notices.

THIS DOCUMENT AND ALL NVIDIA DESIGN SPECIFICATIONS, REFERENCE BOARDS, FILES, DRAWINGS, DIAGNOSTICS, LISTS, AND OTHER DOCUMENTS (TOGETHER AND SEPARATELY, "MATERIALS") ARE BEING PROVIDED "AS IS." NVIDIA MAKES NO WARRANTIES, EXPRESSED, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE MATERIALS, AND EXPRESSLY DISCLAIMS ALL IMPLIED WARRANTIES OF NONINFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE. TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL NVIDIA BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF NVIDIA HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. Notwithstanding any damages that customer might incur for any reason whatsoever, NVIDIA's aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms of Sale for the product.

6.2. OpenCL

OpenCL is a trademark of Apple Inc. used under license to the Khronos Group Inc.

6.3. Trademarks

NVIDIA and the NVIDIA logo are trademarks or registered trademarks of NVIDIA Corporation in the U.S. and other countries. Other company and product names may be trademarks of the respective companies with which they are associated.

Copyright

©2007-2026, NVIDIA Corporation & affiliates. All rights reserved